

VayuReader

Backend v2

Complete Technical Documentation

Version: 2.0

Date: January 2026

Classification: RESTRICTED

Organization: Indian Air Force

Department: VayuReader Development Team

This document contains proprietary information

Table of Contents

Part I: Introduction

Documentation Index	6
---------------------	---

Part II: Design Documents

High-Level Design (HLD)	9
Low-Level Design (LLD)	23

Part III: User Guides

User Manual	65
Setup & Installation	89
Configuration	95

Part IV: API Reference

API Endpoints	98
Authentication	125
Data Models	140

Part V: Infrastructure

Nginx Configuration	158
Rate Limiting	174
Caching Strategy	191
Elasticsearch	194

Part VI: Services & Scripts

Services Documentation	199
CLI Scripts	212
Server-Sent Events	231
SSE Integration Guide	234
Audit System	239

Part VII: Operations

Security Hardening	243
Performance Optimization	256
Deployment Guide	262
Testing	265
Contributing Guidelines	268

Part I: Introduction

Documentation Index

Overview

The VayuReader Backend provides a RESTful API for PDF management, dictionary & abbreviation services, authentication, realtime events (SSE), and admin management. It is built with Node.js, Express, MongoDB, Redis, and Docker for production deployment.

Table of Contents

- Setup & Installation
- Configuration
- API Endpoints
- Services Overview
- Scripts & CLI Tools
- ServerSent Events (SSE)
- Performance & Scaling
- Testing & Linting
- Contribution Guidelines

Quick Start (Local Development)

```
# Clone repository
git clone <repo-url>
cd VayuReader_Backend_v2

# Install dependencies
npm install

# Create .env (copy from .env.example)
cp .env.example .env
# Edit .env with your values (MongoDB URI, Redis URL, etc.)

# Start services (MongoDB, Redis, Nginx) via Docker Compose
```

```
docker-compose up -d

# Run the backend
npm run dev # uses nodemon for hotreloading
```

The API will be available at <http://localhost:3000/api/>.

Docker Production Build

```
docker-compose -f docker-compose.prod.yml up --build -d
```

See `docker-compose.prod.yml` for productionspecific settings (TLS, multistage builds, environment variables).

License

MIT © VayuReader Team

Part II: Design Documents

High-Level Design (HLD)

VayuReader Backend v2

Version: 2.0 Date: January 2026 Author: Development Team (IAF)

1. Executive Summary

VayuReader Backend v2 is a secure, scalable, and containerized RESTful API backend designed for the Indian Air Force's document management and reading application. The system provides secure PDF document management, dictionary/abbreviation lookup services, OTP-based authentication, admin panel functionality, and real-time event notifications.

Key Features

- Dual Authentication System: OTP-based authentication for users with device binding; Password + OTP 2FA for administrators
 - PDF Document Management: Upload, categorize, search, and serve PDF documents
 - Dictionary & Abbreviation Services: Word lookup, meanings, synonyms, and abbreviation expansions
 - Real-time Updates: Server-Sent Events (SSE) for instant PDF notifications
 - Comprehensive Audit Trail: User activity and admin action logging
 - Horizontal Scalability: Docker-based deployment with Redis Pub/Sub
-

2. System Architecture

2.1 Architecture Overview





2.2 Technology Stack

Component	Technology	Version	Purpose
Runtime	Node.js	20.x LTS	JavaScript execution environ
Framework	Express.js	4.x	Web application framework
Database	MongoDB	6.x	Document-oriented NoSQL data
Cache	Redis	7.x	In-memory data structure sto
Gateway	Nginx	Alpine	Reverse proxy and load balan
Containerization	Docker	Latest	Container runtime
Orchestration	Docker Compose	Latest	Multi-container deployment
Search (Optional)	Elasticsearch	8.x	Full-text search engine

3. Component Design

3.1 Gateway Layer (Nginx)

Responsibilities:

- SSL/TLS encryption and certificate management
- HTTP to HTTPS redirection
- Static file serving (PDFs, thumbnails)
- API proxy caching for read-heavy endpoints
- Connection-level rate limiting
- Security headers injection
- SSE connection handling (no buffering)
- Load balancing (future multi-instance support)
- Max upload size: 50MB
- PDF caching: 7 days
- API cache TTL: 5 minutes
- SSE timeout: 24 hours
- Keepalive connections: 32

3.2 Application Layer (Node.js/Express)

Core Modules:

Module	Purpose
Routes	URL endpoint definitions and
Controllers	Request handling and respons
Services	Business logic and data proc
Middleware	Cross-cutting concerns (auth
Models	MongoDB schema definitions w
Config	Environment configuration an
Utils	Helper functions and utiliti

3.3 Data Layer

MongoDB Collections:

Collection	Purpose	Key Indexes
users	End-user accounts	phone_number (unique), devic
admins	Administrator accounts	contact (unique), name
pdfdocuments	PDF metadata	title, category, createdAt,
words	Dictionary words	word (unique), text index
abbreviations	Abbreviation expansions	abbreviation (unique), text
useraudits	User activity logs	timestamp, userId, action
auditlogs	Admin action logs	timestamp, adminName, resour

Redis Data Structures:

Key Pattern	Purpose	TTL
otp:	Encrypted OTP storage	5 minutes
rl::	Rate limit counters	Window-based
search::	Search result cache	60 seconds
pdf:categories	Category list cache	1 hour

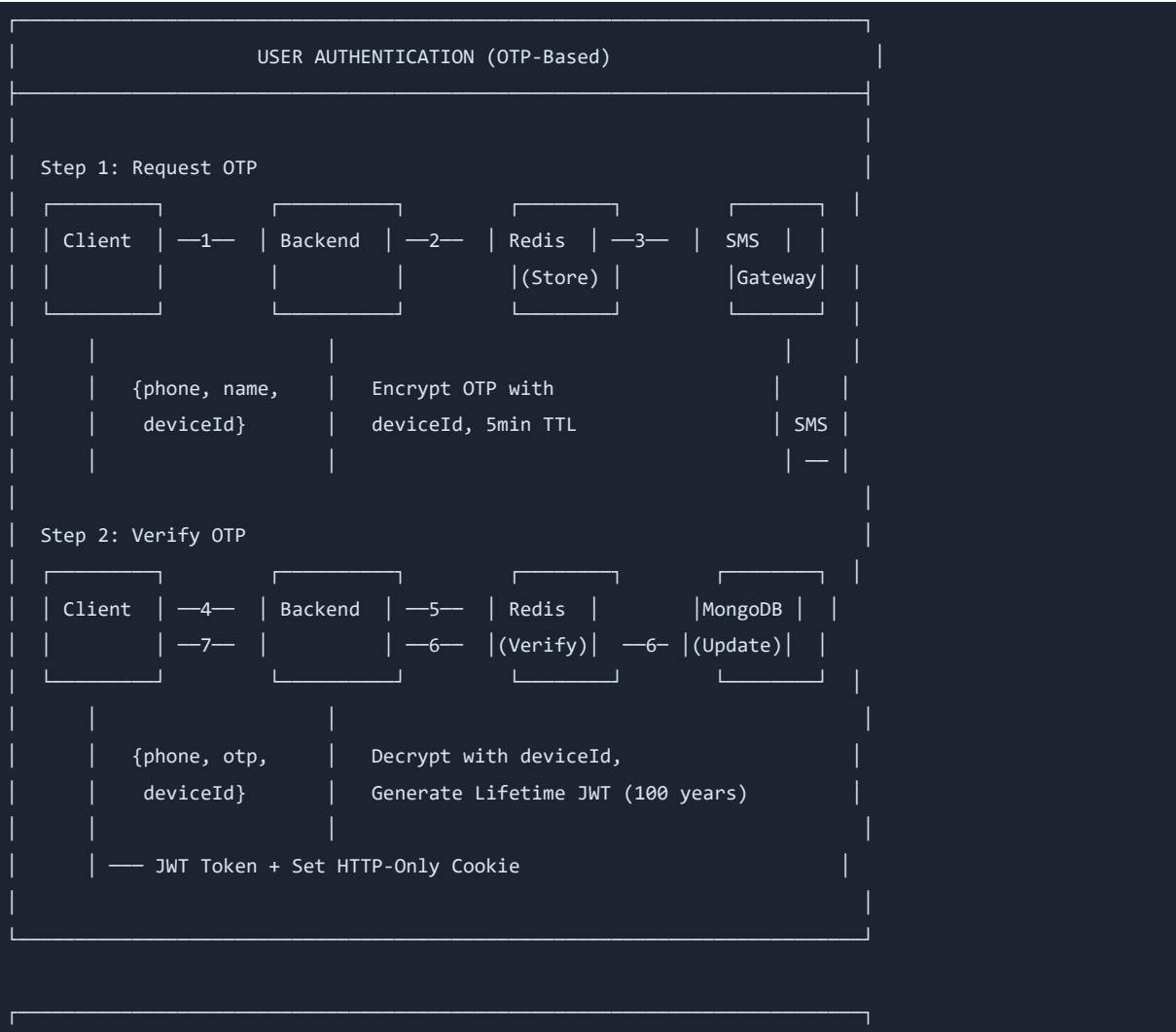
3.4 External Services

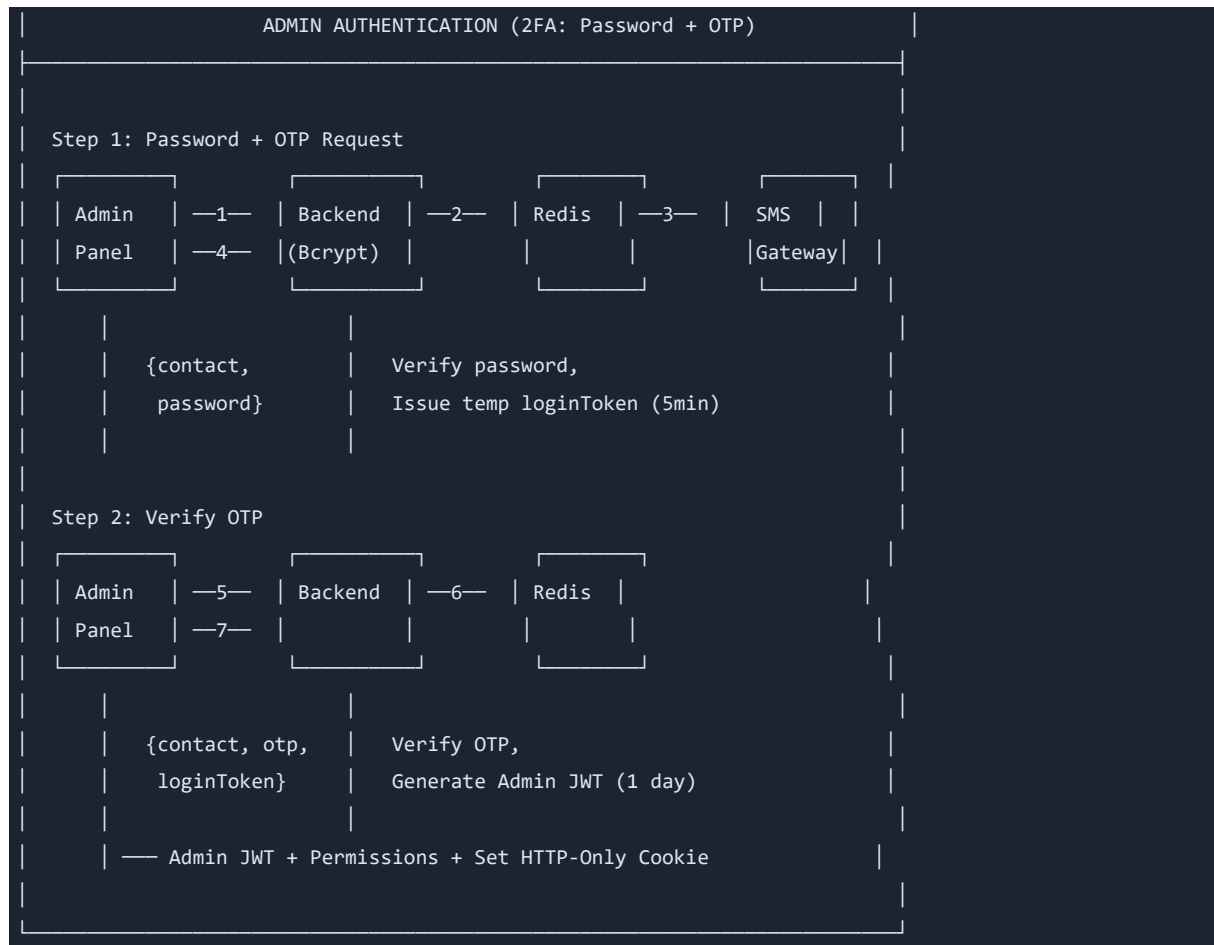
SMS Gateway:

- Purpose: OTP delivery to user phones
- Protocol: HTTP REST API
- Fallback: SMS Simulator for development
- Purpose: Advanced fuzzy search and autocomplete
- Indices: vayuwords, vayuabbreviations
- Sync: Real-time via application, bulk via CLI script

4. Security Architecture

4.1 Authentication Flow





4.2 Security Layers

Layer	Mechanism	Purpose
Transport	HTTPS/TLS 1.2+	Encrypt data in transit
Gateway	Nginx Rate Limiting	Prevent DDoS at connection l
Application	Redis Rate Limiting	Granular per-endpoint rate l
Authentication	JWT + HTTP-Only Cookies	Prevent XSS token theft
Authorization	RBAC (Role-Based Access Cont	Permission enforcement
Validation	express-validator, Mongoose	Input validation and sanitiz
NoSQL Injection	express-mongo-sanitize	Prevent NoSQL injection atta
ReDoS Prevention	Custom regex sanitization	Prevent regex-based DoS
Headers	Helmet.js	Security headers (HSTS, X-Fr
OTP Security	Device-bound AES-256-CBC enc	Prevent OTP interception

4.3 Role-Based Access Control (RBAC)

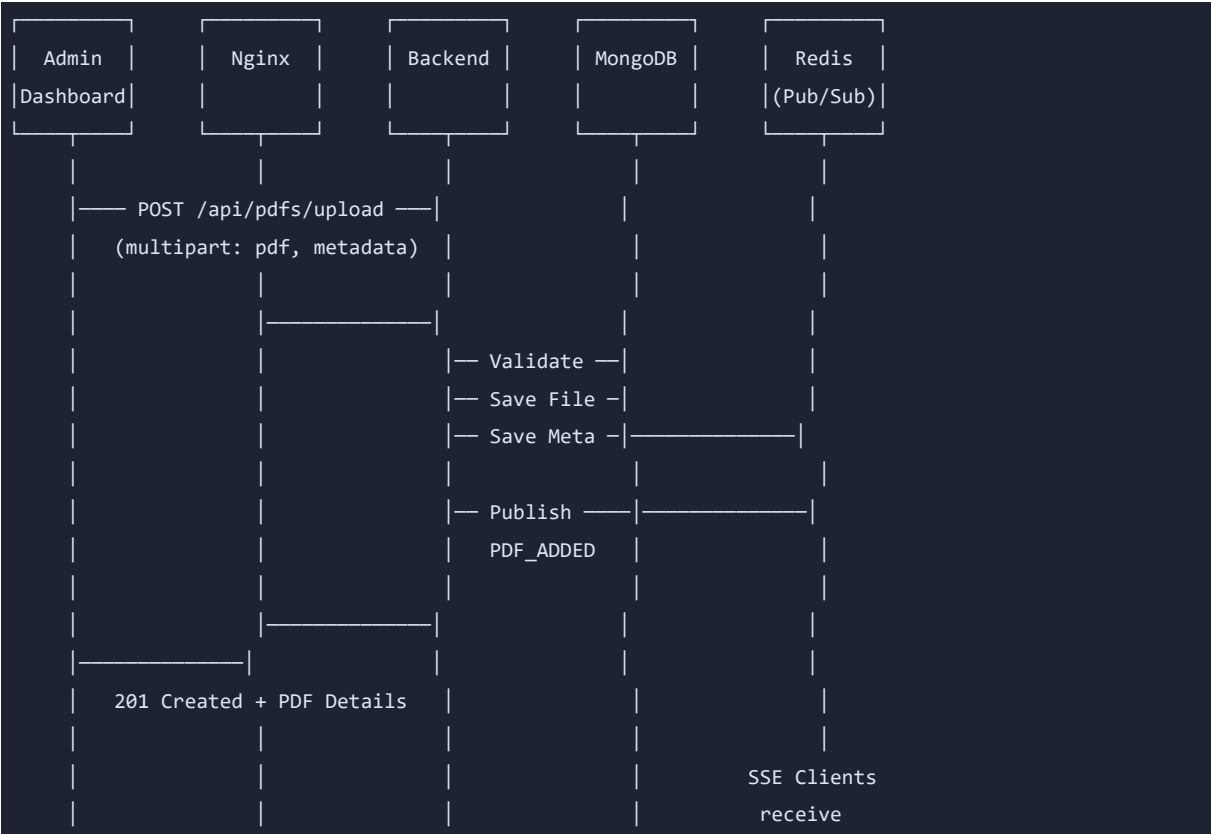
Permission Matrix:

Role	manage_pdfs	manage_dictionary	manage_abbreviations	manage_admins	view_audit
User					
Sub-Admin	*	*	*		*
Super Admin					

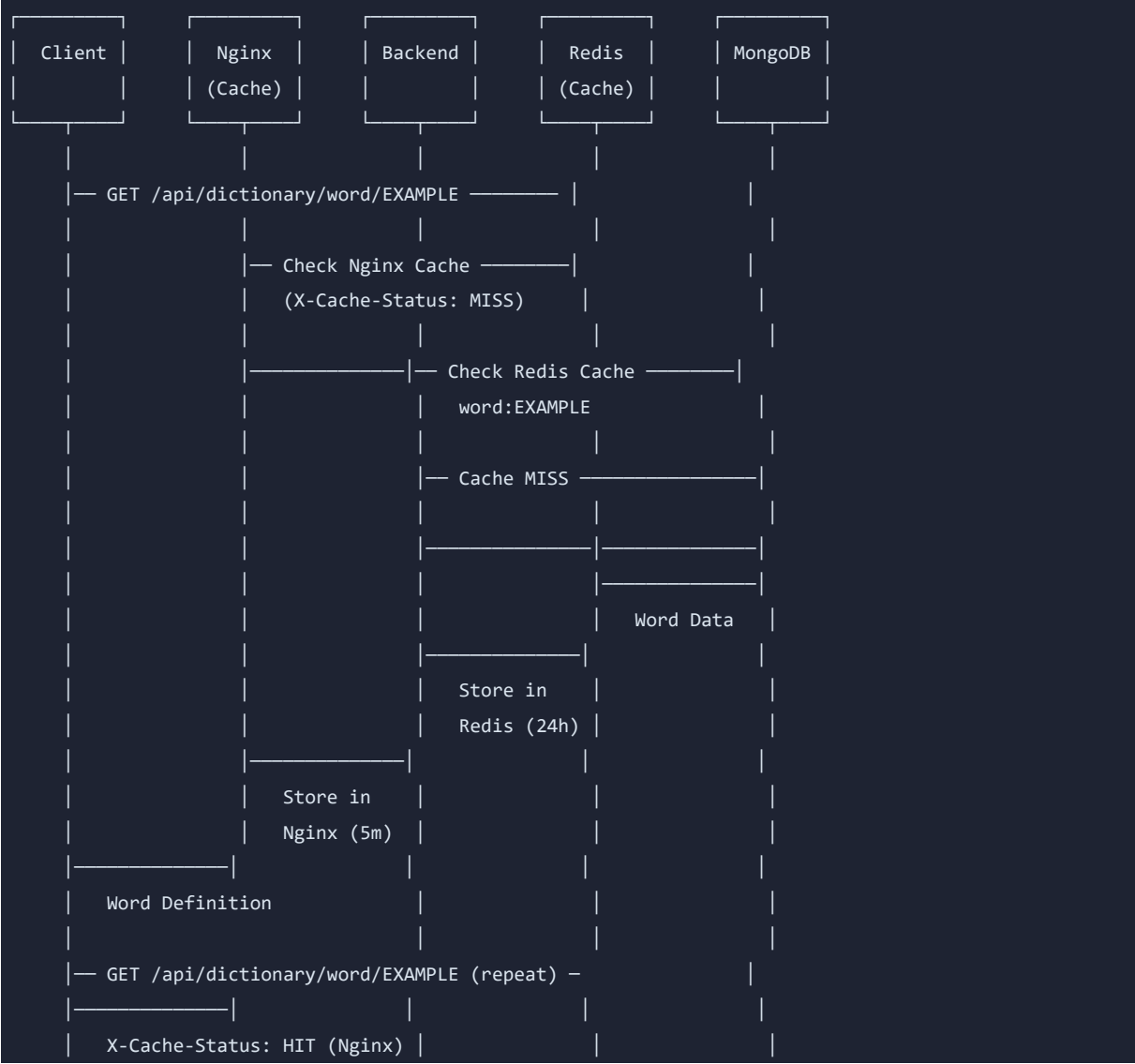
*Configurable per admin

5. Data Flow Diagrams

5.1 PDF Upload Flow

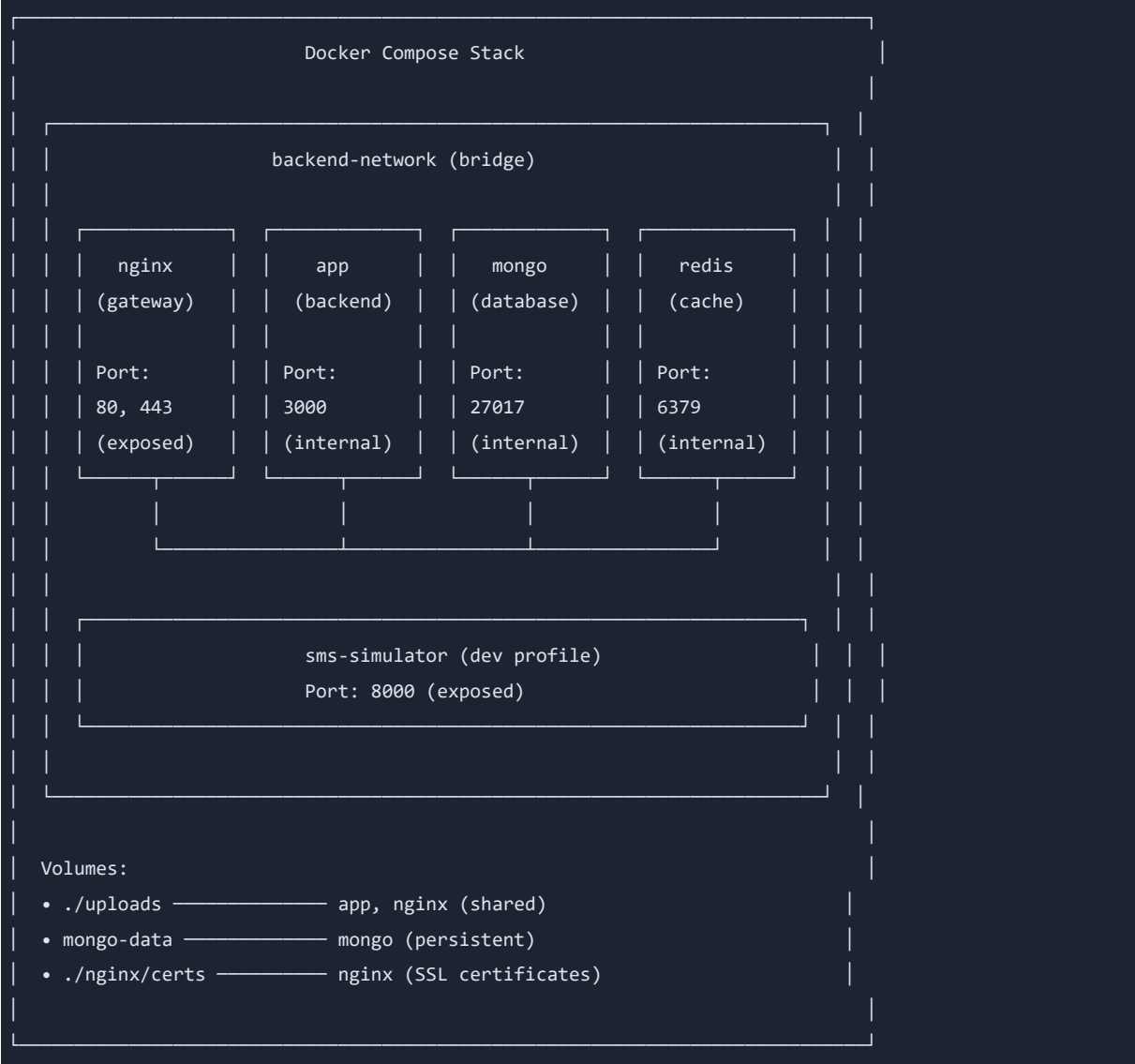


5.2 Dictionary Lookup Flow (Cached)



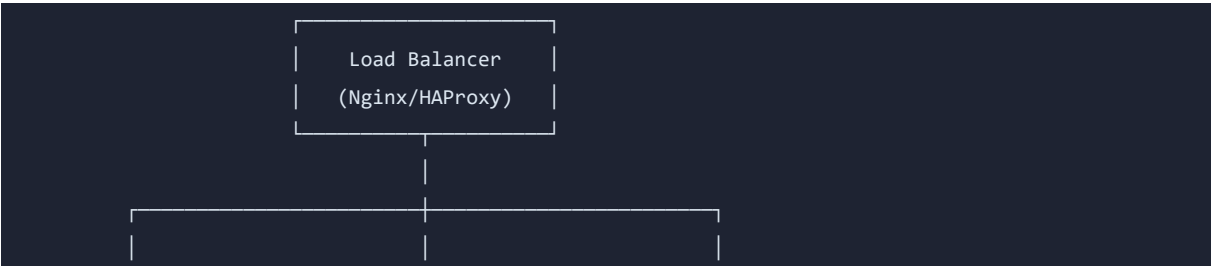
6. Deployment Architecture

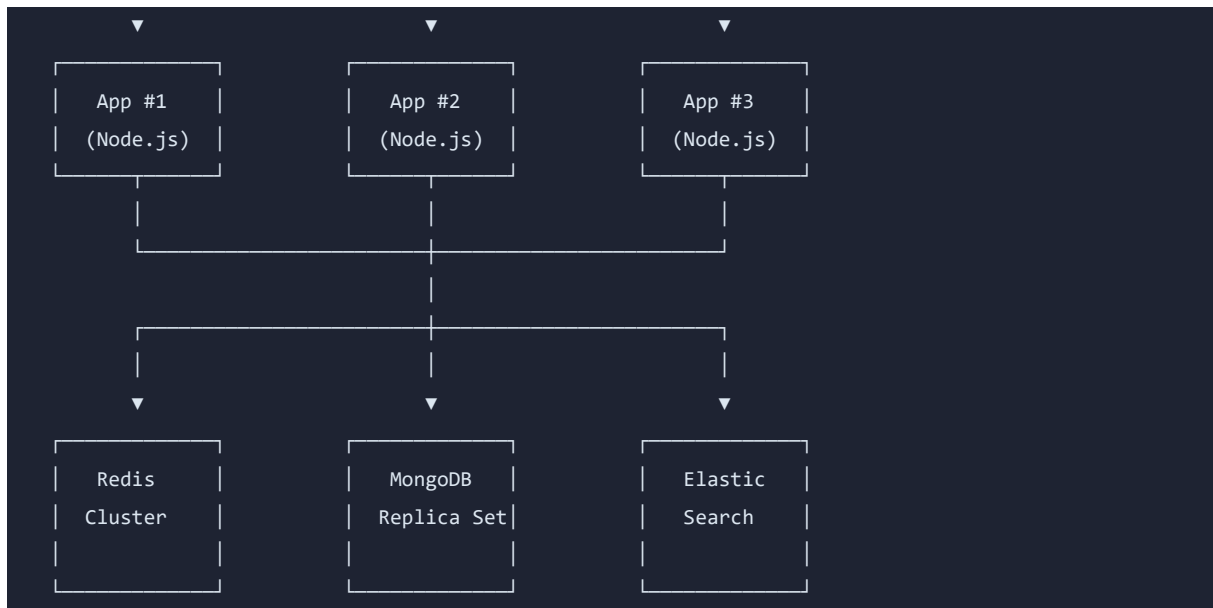
6.1 Docker Container Topology



6.2 Scaling Strategy

Horizontal Scaling (Recommended for 200K+ Users):





7. Non-Functional Requirements

7.1 Performance Requirements

Metric	Target	Measurement
API Response Time (p95)	< 100ms	Cached endpoints
API Response Time (p95)	< 500ms	Database queries
PDF Download Throughput	500+ concurrent	With rate limiting
Cache Hit Rate	> 90%	Redis and Nginx
System Uptime	99.9%	Excluding maintenance

7.2 Scalability Requirements

Metric	Current	Target
Concurrent Users	1,000	200,000+
PDF Storage	10 GB	1 TB+

Dictionary Words	10,000	100,000+
Daily API Requests	100,000	10,000,000+

7.3 Security Requirements

- All data encrypted in transit (TLS 1.2+)
- Passwords hashed with bcrypt (10+ rounds)
- OTPs encrypted with AES-256-CBC
- JWT secrets minimum 32 characters
- Rate limiting on all endpoints
- Regular security audit compliance

8. Integration Points

8.1 Frontend Applications

Application	Type	Integration Method
VayuReader Mobile	React Native	REST API + SSE
Admin Dashboard	React Web	REST API + SSE

8.2 External Services

Service	Purpose	Protocol
SMS Gateway	OTP delivery	HTTPS REST

9. Disaster Recovery

9.1 Backup Strategy

Data	Frequency	Retention	Method
MongoDB	Daily	30 days	mongodump + cloud storage
Uploads (PDFs)	Daily	30 days	File sync to backup storage
Redis	Not critical	N/A	Reconstructable from DB
Configurations	On change	Indefinite	Git version control

9.2 Recovery Procedures

- Database Recovery: Restore from latest mongodump
- File Recovery: Sync from backup storage
- Configuration: Deploy from version control
- Cache Warming: Automatic on first requests

10. Monitoring & Observability

10.1 Health Endpoints

Endpoint	Purpose
GET /health	Basic health check (HTTP 200)
GET /health/ready	Readiness probe (DB + Redis)
GET /nginx_status	Nginx connection statistics

10.2 Key Metrics to Monitor

- API response times (p50, p95, p99)
- Error rates by endpoint
- Redis hit/miss ratio
- MongoDB query times
- Active SSE connections

- Rate limit violations
- Memory and CPU usage

11. Glossary

Term	Definition
SSE	Server-Sent Events - HTTP-ba
JWT	JSON Web Token - Secure toke
OTP	One-Time Password - Temporar
2FA	Two-Factor Authentication -
RBAC	Role-Based Access Control -
TTL	Time-To-Live - Expiration ti
CDN	Content Delivery Network - D
HSTS	HTTP Strict Transport Securi

Document Revision History

Version	Date	Author	Changes
1.0	Jan 2025	Dev Team	Initial HLD
2.0	Jan 2026	Dev Team	Updated with SSE, enhanced s

Low-Level Design (LLD)

VayuReader Backend v2

Version: 2.0 Date: January 2026 Author: Development Team (IAF)

1. Introduction

This Low-Level Design (LLD) document provides detailed technical specifications for the VayuReader Backend v2 system. It covers class/module designs, database schemas, API contracts, algorithm implementations, and code-level architecture.

2. Project Structure

```
VayuReader_Backend_v2/
├─ src/
│   ├─ config/                # Configuration modules
│   │   ├─ environment.js     # Environment variable loader
│   │   ├─ database.js        # MongoDB connection
│   │   ├─ redis.js           # Redis client setup
│   │   ├─ cors.js            # CORS configuration
│   │   ├─ apiLimit-config.js  # Rate limiting configuration
│   │   └─ elasticsearch.js    # Elasticsearch client (optional)
│   └─ controllers/           # Request handlers
│       ├─ auth.controller.js  # User authentication
│       ├─ admin.controller.js # Admin authentication & management
│       └─ pdf.controller.js   # PDF CRUD operations
```

```

| | | dictionary.controller.js # Dictionary operations
| | | abbreviation.controller.js # Abbreviation operations
| | | audit.controller.js # Audit log queries
| | | └ sse.controller.js # Server-Sent Events
| | |
| | └ middleware/ # Express middleware
| | | auth.js # User JWT authentication
| | | adminAuth.js # Admin JWT authentication
| | | rateLimiter.js # Rate limiting middleware
| | | validation.js # Input validation
| | | errorHandler.js # Global error handler
| | | └ upload.js # File upload (Multer)
| | |
| | └ models/ # Mongoose schemas
| | | User.js # End-user model
| | | Admin.js # Administrator model
| | | PdfDocument.js # PDF metadata model
| | | Word.js # Dictionary word model
| | | Abbreviation.js # Abbreviation model
| | | UserAudit.js # User activity log
| | | └ AuditLog.js # Admin action log
| | |
| | └ routes/ # Route definitions
| | | index.js # Route aggregator
| | | auth.routes.js # /api/auth/*
| | | admin.routes.js # /api/admin/*
| | | pdf.routes.js # /api/pdfs/*
| | | dictionary.routes.js # /api/dictionary/*
| | | abbreviation.routes.js # /api/abbreviations/*
| | | audit.routes.js # /api/audit/*
| | | └ sse.routes.js # /api/events
| | |
| | └ services/ # Business logic
| | | jwt.service.js # JWT generation/verification
| | | otp.service.js # OTP generation/encryption/storage
| | | sms.service.js # SMS gateway integration
| | | search.service.js # Search with caching
| | | pdf.service.js # PDF file operations
| | | userAudit.service.js # User activity logging
| | | pubsub.service.js # Redis Pub/Sub
| | | └ sse.service.js # SSE connection management
| | |
| | └ utils/ # Utility functions
| | | response.js # Standardized API responses
| | | sanitize.js # Input sanitization
| | | └ fileValidation.js # File type validation
| | |

```



```
|   ├── server.js           # Express app entry point
|   └── cluster.js         # Cluster mode for multi-core
|
|── scripts/                # CLI utilities
|   ├── seedAdmin.js       # Create super admin
|   ├── manageUsers.js     # User/admin management
|   ├── reset-admin-password.js # Password reset
|   ├── optimizeDb.js      # Database optimization
|   ├── syncElasticsearch.js # ES data sync
|   ├── cleanupLogs.js     # Log retention
|   └── sms-simulator.js   # SMS gateway mock
|
|── nginx/                  # Nginx configuration
|   ├── conf.d/default.conf # Server configuration
|   ├── nginx.conf          # Main nginx.conf
|   └── certs/               # SSL certificates
|
|── docs/                   # Documentation
|── test/                   # Test files
|── uploads/                # PDF/Image storage
|── docker-compose.yml      # Docker orchestration
|── Dockerfile              # Container image
|── package.json            # Dependencies
|── .env.example            # Environment template
```

3. Database Schema Design

3.1 User Schema

Collection: users

```
const UserSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Name is required'],
    trim: true,
    minlength: [2, 'Name must be at least 2 characters'],
    maxlength: [100, 'Name cannot exceed 100 characters']
  },
  email: {
    type: String,
    required: [true, 'Email is required'],
    trim: true,
    lowercase: true,
    unique: true,
    match: [/^((?=[a-z\d-!#$%&'*+.^_`{|}~]+)@(?:[a-z\d-!#$%&'*+.^_`{|}~]+)\.?[a-z\d-!#$%&'*+.^_`{|}~]+)$/i],
    validate: {
      validator: function(v) {
        return /^[a-z\d-!#$%&'*+.^_`{|}~]+@((?=[a-z\d-!#$%&'*+.^_`{|}~]+)\.?[a-z\d-!#$%&'*+.^_`{|}~]+)$/i.test(v);
      },
      message: 'Invalid email address'
    }
  },
  password: {
    type: String,
    required: [true, 'Password is required'],
    trim: true,
    minlength: [8, 'Password must be at least 8 characters'],
    validate: {
      validator: function(v) {
        return /^(?=.*[a-z])(?=.*[A-Z])(?=.*[0-9])(?=.*[!@#%^&*~`{|}~])(?=.{8,})/.test(v);
      },
      message: 'Password must contain at least one uppercase letter, one lowercase letter, one digit, and one special character'
    }
  },
  phone: {
    type: String,
    required: [false, 'Phone is optional'],
    trim: true,
    match: [/^(\+)?(\d){3,4}([ ]?)?(\d){3,4}([ ]?)?(\d){4,5}$/],
    validate: {
      validator: function(v) {
        return /^(\+)?(\d){3,4}([ ]?)?(\d){3,4}([ ]?)?(\d){4,5}$/.test(v);
      },
      message: 'Invalid phone number'
    }
  },
  role: {
    type: String,
    required: [true, 'Role is required'],
    trim: true,
    enum: ['user', 'admin', 'super-admin']
  },
  status: {
    type: String,
    required: [true, 'Status is required'],
    trim: true,
    enum: ['active', 'inactive', 'suspended']
  },
  createdAt: {
    type: Date,
    required: [true, 'Creation date is required'],
    default: Date.now()
  },
  updatedAt: {
    type: Date,
    required: [true, 'Update date is required'],
    default: Date.now()
  }
});
```

```
phone_number: {
  type: String,
  required: [true, 'Phone number is required'],
  unique: true,
  index: true,
  validate: {
    validator: (v) => /^\\+?[1-9]\\d{1,14}$/.test(v),
    message: 'Invalid phone number format'
  }
},
deviceId: {
  type: String,
  index: true,
  default: null
},
previousDeviceId: {
  type: String,
  default: null
},
lastLogin: {
  type: Date,
  default: null
},
isBlocked: {
  type: Boolean,
  default: false
}
}, {
  timestamps: true
});

// Indexes
UserSchema.index({ phone_number: 1 }, { unique: true });
UserSchema.index({ deviceId: 1 });

// Instance Methods
UserSchema.methods.toSafeObject = function() {
  return {
    id: this._id,
    name: this.name,
    phone_number: this.phone_number
  };
};
```

3.2 Admin Schema

Collection: admins

```
const AdminSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true,
    trim: true,
    minlength: 2,
    maxlength: 100
  },
  contact: {
    type: String,
    required: true,
    unique: true,
    index: true
  },
  passwordHash: {
    type: String,
    required: true,
    select: false // Exclude from queries by default
  },
  isSuperAdmin: {
    type: Boolean,
    default: false
  },
  permissions: [{
    type: String,
    enum: [
      'manage_pdfs',
      'manage_dictionary',
      'manage_abbreviations',
      'manage_admins',
      'view_audit'
    ]
  }],
  createdBy: {
    type: String,
    default: 'System'
  }
}, {
  timestamps: true
});

// Instance Methods
AdminSchema.methods.hasPermission = function(permission) {
  if (this.isSuperAdmin) return true;
```

```
    return this.permissions.includes(permission);
  };

AdminSchema.methods.toSafeObject = function() {
  return {
    id: this._id,
    name: this.name,
    contact: this.contact,
    isSuperAdmin: this.isSuperAdmin,
    permissions: this.isSuperAdmin ? PERMISSIONS : this.permissions
  };
};
```

3.3 PdfDocument Schema

Collection: pdfdocuments

```
const PdfDocumentSchema = new mongoose.Schema({
  title: {
    type: String,
    required: true,
    trim: true,
    minlength: 1,
    maxlength: 200,
    index: true
  },
  description: {
    type: String,
    trim: true,
    maxlength: 1000
  },
  category: {
    type: String,
    required: true,
    trim: true,
    index: true
  },
  pdfUrl: {
    type: String,
    required: true
  },
  thumbnailUrl: {
    type: String,
    default: null
  },
});
```

```
    uploadedBy: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'Admin',
      required: true,
      index: true
    },
    views: {
      type: Number,
      default: 0,
      min: 0
    }
  }, {
    timestamps: true
  });

// Text index for full-text search
PdfDocumentSchema.index(
  { title: 'text', description: 'text', category: 'text' },
  { weights: { title: 10, category: 5, description: 1 } }
);

// Compound indexes
PdfDocumentSchema.index({ category: 1, createdAt: -1 });
PdfDocumentSchema.index({ createdAt: -1 });
```

3.4 Word Schema

Collection: words

```
const MeaningSchema = new mongoose.Schema({
  partOfSpeech: {
    type: String,
    enum: ['noun', 'verb', 'adjective', 'adverb', 'pronoun',
      'preposition', 'conjunction', 'interjection', 'other']
  },
  definition: {
    type: String,
    required: true
  },
  synonyms: [String],
  examples: [String]
}, { _id: false });

const WordSchema = new mongoose.Schema({
  word: {
```

```
    type: String,
    required: true,
    unique: true,
    uppercase: true,
    trim: true,
    index: true
  },
  meanings: {
    type: [MeaningSchema],
    required: true,
    validate: {
      validator: (v) => v && v.length > 0,
      message: 'At least one meaning is required'
    }
  },
  synonyms: [String],
  antonyms: [String]
}, {
  timestamps: true
});

// Text index for search
WordSchema.index({ word: 'text' });
```

3.5 Abbreviation Schema

Collection: abbreviations

```
const AbbreviationSchema = new mongoose.Schema({
  abbreviation: {
    type: String,
    required: true,
    unique: true,
    uppercase: true,
    trim: true,
    index: true
  },
  fullForm: {
    type: String,
    required: true,
    trim: true
  }
}, {
  timestamps: true
});
```

```
// Text index for search
AbbreviationSchema.index(
  { abbreviation: 'text', fullForm: 'text' },
  { weights: { abbreviation: 10, fullForm: 1 } }
);
```

3.6 UserAudit Schema

Collection: useraudits

```
const UserAuditSchema = new mongoose.Schema({
  userId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true,
    index: true
  },
  phone_number: {
    type: String,
    required: true,
    index: true
  },
  action: {
    type: String,
    required: true,
    enum: ['LOGIN', 'DEVICE_CHANGE', 'READ_PDF'],
    index: true
  },
  deviceId: {
    type: String,
    index: true
  },
  metadata: {
    type: mongoose.Schema.Types.Mixed
  },
  timestamp: {
    type: Date,
    default: Date.now,
    index: true
  }
}, {
  timestamps: true
});
```

```
// Compound indexes for common queries
UserAuditSchema.index({ userId: 1, action: 1 });
UserAuditSchema.index({ phone_number: 1, timestamp: -1 });
UserAuditSchema.index({ action: 1, timestamp: -1 });
UserAuditSchema.index({ timestamp: -1 });
```

3.7 AuditLog Schema

Collection: auditlogs

```
const AuditLogSchema = new mongoose.Schema({
  action: {
    type: String,
    required: true,
    enum: ['CREATE', 'UPDATE', 'DELETE'],
    index: true
  },
  resourceType: {
    type: String,
    required: true,
    enum: ['PDF', 'DICTIONARY_WORD', 'ABBREVIATION', 'ADMIN'],
    index: true
  },
  resourceId: {
    type: String,
    required: true,
    index: true
  },
  adminId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Admin'
  },
  adminName: {
    type: String,
    required: true,
    index: true
  },
  adminContact: {
    type: String,
    required: true
  },
  details: {
    type: mongoose.Schema.Types.Mixed
  },
  timestamp: {
```



```
    type: Date,
    default: Date.now,
    index: true
  }, {
    timestamps: true
  });

// Compound indexes
AuditLogSchema.index({ resourceType: 1, resourceId: 1 });
AuditLogSchema.index({ adminName: 1, timestamp: -1 });
AuditLogSchema.index({ action: 1, resourceType: 1 });
AuditLogSchema.index({ timestamp: -1 });
```

4. Service Layer Design

4.1 JWT Service

File: src/services/jwt.service.js

```
/**
 * JWT Service - Token generation and verification
 */

const jwt = require('jsonwebtoken');
const { jwt: jwtConfig } = require('../config/environment');
const PERMISSIONS = require('../constants/permissions');

/**
 * Generate lifetime user token (100 years)
 * @param {ObjectId} userId - User's MongoDB ID
 * @param {Object} payload - Additional token data
 * @returns {String} JWT token
 */
const generateLifetimeUserToken = (userId, payload) => {
  return jwt.sign(
    {
      userId,
      ...payload,
    },
    jwtConfig.secretKey,
```

```
        type: 'user'
      },
      jwtConfig.secret,
      { expiresIn: '100y' }
    );
  };

/**
 * Generate admin session token (1 day)
 * @param {Object} admin - Admin document
 * @returns {String} JWT token
 */
const generateAdminToken = (admin) => {
  return jwt.sign(
    {
      adminId: admin._id,
      contact: admin.contact,
      name: admin.name,
      isSuperAdmin: admin.isSuperAdmin,
      permissions: admin.isSuperAdmin ? PERMISSIONS : admin.permissions,
      type: 'admin'
    },
    jwtConfig.secret,
    { expiresIn: '1d' }
  );
};

/**
 * Generate temporary login token for 2FA flow (5 minutes)
 * @param {String} contact - Admin contact
 * @returns {String} JWT token
 */
const generateLoginToken = (contact) => {
  return jwt.sign(
    { contact, type: 'admin_login' },
    jwtConfig.secret,
    { expiresIn: '5m' }
  );
};

/**
 * Verify and decode JWT token
 * @param {String} token - JWT token string
 * @returns {Object} Decoded payload
 * @throws {Error} If token is invalid or expired
 */
const verifyToken = (token) => {
```

```
    return jwt.verify(token, jwtConfig.secret);
  }
};

module.exports = {
  generateLifetimeUserToken,
  generateAdminToken,
  generateLoginToken,
  verifyToken
};
```

4.2 OTP Service

File: src/services/otp.service.js

```
/**
 * OTP Service - Generation, encryption, storage, verification
 */

const crypto = require('crypto');
const { getClient: getRedisClient } = require('../config/redis');
const { otp: otpConfig } = require('../config/environment');

const OTP_TTL = otpConfig.expiryMinutes * 60; // Convert to seconds
const SALT = 'vayureader_otp_salt_v2'; // Fixed salt for key derivation

/**
 * Generate 6-digit random OTP
 * @returns {String} 6-digit OTP
 */
const generateOtp = () => {
  return Math.floor(100000 + Math.random() * 900000).toString();
};

/**
 * Encrypt OTP with device ID using AES-256-CBC
 * @param {String} otp - Plain OTP
 * @param {String} deviceId - Device identifier for key derivation
 * @returns {String} Encrypted OTP (iv:ciphertext)
 */
const encryptOtp = (otp, deviceId) => {
  // Derive key from deviceId using PBKDF2
  const key = crypto.pbkdf2Sync(deviceId, SALT, 100000, 32, 'sha256');
  const iv = crypto.randomBytes(16);

  const cipher = crypto.createCipheriv('aes-256-cbc', key, iv);
```

```

    let encrypted = cipher.update(otp, 'utf8', 'hex');
    encrypted += cipher.final('hex');

    return iv.toString('hex') + ':' + encrypted;
  };

  /**
   * Decrypt OTP using device ID
   * @param {String} encryptedData - Encrypted OTP (iv:ciphertext)
   * @param {String} deviceId - Device identifier
   * @returns {String} Decrypted OTP
   * @throws {Error} If decryption fails (wrong deviceId)
   */
  const decryptOtp = (encryptedData, deviceId) => {
    const [ivHex, encrypted] = encryptedData.split(':');
    const key = crypto.pbkdf2Sync(deviceId, SALT, 100000, 32, 'sha256');
    const iv = Buffer.from(ivHex, 'hex');

    const decipher = crypto.createDecipheriv('aes-256-cbc', key, iv);
    let decrypted = decipher.update(encrypted, 'hex', 'utf8');
    decrypted += decipher.final('utf8');

    return decrypted;
  };

  /**
   * Save OTP to Redis with encryption
   * @param {String} phone_number - User's phone number
   * @param {String} otp - Plain OTP
   * @param {String} deviceId - Device identifier (or 'admin' for admin OTPs)
   */
  const saveOtp = async (phone_number, otp, deviceId) => {
    const redisClient = getRedisClient();
    const encrypted = encryptOtp(otp, deviceId);
    await redisClient.setEx(`otp:${phone_number}`, OTP_TTL, encrypted);
  };

  /**
   * Verify OTP from Redis
   * @param {String} otp - Submitted OTP
   * @param {String} phone_number - User's phone number
   * @param {String} deviceId - Device identifier
   * @returns {Object} { valid: boolean, error?: string }
   */
  const verifyOtp = async (otp, phone_number, deviceId) => {
    const redisClient = getRedisClient();
    const encrypted = await redisClient.get(`otp:${phone_number}`);

```

```
    if (!encrypted) {
      return { valid: false, error: 'OTP expired or not found' };
    }

    try {
      const decrypted = decryptOtp(encrypted, deviceId);
      if (decrypted !== otp) {
        return { valid: false, error: 'Invalid OTP' };
      }
      return { valid: true };
    } catch (error) {
      // Decryption failed - wrong deviceId
      return { valid: false, error: 'Invalid OTP or device mismatch' };
    }
  };

/**
 * Delete OTP from Redis after successful verification
 * @param {String} phone_number - User's phone number
 */
const deleteOtp = async (phone_number) => {
  const redisClient = getRedisClient();
  await redisClient.del(`otp:${phone_number}`);
};

/**
 * Check if OTP sending should be skipped (dev mode)
 * @returns {Boolean}
 */
const shouldSkipSend = () => {
  return otpConfig.skipSend === true;
};

module.exports = {
  generateOtp,
  saveOtp,
  verifyOtp,
  deleteOtp,
  shouldSkipSend
};
```

4.3 Search Service

File: src/services/search.service.js

```
/**
 * Search Service - Dictionary and Abbreviation search with caching
 */

const { getClient: getRedisClient } = require('../config/redis');
const Word = require('../models/Word');
const Abbreviation = require('../models/Abbreviation');

const CACHE_TTL = 60; // 60 seconds

/**
 * Search dictionary words with caching
 * @param {String} term - Search term
 * @param {Object} options - { page, limit }
 * @returns {Object} { results, pagination }
 */
const searchDictionary = async (term, options = {}) => {
  const { page = 1, limit = 20 } = options;
  const skip = (page - 1) * limit;
  const cacheKey = `word:search:${term.toLowerCase()}:${page}:${limit}`;

  // Check cache
  const redisClient = getRedisClient();
  const cached = await redisClient.get(cacheKey);
  if (cached) {
    return JSON.parse(cached);
  }

  // Build query
  let query = {};
  if (term) {
    query = {
      $or: [
        { word: new RegExp(`^${escapeRegex(term)}`, 'i') },
        { $text: { $search: term } }
      ]
    };
  }

  // Execute query
  const [results, total] = await Promise.all([
    Word.find(query)
      .skip(skip)
      .limit(limit)
      .sort({ word: 1 })
      .lean(),
    Word.countDocuments(query)
  ])
```

```
]);

const response = {
  results,
  pagination: {
    currentPage: page,
    totalPages: Math.ceil(total / limit),
    totalItems: total,
    itemsPerPage: limit
  }
};

// Cache results
await redisClient.setEx(cacheKey, CACHE_TTL, JSON.stringify(response));

return response;
};

/**
 * Get single word by exact match
 * @param {String} word - Word to lookup
 * @returns {Object|null} Word document
 */
const getWord = async (word) => {
  const cacheKey = `word:${word.toUpperCase()}`;

  const redisClient = getRedisClient();
  const cached = await redisClient.get(cacheKey);
  if (cached) {
    return JSON.parse(cached);
  }

  const result = await Word.findOne({ word: word.toUpperCase() }).lean();

  if (result) {
    await redisClient.setEx(cacheKey, CACHE_TTL * 60, JSON.stringify(result));
  }
  // 1 hour
}

return result;
};

/**
 * Search abbreviations with caching
 * @param {String} term - Search term
 * @param {Object} options - { page, limit }
 * @returns {Object} { results, pagination }

```

```
*/
const searchAbbreviation = async (term, options = {}) => {
  const { page = 1, limit = 20 } = options;
  const skip = (page - 1) * limit;
  const cacheKey = `abbr:search:${term.toLowerCase()}:${page}:${limit}`;

  const redisClient = getRedisClient();
  const cached = await redisClient.get(cacheKey);
  if (cached) {
    return JSON.parse(cached);
  }

  let query = {};
  if (term) {
    query = {
      $or: [
        { abbreviation: new RegExp(`^${escapeRegex(term)}`, 'i') },
        { $text: { $search: term } }
      ]
    };
  }

  const [results, total] = await Promise.all([
    Abbreviation.find(query)
      .skip(skip)
      .limit(limit)
      .sort({ abbreviation: 1 })
      .lean(),
    Abbreviation.countDocuments(query)
  ]);

  const response = {
    results,
    pagination: {
      currentPage: page,
      totalPages: Math.ceil(total / limit),
      totalItems: total,
      itemsPerPage: limit
    }
  };

  await redisClient.setEx(cacheKey, CACHE_TTL, JSON.stringify(response));

  return response;
};

/**
```



```
* Invalidate cache for a specific word
* @param {String} word - Word to invalidate
*/
const invalidateWordCache = async (word) => {
  const redisClient = getRedisClient();
  await redisClient.del(`word:${word.toUpperCase()}`);
  // Also invalidate search caches (pattern matching)
  const keys = await redisClient.keys('word:search:*');
  if (keys.length > 0) {
    await redisClient.del(keys);
  }
};

/**
 * Escape special regex characters
 * @param {String} string - Input string
 * @returns {String} Escaped string
 */
const escapeRegex = (string) => {
  return string.replace(/[\.*+?^${}()|[\]\\"/g, '\\$&');
};

module.exports = {
  searchDictionary,
  searchAbbreviation,
  getWord,
  invalidateWordCache
};
```

4.4 Pub/Sub Service

File: src/services/pubsub.service.js

```
/**
 * Redis Pub/Sub Service for real-time event distribution
 */

const { createClient } = require('redis');
const { redis: redisConfig } = require('../config/environment');

let publisher = null;
let subscriber = null;
const subscriptions = new Map();

/**
```

```
* Initialize Pub/Sub clients
*/
const initialize = async () => {
  // Publisher client (reuse main client)
  publisher = createClient({ url: redisConfig.url });
  await publisher.connect();

  // Dedicated subscriber client (required by Redis protocol)
  subscriber = createClient({ url: redisConfig.url });
  await subscriber.connect();

  console.log('Redis Pub/Sub initialized');
};

/**
 * Publish event to channel
 * @param {String} channel - Channel name
 * @param {Object} data - Event data
 */
const publishEvent = async (channel, data) => {
  if (!publisher || !publisher.isOpen) {
    console.warn('Publisher not ready, event dropped:', channel);
    return;
  }

  const message = JSON.stringify({
    ...data,
    timestamp: new Date().toISOString()
  });

  await publisher.publish(channel, message);
};

/**
 * Subscribe to channel
 * @param {String} channel - Channel name
 * @param {Function} callback - Handler function(message)
 */
const subscribe = async (channel, callback) => {
  if (!subscriber || !subscriber.isOpen) {
    throw new Error('Subscriber not ready');
  }

  await subscriber.subscribe(channel, (message) => {
    try {
      callback(message);
    } catch (error) {

```

```
        console.error('Subscription callback error:', error);
    }
});

subscriptions.set(channel, callback);
};

/**
 * Unsubscribe from channel
 * @param {String} channel - Channel name
 */
const unsubscribe = async (channel) => {
    if (subscriber && subscriber.isOpen) {
        await subscriber.unsubscribe(channel);
    }
    subscriptions.delete(channel);
};

/**
 * Cleanup connections
 */
const shutdown = async () => {
    if (publisher) await publisher.quit();
    if (subscriber) await subscriber.quit();
};

module.exports = {
    initialize,
    publishEvent,
    subscribe,
    unsubscribe,
    shutdown
};
```

5. Middleware Design

5.1 Authentication Middleware

File: src/middleware/auth.js

```
/**
 * User Authentication Middleware
 */

const { verifyToken } = require('../services/jwt.service');
const User = require('../models/User');
const response = require('../utils/response');

/**
 * Authenticate user requests
 * - Extracts token from cookie or Authorization header
 * - Verifies JWT signature
 * - Checks user exists and is not blocked
 * - Attaches user info to req.user
 */
const authenticateUser = async (req, res, next) => {
  try {
    // Extract token
    const token = req.cookies?.auth_token ||
      req.headers.authorization?.replace('Bearer ', '');

    if (!token) {
      return response.unauthorized(res, 'No token provided');
    }

    // Verify token
    let decoded;
    try {
      decoded = verifyToken(token);
    } catch (error) {
      return response.unauthorized(res, 'Invalid or expired token');
    }

    // Check token type
    if (decoded.type !== 'user') {
      return response.unauthorized(res, 'Invalid token type');
    }

    // Verify user exists and is not blocked
    const user = await User.findById(decoded.userId).select('isBlocked').lean();

    if (!user) {
      return response.unauthorized(res, 'User not found');
    }

    if (user.isBlocked) {
      return response.unauthorized(res, 'User account is blocked');
    }
  }
}
```

```
}

// Attach user info to request
req.user = {
  userId: decoded.userId,
  phone_number: decoded.phone_number,
  deviceId: decoded.deviceId,
  name: decoded.name
};

next();
} catch (error) {
  console.error('Authentication error:', error);
  return response.serverError(res, 'Authentication failed');
}
};

module.exports = { authenticateUser };
```

5.2 Rate Limiter Middleware

File: src/middleware/rateLimiter.js

```
/**
 * Rate Limiting Middleware with Redis storage
 */

const rateLimit = require('express-rate-limit');
const RedisStore = require('rate-limit-redis');
const { getClient: getRedisClient, connectRedis } = require('../config/redis');
const response = require('../utils/response');

/**
 * Create rate limiter with Redis store
 * @param {Object} config - { prefix, windowMs, max, message }
 * @returns {Function} Express middleware
 */
const createLimiter = ({ prefix, windowMs, max, message }) => {
  return rateLimit({
    windowMs: windowMs || 15 * 60 * 1000, // Default: 15 minutes
    max: max || 100, // Default: 100 requests
    standardHeaders: true, // RateLimit-* headers
    legacyHeaders: false, // Disable X-RateLimit-* headers

    // Redis store for distributed rate limiting
  });
};
```

```
store: new RedisStore({
  sendCommand: async (...args) => {
    const client = getRedisClient();
    if (!client.isOpen) {
      await connectRedis();
    }
    return client.sendCommand(args);
  },
  prefix: `rl:${prefix}:`,
}),

// Custom error handler
handler: (req, res) => {
  console.warn(`Rate limit exceeded: ${req.ip} on ${req.path}`);
  return response.tooManyRequests(res, message || 'Too many requests');
},

// Key generator (default: IP address)
keyGenerator: (req) => {
  return req.ip;
}
});

// Pre-configured limiters
const apiLimiter = createLimiter({
  prefix: 'api',
  windowMs: 15 * 60 * 1000,
  max: 100
});

const otpLimiter = createLimiter({
  prefix: 'otp',
  windowMs: 15 * 60 * 1000,
  max: 5,
  message: 'Too many OTP requests, please try again in 15 minutes'
});

const loginLimiter = createLimiter({
  prefix: 'login',
  windowMs: 60 * 1000,
  max: 5,
  message: 'Too many login attempts, please try again in 1 minute'
});

const searchLimiter = createLimiter({
  prefix: 'search',
```

```
    windowMs: 60 * 1000,  
    max: 60,  
    message: 'Search rate limit exceeded'  
  });  
  
module.exports = {  
  createLimiter,  
  apiLimiter,  
  otpLimiter,  
  loginLimiter,  
  searchLimiter  
};
```

6. API Endpoint Specifications

6.1 Authentication Endpoints

POST /api/auth/login/request-otp

Request:

```
{  
  "phone_number": "+919876543210",  
  "name": "John Doe",  
  "deviceId": "abc123def456"  
}
```

Response (200 OK):

```
{  
  "success": true,  
  "data": {  
    "message": "OTP sent successfully",  
    "otp": "123456" // Only in dev mode  
  }  
}
```

Error Responses:

- 400 Bad Request - Invalid input
- 401 Unauthorized - User is blocked
- 429 Too Many Requests - Rate limit exceeded

POST /api/auth/login/verify-otp

Request:

```
{
  "phone_number": "+919876543210",
  "otp": "123456",
  "deviceId": "abc123def456"
}
```

Response (200 OK):

```
{
  "success": true,
  "data": {
    "user": {
      "id": "507f1f77bcf86cd799439011",
      "name": "John Doe",
      "phone_number": "+919876543210"
    },
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "isNewDevice": false,
    "deviceChanged": true
  },
  "message": "Login successful"
}
```

Cookies Set:

```
Set-Cookie: auth_token=<jwt>; HttpOnly; Secure; SameSite=Lax; Max-Age=315360000
```

6.2 PDF Endpoints

GET /api/pdfs

Query Parameters:

Parameter	Type	Default	Description
-----------	------	---------	-------------

q	string	-	Search term
category	string	-	Filter by category
page	number	1	Page number
limit	number	20	Items per page (max 100)

Response (200 OK):

```
{
  "success": true,
  "data": {
    "pdfs": [
      {
        "id": "507f1f77bcf86cd799439011",
        "title": "Flight Operations Manual",
        "description": "Standard procedures...",
        "category": "Operations",
        "pdfUrl": "/uploads/abc123/manual.pdf",
        "thumbnailUrl": "/uploads/abc123/thumb.jpg",
        "uploadedBy": "Admin Name",
        "views": 1234,
        "createdAt": "2026-01-19T10:00:00.000Z"
      }
    ],
    "pagination": {
      "currentPage": 1,
      "totalPages": 5,
      "totalItems": 100,
      "itemsPerPage": 20
    }
  }
}
```

POST /api/pdfs/upload

Request: multipart/form-data

Field	Type	Required	Description
pdf	file	Yes	PDF file (max 50MB)
thumbnail	file	No	Image file (jpg/png/webp)
title	string	Yes	PDF title
description	string	No	Description
category	string	Yes	Category name

Response (201 Created):

```
{
  "success": true,
  "data": {
    "pdf": {
      "id": "507f1f77bcf86cd799439011",
      "title": "New Manual",
      "pdfUrl": "/uploads/uuid123/file.pdf",
      "thumbnailUrl": "/uploads/uuid123/thumb.jpg"
    }
  },
  "message": "PDF uploaded successfully"
}
```

6.3 SSE Endpoint

GET /api/events

Headers:

```
Authorization: Bearer <token>
Accept: text/event-stream
```

Response Stream:

```
event: connected
data: {"message":"Connected to events stream"}

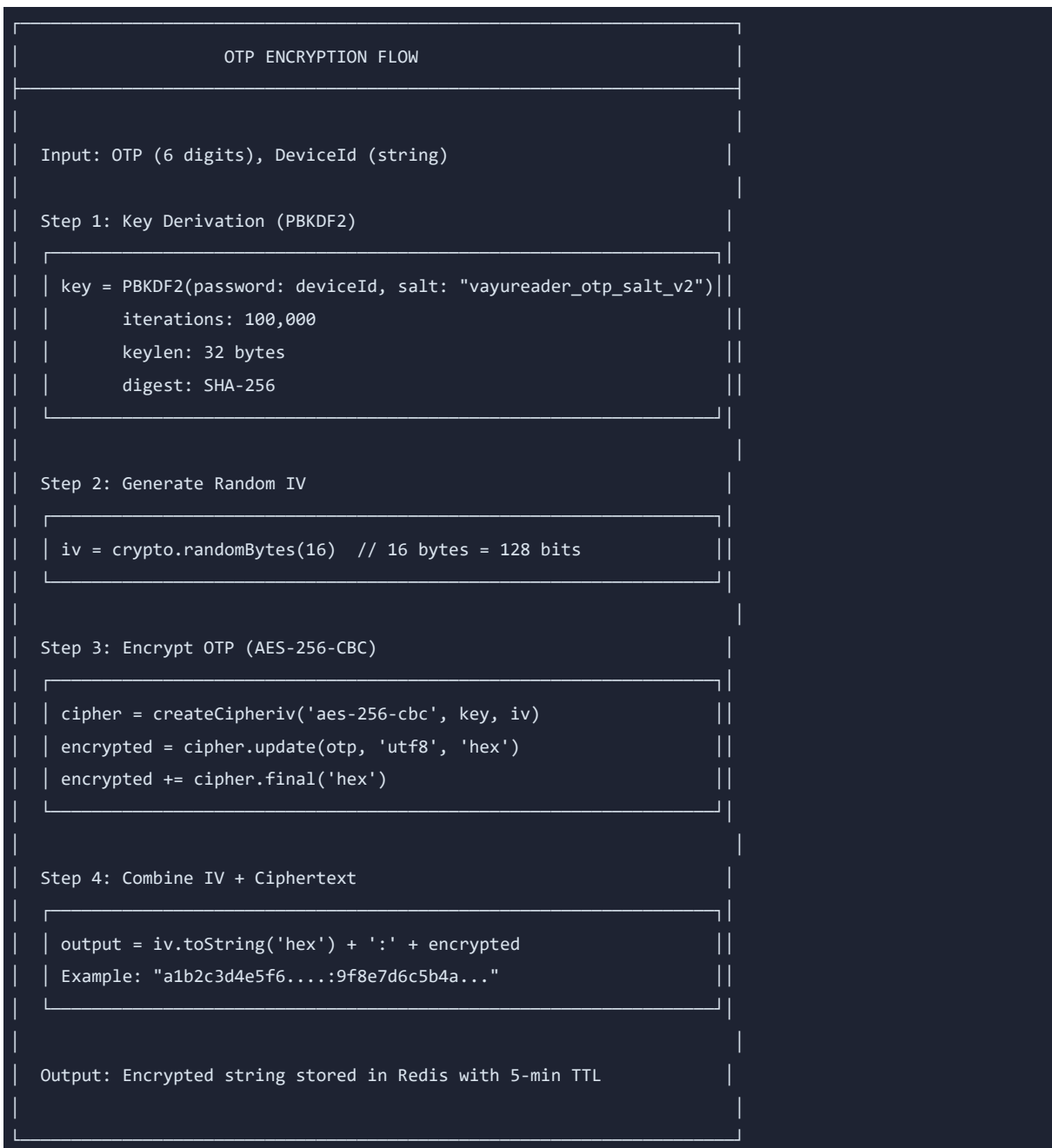
event: PDF_ADDED
data: {"id":"507f...", "title":"New
PDF", "category":"Ops", "timestamp":"2026-01-19T10:00:00.000Z"}

:heartbeat

event: PDF_DELETED
data: {"id":"507f...", "timestamp":"2026-01-19T11:00:00.000Z"}
```

7. Algorithm Specifications

7.1 OTP Encryption Algorithm



7.2 Password Hashing (Admin)

```
// Hashing (during admin creation)
const bcrypt = require('bcryptjs');
const SALT_ROUNDS = 10;

const hashPassword = async (plainPassword) => {
  return await bcrypt.hash(plainPassword, SALT_ROUNDS);
};

// Verification (during login)
const verifyPassword = async (plainPassword, hash) => {
  return await bcrypt.compare(plainPassword, hash);
};
```

Time Complexity: $O(2^{\text{SALT_ROUNDS}})$ - intentionally slow to prevent brute force

7.3 Search Ranking Algorithm

```
/**
 * Dictionary Word Search Ranking
 *
 * Priority Order:
 * 1. Exact match (uppercase) - Score: 100
 * 2. Prefix match - Score: 50
 * 3. Text search match - Score: text score from MongoDB
 */

const searchWord = async (term) => {
  const results = await Word.aggregate([
    {
      $match: {
        $or: [
          { word: term.toUpperCase() }, // Exact match
          { word: { $regex: `^${escapeRegex(term)}`, $options: 'i' } }, //
Prefix
          { $text: { $search: term } } // Full-text
        ]
      }
    },
    {
      $addFields: {
        score: {
          $cond: {
            if: { $eq: ['$word', term.toUpperCase()] },
            then: 100,
            else: {
```

```
        $cond: {
          if: { $regexMatch: { input: '$word', regex: `^${term}`, options:
'i' } },
          then: 50,
          else: { $meta: 'textScore' }
        }
      }
    }
  }
},
{ $sort: { score: -1 } },
{ $limit: 20 }
]);

return results;
};
```

8. Error Handling

8.1 Error Response Format

```
// utils/response.js

const success = (res, data, message = 'Success', statusCode = 200) => {
  return res.status(statusCode).json({
    success: true,
    data,
    message
  });
};

const error = (res, message, errorCode, statusCode = 400) => {
  return res.status(statusCode).json({
    success: false,
    message,
    errorCode
  });
};
```

```
// Standard error methods
const badRequest = (res, message = 'Bad Request') =>
  error(res, message, 'BAD_REQUEST', 400);

const unauthorized = (res, message = 'Unauthorized') =>
  error(res, message, 'UNAUTHORIZED', 401);

const forbidden = (res, message = 'Forbidden') =>
  error(res, message, 'FORBIDDEN', 403);

const notFound = (res, message = 'Not Found') =>
  error(res, message, 'NOT_FOUND', 404);

const tooManyRequests = (res, message = 'Too Many Requests') =>
  error(res, message, 'RATE_LIMIT_EXCEEDED', 429);

const serverError = (res, message = 'Internal Server Error') =>
  error(res, message, 'SERVER_ERROR', 500);
```

8.2 Global Error Handler

```
// middleware/errorHandler.js

const response = require('../utils/response');

const errorHandler = (err, req, res, next) => {
  console.error('Unhandled error:', err);

  // Mongoose validation error
  if (err.name === 'ValidationError') {
    const messages = Object.values(err.errors).map(e => e.message);
    return response.badRequest(res, messages.join(', '));
  }

  // Mongoose duplicate key error
  if (err.code === 11000) {
    const field = Object.keys(err.keyPattern)[0];
    return response.badRequest(res, `${field} already exists`);
  }

  // JWT errors
  if (err.name === 'JsonWebTokenError') {
    return response.unauthorized(res, 'Invalid token');
  }
}
```

```
if (err.name === 'TokenExpiredError') {
  return response.unauthorized(res, 'Token expired');
}

// Multer file upload errors
if (err.code === 'LIMIT_FILE_SIZE') {
  return response.badRequest(res, 'File too large (max 50MB)');
}

// Default server error
return response.serverError(res);
};

module.exports = errorHandler;
```

9. Configuration Management

9.1 Environment Variables

File: .env.example

```
# Server
NODE_ENV=development
PORT=3000

# Database
MONGODB_URI=mongodb://localhost:27017/vayureader

# Redis
REDIS_URL=redis://localhost:6379

# JWT
JWT_SECRET=your_jwt_secret_minimum_32_characters_long
JWT_EXPIRY_DAYS=1

# OTP
OTP_GATEWAY_URL=http://localhost:8000/smsc/sends
OTP_EXPIRY_MINUTES=5
```

```
SKIP_OTP_SEND=true

# CORS
ALLOWED_ORIGINS=http://localhost:3000,http://localhost:8080

# Rate Limiting
RATE_LIMIT_WINDOW_MS=900000
RATE_LIMIT_MAX_REQUESTS=100
OTP_RATE_LIMIT_MAX=5

# Elasticsearch (Optional)
ELASTICSEARCH_NODE=http://localhost:9200

# Testing
TESTING=false
```

9.2 Configuration Module

File: src/config/environment.js

```
require('dotenv').config();

// Fail-fast validation
const requiredVars = ['MONGODB_URI', 'JWT_SECRET', 'OTP_GATEWAY_URL'];
for (const varName of requiredVars) {
  if (!process.env[varName]) {
    console.error(`FATAL: Missing required environment variable: ${varName}`);
    process.exit(1);
  }
}

// JWT secret minimum length check
if (process.env.JWT_SECRET.length < 32) {
  console.error('FATAL: JWT_SECRET must be at least 32 characters');
  process.exit(1);
}

module.exports = {
  server: {
    port: parseInt(process.env.PORT, 10) || 3000,
    nodeEnv: process.env.NODE_ENV || 'development',
    isDevelopment: process.env.NODE_ENV !== 'production'
  },
  database: {
    uri: process.env.MONGODB_URI
```



```
},
redis: {
  url: process.env.REDIS_URL || 'redis://localhost:6379'
},
jwt: {
  secret: process.env.JWT_SECRET,
  expiryDays: parseInt(process.env.JWT_EXPIRY_DAYS, 10) || 1
},
otp: {
  gatewayUrl: process.env.OTP_GATEWAY_URL,
  expiryMinutes: parseInt(process.env.OTP_EXPIRY_MINUTES, 10) || 5,
  skipSend: process.env.SKIP_OTP_SEND === 'true'
},
cors: {
  allowedOrigins: (process.env.ALLOWED_ORIGINS ||
'').split(',').filter(Boolean)
},
rateLimit: {
  windowMs: parseInt(process.env.RATE_LIMIT_WINDOW_MS, 10) || 15 * 60 * 1000,
  maxRequests: parseInt(process.env.RATE_LIMIT_MAX_REQUESTS, 10) || 100,
  otpMaxRequests: parseInt(process.env.OTP_RATE_LIMIT_MAX, 10) || 5
}
};
```

10. Testing Specifications

10.1 Unit Test Structure

```
// tests/services/otp.service.test.js

const { generateOtp, saveOtp, verifyOtp, deleteOtp } =
require('../../src/services/otp.service');

describe('OTP Service', () => {
  describe('generateOtp', () => {
    it('should generate 6-digit numeric OTP', () => {
      const otp = generateOtp();
      expect(otp).toMatch(/^\d{6}$/);
    });
  });
});
```

```
it('should generate unique OTPs', () => {
  const otps = new Set();
  for (let i = 0; i < 100; i++) {
    otps.add(generateOtp());
  }
  expect(otps.size).toBeGreaterThan(90); // Allow some duplicates
});

describe('saveOtp and verifyOtp', () => {
  const phone = '+1234567890';
  const deviceId = 'test-device';
  const otp = '123456';

  afterEach(async () => {
    await deleteOtp(phone);
  });

  it('should save and verify correct OTP', async () => {
    await saveOtp(phone, otp, deviceId);
    const result = await verifyOtp(otp, phone, deviceId);
    expect(result.valid).toBe(true);
  });

  it('should reject wrong OTP', async () => {
    await saveOtp(phone, otp, deviceId);
    const result = await verifyOtp('999999', phone, deviceId);
    expect(result.valid).toBe(false);
  });

  it('should reject wrong deviceId', async () => {
    await saveOtp(phone, otp, deviceId);
    const result = await verifyOtp(otp, phone, 'wrong-device');
    expect(result.valid).toBe(false);
  });
});
```

10.2 Integration Test Structure

```
// tests/routes/auth.routes.test.js

const request = require('supertest');
const app = require('../../src/server');
```

```
const User = require('../../src/models/User');

describe('Auth Routes', () => {
  beforeEach(async () => {
    await User.deleteMany({});
  });

  describe('POST /api/auth/login/request-otp', () => {
    it('should return 200 for valid request', async () => {
      const res = await request(app)
        .post('/api/auth/login/request-otp')
        .send({
          phone_number: '+1234567890',
          name: 'Test User',
          deviceId: 'test-device'
        });

      expect(res.status).toBe(200);
      expect(res.body.success).toBe(true);
    });

    it('should return 400 for missing fields', async () => {
      const res = await request(app)
        .post('/api/auth/login/request-otp')
        .send({
          phone_number: '+1234567890'
          // Missing name and deviceId
        });

      expect(res.status).toBe(400);
      expect(res.body.success).toBe(false);
    });

    it('should return 429 when rate limit exceeded', async () => {
      const payload = {
        phone_number: '+1234567890',
        name: 'Test User',
        deviceId: 'test-device'
      };

      // Make 5 requests
      for (let i = 0; i < 5; i++) {
        await request(app)
          .post('/api/auth/login/request-otp')
          .send(payload);
      }
    });
  });
});
```

```
// 6th request should be rate limited
const res = await request(app)
  .post('/api/auth/login/request-otp')
  .send(payload);

expect(res.status).toBe(429);
});
});
});
```

11. Deployment Specifications

11.1 Dockerfile

```
# Multi-stage build for production
FROM node:20-alpine AS base
WORKDIR /usr/src/app

# Dependencies stage
FROM base AS deps
COPY package*.json ./
RUN npm ci --only=production

# Build stage (if needed)
FROM base AS builder
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run lint

# Production stage
FROM base AS production
ENV NODE_ENV=production

# Copy production dependencies
COPY --from=deps /usr/src/app/node_modules ./node_modules

# Copy application code
COPY --chown=node:node . .
```

```
# Create uploads directory
RUN mkdir -p uploads && chown node:node uploads

# Switch to non-root user
USER node

# Health check
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \
  CMD wget --no-verbose --tries=1 --spider http://localhost:3000/health || exit
1

EXPOSE 3000

CMD ["node", "src/server.js"]
```

11.2 Docker Compose

```
version: '3.8'

services:
  nginx:
    image: nginx:alpine
    container_name: vayureader_gateway
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx/conf.d:/etc/nginx/conf.d
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf
      - ./nginx/certs:/etc/nginx/certs
      - ./uploads:/usr/src/app/uploads:ro
    depends_on:
      app:
        condition: service_healthy
    networks:
      - backend-network
    restart: unless-stopped

  app:
    build:
      context: .
      target: production
    container_name: vayureader_api
    environment:
```

```
- NODE_ENV=production
- MONGODB_URI=mongodb://mongo:27017/vayureader
- REDIS_URL=redis://redis:6379
env_file:
- .env
volumes:
- ./uploads:/usr/src/app/uploads
depends_on:
- mongo
- redis
networks:
- backend-network
restart: unless-stopped
healthcheck:
  test: ["CMD", "wget", "-q", "--spider", "http://localhost:3000/health"]
  interval: 30s
  timeout: 10s
  retries: 3
  start_period: 30s

mongo:
  image: mongo:6
  container_name: vayureader_db
  volumes:
    - mongo-data:/data/db
  networks:
    - backend-network
  restart: unless-stopped

redis:
  image: redis:7-alpine
  container_name: vayureader_cache
  command: redis-server --appendonly yes
  volumes:
    - redis-data:/data
  networks:
    - backend-network
  restart: unless-stopped

sms-simulator:
  build:
    context: .
    dockerfile: Dockerfile
  container_name: vayureader_sms
  command: node scripts/sms-simulator.js
  ports:
    - "8000:8000"
```

```
networks:
  - backend-network
profiles:
  - dev
```

```
networks:
  backend-network:
    driver: bridge
```

```
volumes:
  mongo-data:
  redis-data:
```

12. Document Revision History

Version	Date	Author	Changes
1.0	Jan 2025	Dev Team	Initial LLD
2.0	Jan 2026	Dev Team	Added SSE, enhanced security

Part III: User Guides

User Manual

System Administration Guide

Version: 2.0 Date: January 2026 Classification: RESTRICTED - For Authorized Personnel Only

Table of Contents

- Introduction
 - System Overview
 - Getting Started
 - User Management
 - PDF Management
 - Dictionary Management
 - Abbreviation Management
 - Audit & Monitoring
 - Troubleshooting
 - API Reference Quick Guide
 - Appendix
-

1. Introduction

1.1 Purpose

This User Manual provides comprehensive guidance for administrators and operators of the VayuReader Backend system. It covers day-to-day operations, system administration, content

management, and troubleshooting procedures.

1.2 Intended Audience

- System Administrators: Personnel responsible for system deployment, maintenance, and monitoring
- Content Administrators: Personnel responsible for PDF uploads, dictionary updates, and content management
- Super Administrators: Personnel with full system access including user and admin management

1.3 Prerequisites

Before using this system, ensure you have:

- Access credentials (contact and password)
 - Registered phone number for OTP authentication
 - Authorized device for admin access
 - Basic understanding of REST APIs (for API integration)
-

2. System Overview

2.1 What is VayuReader Backend?

VayuReader Backend is a secure document management and reference system designed for the Indian Air Force. It provides:

- Secure PDF Document Management: Upload, organize, and distribute PDF documents
- Dictionary Services: Word definitions, meanings, synonyms, and antonyms
- Abbreviation Database: Military and technical abbreviation expansions
- User Authentication: OTP-based secure login for mobile app users
- Admin Portal: Web-based administration for content and user management
- Real-time Updates: Instant notifications when content changes

2.2 System Components

Component	Purpose	Access
Nginx Gateway	HTTPS entry point	https://your-domain.com
Backend API	REST API services	https://your-domain.com/api
MongoDB	Data storage	Internal only
Redis	Caching and sessions	Internal only
Admin Dashboard	Web-based admin UI	https://admin.your-domain.co

2.3 User Roles

Role	Description	Capabilities
End User	Mobile app users	View PDFs, search dictionary
Sub-Admin	Limited admin access	Specific permissions (PDF, D
Super Admin	Full system access	All operations including adm

3. Getting Started

3.1 Installation (Docker)

Prerequisites

- Docker Desktop installed and running
- Docker Compose (included with Docker Desktop)
- SSL Certificates (self-signed for development, CA-signed for production)

Step 1: Generate SSL Certificates

For local development with self-signed certificates:

Windows (PowerShell):

```
docker run --rm -v "${PWD}/nginx/certs:/certs" alpine /bin/sh -c "apk add --no-cache openssl && openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout /certs/server.key -out /certs/server.crt -subj
```

```
'/C=IN/ST=Delhi/L=NewDelhi/O=IAF/CN=localhost'
```

Linux/Mac:

```
docker run --rm -v "$(pwd)/nginx/certs:/certs" alpine /bin/sh -c "apk add --no-cache openssl && openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout /certs/server.key -out /certs/server.crt -subj '/C=IN/ST=Delhi/L=NewDelhi/O=IAF/CN=localhost'"
```

Step 2: Configure Environment

Create .env file from template:

```
cp .env.example .env
```

Edit .env with your configuration:

```
# Required settings
MONGODB_URI=mongodb://mongo:27017/vayureader
REDIS_URL=redis://redis:6379
JWT_SECRET=your_super_secret_key_at_least_32_characters_long
OTP_GATEWAY_URL=http://sms-simulator:8000/smsc/sends

# Development settings
NODE_ENV=development
SKIP_OTP_SEND=true

# Production settings (uncomment for production)
# NODE_ENV=production
# SKIP_OTP_SEND=false
# ALLOWED_ORIGINS=https://admin.yourdomain.com,https://app.yourdomain.com
```

Step 3: Start the System

Development (with SMS Simulator):

```
docker-compose --profile dev up -d --build
```

Production (without SMS Simulator):

```
docker-compose up -d --build
```

Step 4: Verify Installation

- Check container status:

```
docker-compose ps
```

All containers should show "Up" status.

- Check health endpoint:

```
curl -k https://localhost/health
```

Should return: {"status":"ok"}

- View logs:

```
docker-compose logs -f
```

Step 5: Create Super Admin

Run the seed script to create the first administrator:

```
docker-compose exec app node scripts/seedAdmin.js "Admin Name" "9876543210"
"StrongPassword123!"
```

Output:

```
Connecting to MongoDB...
Connected to MongoDB
Creating super admin...
Super admin created successfully!
Name: Admin Name
Contact: 9876543210
Disconnected from DB
```

3.2 Accessing the System

Admin Dashboard Login

- Navigate to the Admin Dashboard URL
- Enter your registered contact number
- Enter your password
- Click "Request OTP"

- Enter the 6-digit OTP received on your phone
- Click "Verify" to complete login

API Access

All API endpoints are available at: <https://your-domain.com/api>

Example request:

```
curl -X GET https://localhost/api/pdfs \
-H "Authorization: Bearer YOUR_TOKEN" \
-H "Content-Type: application/json"
```

3.3 Stopping the System

Graceful shutdown:

```
docker-compose down
```

Stop and remove all data (CAUTION):

```
docker-compose down -v
```

4. User Management

4.1 Viewing Users

Super Admins can view all registered users through the admin dashboard or API.

Via API:

```
# Get all users (paginated)
GET /api/admin/users?page=1&limit=20

# Search users by phone
GET /api/admin/users?search=9876543210
```

4.2 Blocking a User

When a user needs to be restricted from accessing the system:

Via CLI:

```
docker-compose exec app node scripts/manageUsers.js user block --contact
+919876543210
```

Effect:

- User cannot request OTP
- User cannot verify OTP
- Existing JWT tokens become invalid (checked in middleware)
- User sees "Account blocked" error

4.3 Unblocking a User

To restore a blocked user's access:

Via CLI:

```
docker-compose exec app node scripts/manageUsers.js user unblock --contact
+919876543210
```

4.4 Deleting a User

To permanently remove a user:

Via CLI:

```
# By phone number
docker-compose exec app node scripts/manageUsers.js user delete --contact
+919876543210

# By user ID
docker-compose exec app node scripts/manageUsers.js user delete --id
507f1f77bcf86cd799439011

# By device ID
docker-compose exec app node scripts/manageUsers.js user delete --device
abc123def456
```

Warning: This action is irreversible. The user will need to re-register.

4.5 Managing Admins

Creating a Sub-Admin

Only Super Admins can create new administrators.

Via API:

```
POST /api/admin/sub-admins
Content-Type: application/json
Authorization: Bearer SUPER_ADMIN_TOKEN

{
  "name": "New Admin",
  "contact": "+919876543211",
  "password": "SecurePassword123!",
  "permissions": ["manage_pdfs", "manage_dictionary"]
}
```

Available Permissions:

- manage_pdfs - Upload, edit, delete PDFs
- manage_dictionary - Manage dictionary words
- manage_abbreviations - Manage abbreviations
- view_audit - View audit logs
- manage_admins - Create/delete sub-admins (Super Admin only)

Deleting an Admin

Via CLI:

```
docker-compose exec app node scripts/manageUsers.js admin delete --contact
+919876543211
```

Note: Super Admins cannot be deleted via API for security. Use the CLI script.

Resetting Admin Password

If an admin forgets their password:

Via CLI:


```
docker-compose exec app node scripts/reset-admin-password.js +919876543210
"NewSecurePassword123!"
```

5. PDF Management

5.1 Uploading PDFs

Via Admin Dashboard:

- Navigate to "PDFs" → "Upload New"
- Fill in the details:
 - Title (required): Descriptive name
 - Category (required): Select or create category
 - Description (optional): Brief summary
 - PDF File (required): Select PDF file (max 50MB)
 - Thumbnail (optional): Cover image (JPG/PNG)
- Click "Upload"

```
curl -X POST https://localhost/api/pdfs/upload \
-H "Authorization: Bearer ADMIN_TOKEN" \
-F "pdf=@/path/to/document.pdf" \
-F "thumbnail=@/path/to/cover.jpg" \
-F "title=Flight Operations Manual" \
-F "description=Standard procedures for flight operations" \
-F "category=Operations"
```

5.2 Updating PDFs

Via Admin Dashboard:

- Navigate to "PDFs" → Find the PDF
- Click "Edit"
- Modify fields as needed
- Click "Save Changes"

```
curl -X PUT https://localhost/api/pdfs/507f1f77bcf86cd799439011 \
-H "Authorization: Bearer ADMIN_TOKEN" \
```

```
-F "title=Updated Title" \  
-F "category=New Category"
```

5.3 Deleting PDFs

Via Admin Dashboard:

- Navigate to "PDFs" → Find the PDF
- Click "Delete"
- Confirm deletion

```
curl -X DELETE https://localhost/api/pdfs/507f1f77bcf86cd799439011 \  
-H "Authorization: Bearer ADMIN_TOKEN"
```

Warning: This removes both the database record and the physical files.

5.4 PDF Categories

Categories are created automatically when uploading PDFs. To view all categories:

Via API:

```
GET /api/pdfs/categories
```

5.5 File Storage

PDFs are stored in the uploads/ directory with the following structure:

```
uploads/  
├─ abc123-uuid/  
|   ├─ document.pdf  
|   └─ thumbnail.jpg  
├─ def456-uuid/  
|   ├─ another.pdf  
|   └─ cover.png
```

Backup Recommendation: Regularly backup the uploads/ directory alongside database backups.

6. Dictionary Management

6.1 Adding Words

Via Admin Dashboard:

- Navigate to "Dictionary" → "Add Word"
- Enter word details:
 - Word (required): The word to define
 - Part of Speech: noun, verb, adjective, etc.
 - Definition (required): Word meaning
 - Synonyms (optional): Related words
 - Antonyms (optional): Opposite words
 - Example Sentences (optional): Usage examples
- Click "Save"

```
curl -X POST https://localhost/api/dictionary \
-H "Authorization: Bearer ADMIN_TOKEN" \
-H "Content-Type: application/json" \
-d '{
  "word": "AIRCRAFT",
  "meanings": [
    {
      "partOfSpeech": "noun",
      "definition": "A vehicle that can fly, such as a plane or helicopter",
      "synonyms": ["plane", "aeroplane", "jet"],
      "examples": ["The aircraft landed safely on the runway"]
    }
  ],
  "synonyms": ["plane", "jet"],
  "antonyms": []
}'
```

6.2 Bulk Upload

For large dictionary imports:

Via API:

```
curl -X POST https://localhost/api/dictionary/upload \
-H "Authorization: Bearer ADMIN_TOKEN" \
```

```
-H "Content-Type: application/json" \  
-d '{  
  "words": [  
    {  
      "word": "WORD1",  
      "meanings": [{"definition": "Definition 1"}]  
    },  
    {  
      "word": "WORD2",  
      "meanings": [{"definition": "Definition 2"}]  
    }  
  ]  
'
```

Response:

```
{  
  "success": true,  
  "data": {  
    "imported": 2,  
    "skipped": 0,  
    "errors": []  
  }  
}
```

6.3 Updating Words

Via API:

```
curl -X PUT https://localhost/api/dictionary/507f1f77bcf86cd799439011 \  
-H "Authorization: Bearer ADMIN_TOKEN" \  
-H "Content-Type: application/json" \  
-d '{  
  "meanings": [  
    {  
      "definition": "Updated definition"  
    }  
  ]  
'
```

6.4 Exporting Dictionary

To backup or transfer dictionary data:

Via API:

```
GET /api/dictionary/export/all
```

6.5 Searching Words

Users can search the dictionary:

```
# Exact lookup
GET /api/dictionary/word/AIRCRAFT

# Search
GET /api/dictionary/search/air?page=1&limit=20
```

7. Abbreviation Management

7.1 Adding Abbreviations

Via Admin Dashboard:

- Navigate to "Abbreviations" → "Add New"
- Enter details:
 - Abbreviation (required): e.g., "NATO"
 - Full Form (required): e.g., "North Atlantic Treaty Organization"
- Click "Save"

```
curl -X POST https://localhost/api/abbreviations \
-H "Authorization: Bearer ADMIN_TOKEN" \
-H "Content-Type: application/json" \
-d '{
  "abbreviation": "IAF",
  "fullForm": "Indian Air Force"
}'
```

7.2 Bulk Upload

Via API:

```
curl -X POST https://localhost/api/abbreviations/upload \
-H "Authorization: Bearer ADMIN_TOKEN" \
-H "Content-Type: application/json" \
-d '{
  "abbreviations": [
    {"abbreviation": "AOC", "fullForm": "Air Officer Commanding"},
    {"abbreviation": "CAS", "fullForm": "Chief of Air Staff"}
  ]
}'
```

7.3 Searching Abbreviations

```
# Exact lookup
GET /api/abbreviations/IAF

# Search
GET /api/abbreviations/search/air?page=1&limit=20
```

8. Audit & Monitoring

8.1 User Activity Logs

Track user activities including logins, device changes, and PDF views.

Via API:

```
# Get user activity logs
GET /api/audit/user-logs?page=1&limit=50

# Filter by action type
GET /api/audit/user-logs?action=LOGIN

# Filter by user
GET /api/audit/user-logs?phone=+919876543210
```

Logged Actions:

- LOGIN - User login events
- DEVICE_CHANGE - User switched to new device
- READ_PDF - User viewed a PDF

8.2 Admin Action Logs

Track all administrative changes for compliance.

Via API:

```
# Get admin action logs
GET /api/audit/logs?page=1&limit=50

# Filter by admin
GET /api/audit/logs?adminName=Admin%20Name

# Filter by resource type
GET /api/audit/logs?resourceType=PDF
```

Logged Actions:

- CREATE - Resource created
- UPDATE - Resource modified
- DELETE - Resource deleted
- PDF - PDF documents
- DICTIONARY_WORD - Dictionary entries
- ABBREVIATION - Abbreviation entries
- ADMIN - Administrator accounts

8.3 System Health Monitoring

Health Endpoint:

```
curl https://localhost/health
```

Response:

```
{
  "status": "ok",
  "timestamp": "2026-01-19T10:00:00.000Z"
}
```

8.4 Container Logs

View all logs:

```
docker-compose logs -f
```

View specific service logs:

```
docker-compose logs -f app      # Backend logs
docker-compose logs -f nginx    # Gateway logs
docker-compose logs -f mongo    # Database logs
docker-compose logs -f redis    # Cache logs
```

8.5 Database Optimization

Run periodically to ensure optimal performance:

```
docker-compose exec app node scripts/optimizeDb.js
```

Output:

```
Optimizing database...

Creating indexes...
  User indexes created
  Admin indexes created
  PdfDocument indexes created
...

Analyzing collections...
Users: 1,234 documents
PDFs: 567 documents
...

Optimization completed!
```

8.6 Log Cleanup

Remove old audit logs to free space:

```
# Dry run (see what would be deleted)
```



```
docker-compose exec app node scripts/cleanupLogs.js --days 90 --dry-run

# Actually delete
docker-compose exec app node scripts/cleanupLogs.js --days 90
```

9. Troubleshooting

9.1 Common Issues and Solutions

Cannot Login - "Invalid OTP"

Cause: Device ID mismatch between OTP request and verification

Solution:

- Ensure the same device ID is sent in both requests
- Clear app cache and retry
- Check if OTP has expired (5 minutes validity)

Cannot Login - "User blocked"

Cause: User account has been blocked by administrator

Solution:

- Contact administrator to unblock account
- Admin runs: `node scripts/manageUsers.js user unblock --contact +91XXXXXXXXXX`

502 Bad Gateway

Cause: Backend container is not running or not responding

Solution:

- Check backend status: `docker-compose ps app`
- View backend logs: `docker-compose logs app`
- Restart backend: `docker-compose restart app`
- If persists, check MongoDB connection

429 Too Many Requests

Cause: Rate limit exceeded

Solution:

- Wait for rate limit window to reset (check Retry-After header)
- For OTP: Wait 15 minutes
- For general API: Wait 15 minutes
- For login attempts: Wait 1 minute

CORS Error

Cause: Frontend origin not in allowed list

Solution:

- Check ALLOWED_ORIGINS in .env
- Add frontend URL: ALLOWED_ORIGINS=https://admin.yourdomain.com
- Restart: docker-compose up -d

SSL Certificate Warning

Cause: Self-signed certificate (development)

Solution:

- Development: Click "Advanced" → "Proceed to localhost"
- Production: Replace with CA-signed certificate

PDF Upload Failed - 413 Payload Too Large

Cause: File exceeds maximum size limit

Solution:

- Ensure file is under 50MB
- If larger files needed, update Nginx config:

```
client_max_body_size 100M;
```

- Restart Nginx: docker-compose restart nginx

MongoDB Connection Error

Cause: Database container not ready or crashed

Solution:

- Check MongoDB status: `docker-compose ps mongo`
- View logs: `docker-compose logs mongo`
- Restart MongoDB: `docker-compose restart mongo`
- Wait 30 seconds for initialization

9.2 Checking System Status

Container Status

```
docker-compose ps
```

Expected Output:

NAME	STATUS	PORTS
vayureader_api	Up (healthy)	3000/tcp
vayureader_db	Up	27017/tcp
vayureader_cache	Up	6379/tcp
vayureader_gateway	Up	80->80, 443->443

Redis Status

```
docker-compose exec redis redis-cli ping
# Expected: PONG
```

MongoDB Status

```
docker-compose exec mongo mongosh --eval "db.stats()"
```

9.3 Emergency Procedures

System Unresponsive

- Stop all containers:

```
docker-compose down
```

- Check disk space:

```
docker system df
```

- Clean up if needed:

```
docker system prune -f
```

- Restart:

```
docker-compose up -d
```

Data Recovery

If data corruption is suspected:

- Stop the system
- Backup current data:

```
docker-compose exec mongo mongodump --out=/backup
```

- Restore from previous backup:

```
docker-compose exec mongo mongorestore /backup/previous
```

10. API Reference Quick Guide

10.1 Authentication Endpoints

Method	Endpoint	Description	Auth
POST	/api/auth/login/request-otp	Request OTP for user login	None
POST	/api/auth/login/verify-otp	Verify OTP and get token	None
GET	/api/auth/profile	Get current user profile	User
POST	/api/auth/logout	Clear auth cookie	None

10.2 Admin Endpoints

Method	Endpoint	Description	Auth
--------	----------	-------------	------

POST	/api/admin/login/request-otp	Admin login step 1	None
POST	/api/admin/login/verify-otp	Admin login step 2	None
GET	/api/admin/me	Get admin profile	Admin
POST	/api/admin/sub-admins	Create sub-admin	Super Admin
DELETE	/api/admin/sub-admins/:id	Delete sub-admin	Super Admin

10.3 PDF Endpoints

Method	Endpoint	Description	Auth
GET	/api/pdfs	Search/list PDFs	User/Admin
GET	/api/pdfs/:id	Get single PDF	User/Admin
POST	/api/pdfs/upload	Upload PDF	Admin
PUT	/api/pdfs/:id	Update PDF	Admin
DELETE	/api/pdfs/:id	Delete PDF	Admin
GET	/api/pdfs/categories	Get categories	User/Admin

10.4 Dictionary Endpoints

Method	Endpoint	Description	Auth
GET	/api/dictionary/word/:word	Lookup word	Public
GET	/api/dictionary/search/:term	Search words	Public
POST	/api/dictionary	Add word	Admin
PUT	/api/dictionary/:id	Update word	Admin
DELETE	/api/dictionary/:id	Delete word	Admin
POST	/api/dictionary/upload	Bulk upload	Admin
GET	/api/dictionary/export/all	Export all	Admin

10.5 Abbreviation Endpoints

Method	Endpoint	Description	Auth
GET	/api/abbreviations/:abbr	Lookup abbreviation	Public
GET	/api/abbreviations/search/:t	Search	Public
POST	/api/abbreviations	Add abbreviation	Admin

POST	/api/abbreviations/upload	Bulk upload	Admin
------	---------------------------	-------------	-------

10.6 Audit Endpoints

Method	Endpoint	Description	Auth
GET	/api/audit/logs	Get admin action logs	Admin
GET	/api/audit/user-logs	Get user activity logs	Admin

10.7 SSE Endpoint

Method	Endpoint	Description	Auth
GET	/api/events	Real-time event stream	User/Admin

Appendix

A. Environment Variables Reference

Variable	Required	Default	Description
NODE_ENV	No	development	Environment mode
PORT	No	3000	Server port
MONGODB_URI	Yes	-	MongoDB connection string
REDIS_URL	Yes	-	Redis connection URL
JWT_SECRET	Yes	-	JWT signing secret (min 32 c
OTPGATEWAYURL	Yes	-	SMS gateway URL
OTPEXPIRYMINUTES	No	5	OTP validity period
SKIPOTPSEND	No	false	Skip actual SMS sending
ALLOWED_ORIGINS	No	-	CORS allowed origins

B. Permission Reference

Permission	Description	Endpoints
manage_pdfs	Upload, edit, delete PDFs	/api/pdfs/* (POST, PUT, DELETE)
manage_dictionary	Manage dictionary	/api/dictionary/* (POST, PUT)
manage_abbreviations	Manage abbreviations	/api/abbreviations/* (POST, PUT)
manage_admins	Create/delete admins	/api/admin/sub-admins/*
view_audit	View audit logs	/api/audit/*

C. Error Codes Reference

Code	HTTP Status	Description
BAD_REQUEST	400	Invalid input data
UNAUTHORIZED	401	Authentication required
FORBIDDEN	403	Permission denied
NOT_FOUND	404	Resource not found
RATELIMITEXCEEDED	429	Too many requests
SERVER_ERROR	500	Internal server error

D. Contact & Support

For technical support, contact your system administrator or IT department.

Emergency Contacts:

- System Administrator: [Contact Information]
- IT Support: [Contact Information]

Document Revision History

Version	Date	Author	Changes
---------	------	--------	---------

1.0	Jan 2025	Dev Team	Initial manual
2.0	Jan 2026	Dev Team	Updated with SSE, enhanced a

Setup & Installation

This document provides a comprehensive guide to understanding, running, and debugging the VayuReader Backend in a containerized Docker environment.

Architecture Overview

The system runs as a multi-container application orchestrated by Docker Compose. It consists of 5 isolated services that communicate over a private bridge network.

Service Map

Service	Container Name	Internal Port	Host Port	Description
Nginx	vayureader_gateway	80, 443	80, 443	Entry Point. Reverse proxy,
App	vayureader_api	3000	-	Backend. Node.js API server
Mongo	vayureader_db	27017	-	Database. Persistent primary
Redis	vayureader_cache	6379	-	Cache. Session storage, OTPs
SMS	vayureader_sms	8000	8000	Dev Tool. Local SMS simulato

Note: Only Nginx (80/443) and SMS Simulator (8000) are exposed to your host machine. The Backend, Database, and Redis are locked inside the internal network for security.

Configuration Explained

1. docker-compose.yml Breakdown

This file is the blueprint for the entire stack.

- **networks:** We use a custom bridge network (backend-network). This allows services to resolve each other by name (e.g., the backend connects to `mongodb://mongo:27017` because the service name is `mongo`).
- **volumes:**
 - `./uploads`: Shared between App (to write files) and Nginx (to serve files).
 - `mongo-data`: A managed volume to persist database records even if containers are destroyed.
- **healthcheck:**
 - The App service has a healthcheck that pings `/health`.
 - Nginx relies on `dependson: { condition: servicehealthy }`. This ensures Nginx waits until the backend is fully running before starting, preventing "502 Bad Gateway" errors on startup.
- **profiles:**
 - The `sms-simulator` is assigned to the `dev` profile. It will NOT start in production.

2. Environment Variables (.env)

The Docker container loads environment variables from your `.env` file. Crucial variables for Docker:

- `NODE_ENV=production`: Optimizes Node.js for performance.
- `MONGODB_URI`: set to `mongodb://mongo:27017/vayureader` (Docker DNS).
- `ALLOWED_ORIGINS`: Must include your frontend URL (e.g., `https://localhost:3000`) for CORS.
- `OTPGATEWAYURL`:
 - Development: `http://sms-simulator:8000/smsc/sends`
 - Production: `https://api.your-sms-provider.com/send` (Replace with real vendor URL)

3. Nginx Configuration (nginx/conf.d/default.conf)

- **SSL Termination**: Handles HTTPS encryption so your Node.js app doesn't have to.
- **Proxy Pass**: Forwards requests to `http://app:3000`.
- **Client Body Size**: Set to 50M to allow large PDF uploads.

Installation & Setup

Prerequisites

- Docker Desktop installed and running.

step 1: Generate SSL Certificates

Since we use HTTPS locally, we need self-signed certificates. You can generate these using Docker (no need to install OpenSSL on Windows):

```
docker run --rm -v ${PWD}/nginx/certs:/certs -w /certs alpine/openssl req -x509  
-nodes -days 365 -newkey rsa:2048 -keyout server.key -out server.crt -subj  
"/C=IN/ST=Delhi/L=NewDelhi/O=IAF/OU=IT/CN=localhost"
```

Step 2: Start the Stack

For Development (with SMS Simulator):

```
docker-compose --profile dev up -d --build
```

For Production (No Simulator):

```
docker-compose up -d --build
```

(Only Nginx, App, Mongo, and Redis will start)

Step 3: Create Super Admin

You cannot register a Super Admin via the API. You must seed it directly into the running container:

```
# Syntax: node scripts/seedAdmin.js "<Name>" "<Phone>" "<Password>"  
docker-compose exec app node scripts/seedAdmin.js "Admin User" "9999999999"  
"StrongPassword123"
```

Switching to Production

To go live, follow these steps:

- Stop the dev stack: `docker-compose down`
- Update `.env`:
 - Change `OTPGATEWAYURL` to your real SMS provider's endpoint.
 - Ensure `ALLOWED_ORIGINS` matches your production domain (e.g., `https://reader.iaf.in`).
- Run without dev profile:

```
docker-compose up -d --build
```

The `vayureader_sms` container will NOT be created.

Debugging & Troubleshooting

Viewing Logs

If something isn't working, the logs are your best friend.

View all logs (streaming):

```
docker-compose logs -f
```

View specific service logs:

```
docker-compose logs -f app
docker-compose logs -f nginx
docker-compose logs -f mongo
```

Accessing Containers (Shell)

To inspect files or run commands inside a container:

Access Backend Shell:

```
docker-compose exec app sh
# Now you are inside the container at /usr/src/app
```

```
ls -la
exit
```

Access Database Shell:

```
docker-compose exec mongo mongosh
# Inside Mongo shell:
# use vayureader
# db.users.find()
```

Common Issues & Solutions

1. 502 Bad Gateway (Nginx)

- Cause: Nginx is running, but the Backend is down or not reachable.
- Fix:
 - Check backend logs: `docker-compose logs app`.
 - If backend is up, restart Nginx to force a reconnect: `docker-compose restart nginx`.

2. CORS Errors (Frontend)

- Cause: The browser blocked the request because the origin isn't allowed.
- Fix:
 - Check `docker-compose.yml`. Ensure `ALLOWED_ORIGINS` includes your frontend URL exactly (e.g., `https://localhost:3000`).
 - Apply changes: `docker-compose up -d`.

3. Upload Failed (413 Payload Too Large)

- Cause: File is larger than Nginx's limit (default 1MB).
- Fix:
 - Edit `nginx/conf.d/default.conf`.
 - Ensure `clientmaxbody_size 50M`; is present.
 - Restart Nginx: `docker-compose restart nginx`.

4. SSL/Https Warnings in Browser

- Cause: Self-signed certificate.
- Fix: This is normal for localhost. Click "Advanced" -> "Proceed to localhost (unsafe)".

5. "MongoNetworkError" or Database Connection Failures

- Cause: The backend started before the database was ready.
- Fix: The app should auto-retry. If not, restart the app: `docker-compose restart app`.

Cheat Sheet: Useful Commands

Action	Command
Start Dev (with Sim)	<code>docker-compose --profile dev</code>
Start Prod (No Sim)	<code>docker-compose up -d --build</code>
Stop Everything	<code>docker-compose down</code>
View Logs	<code>docker-compose logs -f</code>
Restart Service	<code>docker-compose restart</code>
Prune (Clean Up)	<code>docker system prune -a (warn</code>

Configuration

This document explains every environment variable and configuration object used by the VayuReader Backend. All variables are loaded from a `.env` file (via `dotenv`). Missing required variables cause the server to abort on startup.

Core Environment Variables

Variable	Required	Default	Description
MONGODB_URI		–	MongoDB connection string. E
REDIS_URL		redis://localhost:6379	Redis connection URL used fo
JWT_SECRET		–	Secret for signing JWTs (min
JWTEXPIRYDAYS		1	Number of days a user JWT is
OTPGATEWAYURL		–	URL of the SMS gateway used
OTPEXPIRYMINUTES		5	OTP validity period in minut
SKIPOTPSSEND		false	Set to true in development t
ALLOWED_ORIGINS		[]	Comma-separated list of orig
RATELIMITWINDOW_MS		900000 (15 min)	Time window for generic API
RATELIMITMAX_REQUESTS		100	Max generic requests per IP
OTPRATELIMIT_MAX		5	Max OTP requests per IP per
NODE_ENV		development	Determines environment (deve
PORT		3000	Port the Express server list
TESTING		false	When true, cookies use SameS

Exported Configuration Objects

The file `src/config/environment.js` aggregates the above variables into convenient objects:

```
module.exports = {  
  server,      // { port, nodeEnv, isDevelopment }  
  database,    // { uri, options }  
  jwt,         // { secret, expiryDays }  
  otp,         // { gatewayUrl, expiryMinutes, skipSend }
```

```
cors,      // { allowedOrigins }
rateLimit, // { windowMs, maxRequests, otpMaxRequests }
redis      // { url }
};
```

These objects are imported throughout the codebase (e.g., `ratelimiter`, `JWT service`, `Redis client`).

RateLimiting Configuration (`apiLimit-config.js`)

A dedicated config file defines granular limits per endpoint:

```
module.exports = {
  default: { windowMs: 15*60*1000, max: 100, message: 'Too many requests' },
  auth: {
    otp: { prefix: 'otp', windowMs: 15*60*1000, max: 5, message: 'Too many OTP
requests' },
    login: { prefix: 'login', windowMs: 60*1000, max: 5, message: 'Too many
login attempts' }
  },
  uploads: {
    dictionary: { prefix: 'upload-dict', windowMs: 60*60*1000, max: 10, message:
'Upload limit exceeded' },
    abbreviation: { prefix: 'upload-abbr', windowMs: 60*60*1000, max: 10,
message: 'Upload limit exceeded' }
  },
  search: { prefix: 'search', windowMs: 60*1000, max: 60, message: 'Search limit
exceeded' }
};
```

The `rateLimiter` middleware consumes this config via `createLimiter`.

Tip: When adding a new endpoint, extend this file and reference the new limiter in the route definition.

Part IV: API Reference

API Endpoints

Complete API reference for VayuReader Backend v2. All routes are prefixed with /api.

Response Format

All endpoints return JSON in this standard format:

```
{
  "success": true|false,
  "data": { ... },           // Present on success
  "message": "...",         // Optional descriptive message
  "errorCode": "ERROR_CODE" // Present on error
}
```

Rate Limiting

Rate limits are enforced per IP address using Redis. Headers returned:

- RateLimit-Limit: Maximum requests allowed
- RateLimit-Remaining: Requests remaining in current window
- RateLimit-Reset: Unix timestamp when limit resets

Authentication Endpoints

POST /auth/login/request-otp

Request an OTP for user login. Creates a new user if the phone number doesn't exist.

Authentication: None (public) Rate Limit: auth.otp (5 requests per 15 minutes)

Request Body:

```
{
  "phone_number": "string (required)", // E.164 format recommended
  "name": "string (required)",         // 2-100 characters
  "deviceId": "string (required)"      // Unique device identifier
}
```

Success Response (200):

```
{
  "success": true,
  "data": {
    "message": "OTP request received",
    "otp": "123456" // Only in dev mode (SKIP_OTP_SEND=true)
  }
}
```

Error Responses:

- 400: Missing required fields, invalid phone number
- 401: User is blocked
- 429: Rate limit exceeded
- OTP is encrypted with deviceId and stored in Redis (5 min TTL)
- If user exists and is blocked, returns 401
- SMS is sent asynchronously (fire-and-forget)

POST /auth/login/verify-otp

Verify OTP and complete login. Issues a lifetime JWT token.

Authentication: None (public) Rate Limit: auth.login (5 requests per 1 minute)

Request Body:

```
{
  "phone_number": "string (required)",
  "otp": "string (required)",          // 6-digit code
  "deviceId": "string (required)"      // Must match OTP request
}
```

Success Response (200):

```
{
  "success": true,
  "data": {
    "user": {
      "id": "507f1f77bcf86cd799439011",
      "name": "John Doe",
      "phone_number": "+1234567890"
    },
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "isNewDevice": false,
    "deviceChanged": true
  },
  "message": "Login successful (device changed)"
}
```

Error Responses:

- 400: Invalid OTP, expired OTP, user not found
- 401: User is blocked
- 429: Rate limit exceeded
- Sets auth_token HTTP-only cookie (100-year expiration)
- Detects device changes and logs to UserAudit
- OTP is deleted from Redis after verification

GET /auth/profile

Get current authenticated user's profile.

Authentication: Required (JWT) Rate Limit: default (100 requests per 15 minutes)

Request Headers:

```
Authorization: Bearer <token>
```

Or cookie: auth_token

Success Response (200):

```
{
```

```
"success": true,
"data": {
  "user": {
    "id": "507f1f77bcf86cd799439011",
    "name": "John Doe",
    "phone_number": "+1234567890"
  }
}
```

Error Responses:

- 401: No token, invalid token, user blocked/deleted
- 404: User not found

POST /auth/logout

Clear authentication cookie.

Authentication: None Rate Limit: default

Success Response (200):

```
{
  "success": true,
  "message": "Logged out successfully"
}
```

Admin Endpoints

POST /admin/login/request-otp

Step 1 of admin 2FA login: verify password and send OTP.

Authentication: None (public) Rate Limit: auth.otp (5 requests per 15 minutes)

Request Body:

```
{
  "contact": "string (required)",    // Admin phone number
  "password": "string (required)"    // Plain text password
}
```

Success Response (200):

```
{
  "success": true,
  "data": {
    "loginToken": "temp-jwt-token",
    "message": "OTP sent to your registered contact",
    "otp": "123456" // Only in dev mode
  }
}
```

Error Responses:

- 400: Missing fields
- 401: Invalid credentials
- 404: Admin not found
- 429: Rate limit exceeded
- Password is verified using bcrypt
- Temporary loginToken (5 min expiry) is issued for OTP verification
- OTP sent via SMS gateway

POST /admin/login/verify-otp

Step 2 of admin 2FA login: verify OTP and complete login.

Authentication: None (requires loginToken from step 1) Rate Limit: auth.login (5 requests per 1 minute)

Request Body:

```
{
  "contact": "string (required)",
  "otp": "string (required)",
  "loginToken": "string (required)" // From request-otp response
}
```

```
}
```

Success Response (200):

```
{
  "success": true,
  "data": {
    "admin": {
      "id": "507f1f77bcf86cd799439011",
      "name": "Admin Name",
      "contact": "+1234567890",
      "isSuperAdmin": true,
      "permissions": ["manage_pdfs", "manage_dictionary", ...]
    },
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
  },
  "message": "Login successful"
}
```

Error Responses:

- 400: Invalid OTP, expired OTP
- 401: Invalid loginToken
- 429: Rate limit exceeded
- Sets admin_token HTTP-only cookie (1-day expiration)
- Logs admin login to AuditLog

GET /admin/me

Get current authenticated admin's profile.

Authentication: Required (Admin JWT) Rate Limit: default

Request Headers:

```
Authorization: Bearer <admin_token>
```

Or cookie: admin_token

Success Response (200):

```
{
```

```
"success": true,
"data": {
  "admin": {
    "id": "507f1f77bcf86cd799439011",
    "name": "Admin Name",
    "contact": "+1234567890",
    "isSuperAdmin": false,
    "permissions": ["manage_pdfs"]
  }
}
```

Error Responses:

- 401: No token, invalid token
- 404: Admin not found

POST /admin/logout

Clear admin authentication cookie.

Authentication: None Rate Limit: default

Success Response (200):

```
{
  "success": true,
  "message": "Logged out successfully"
}
```

PDF Endpoints

GET /pdfs

Search PDFs with optional query parameters.

Authentication: Required (User or Admin JWT) Rate Limit: search (60 requests per 1 minute)

Query Parameters:

- `q` (string, optional): Search term (title, description, category)
- `category` (string, optional): Filter by category
- `page` (number, optional): Page number (default: 1)
- `limit` (number, optional): Results per page (default: 20, max: 100)

```
{
  "success": true,
  "data": {
    "pdfs": [
      {
        "id": "507f1f77bcf86cd799439011",
        "title": "Document Title",
        "description": "Description text",
        "category": "Technical",
        "pdfUrl": "/uploads/abc123/file.pdf",
        "thumbnailUrl": "/uploads/abc123/thumb.jpg",
        "uploadedBy": "Admin Name",
        "views": 42,
        "createdAt": "2024-01-10T10:00:00.000Z"
      }
    ],
    "pagination": {
      "currentPage": 1,
      "totalPages": 5,
      "totalItems": 100,
      "itemsPerPage": 20
    }
  }
}
```

Error Responses:

- 401: Not authenticated
- 429: Rate limit exceeded

GET /pdfs/all

Get all PDFs (paginated, no search).

Authentication: Required (User or Admin JWT) Rate Limit: default

Query Parameters:

- page (number, optional): Page number (default: 1)
 - limit (number, optional): Results per page (default: 20, max: 100)
-

GET /pdfs/categories

Get list of distinct PDF categories.

Authentication: Required (User or Admin JWT) Rate Limit: default

Success Response (200):

```
{
  "success": true,
  "data": {
    "categories": ["Technical", "Medical", "Legal", "Educational"]
  }
}
```

GET /pdfs/:id

Get single PDF by ID and increment view count.

Authentication: Required (User or Admin JWT) Rate Limit: default

URL Parameters:

- id (string, required): MongoDB ObjectId

```
{
  "success": true,
  "data": {
    "pdf": {
      "id": "507f1f77bcf86cd799439011",
      "title": "Document Title",
      "description": "Description text",

```

```
{
  "category": "Technical",
  "pdfUrl": "/uploads/abc123/file.pdf",
  "thumbnailUrl": "/uploads/abc123/thumb.jpg",
  "uploadedBy": "Admin Name",
  "views": 43,
  "createdAt": "2024-01-10T10:00:00.000Z",
  "updatedAt": "2024-01-10T15:30:00.000Z"
}
```

Error Responses:

- 400: Invalid ObjectId format
- 401: Not authenticated
- 404: PDF not found
- View count is incremented atomically
- Logs view to UserAudit if user is authenticated

POST /pdfs/upload

Upload a new PDF document (admin only).

Authentication: Required (Admin JWT with manage_pdfs permission) Rate Limit: uploads.pdf (10 requests per 1 hour)

Request: multipart/form-data

Form Fields:

- pdf (file, required): PDF file (max 50MB)
- thumbnail (file, optional): Image file (JPG/PNG/WEBP, max 50MB)
- title (string, required): PDF title
- description (string, optional): Description
- category (string, required): Category name

```
{
  "success": true,
  "data": {
    "pdf": {
      "id": "507f1f77bcf86cd799439011",
      "title": "New Document",
```

```
"pdfUrl": "/uploads/abc123/file.pdf",
"thumbnailUrl": "/uploads/abc123/thumb.jpg"
},
"message": "PDF uploaded successfully"
}
```

Error Responses:

- 400: Missing required fields, invalid file type
- 401: Not authenticated or insufficient permissions
- 413: File too large
- 429: Rate limit exceeded
- Files stored in uploads// directory
- Publishes PDF_ADDED event to SSE subscribers
- Logs action to AuditLog

PUT /pdfs/:id

Update existing PDF (admin only).

Authentication: Required (Admin JWT with manage_pdfs permission) Rate Limit: default

URL Parameters:

- id (string, required): MongoDB ObjectId
- pdf (file): New PDF file
- thumbnail (file): New thumbnail
- title (string): New title
- description (string): New description
- category (string): New category

```
{
  "success": true,
  "data": {
    "pdf": { /* updated PDF object */ }
  },
  "message": "PDF updated successfully"
}
```

Error Responses:

- 400: Invalid ObjectId
 - 401: Not authenticated or insufficient permissions
 - 404: PDF not found
 - 429: Rate limit exceeded
 - Old files are deleted if replaced
 - Publishes PDF_UPDATED event to SSE
-

DELETE /pdfs/:id

Delete a PDF document (admin only).

Authentication: Required (Admin JWT with manage_pdfs permission) Rate Limit: default

URL Parameters:

- id (string, required): MongoDB ObjectId

```
{
  "success": true,
  "message": "PDF deleted successfully"
}
```

Error Responses:

- 400: Invalid ObjectId
 - 401: Not authenticated or insufficient permissions
 - 404: PDF not found
 - Deletes files from filesystem
 - Publishes PDF_DELETED event to SSE
 - Logs action to AuditLog
-

Dictionary Endpoints

GET /dictionary/word/:word

Look up a word and get related words.

Authentication: None (public) Rate Limit: search

URL Parameters:

- word (string, required): Word to look up

```
{
  "success": true,
  "data": {
    "word": {
      "id": "507f1f77bcf86cd799439011",
      "word": "example",
      "meanings": ["A thing characteristic of its kind"],
      "relatedWords": ["sample", "instance"]
    }
  }
}
```

Error Responses:

- 404: Word not found
- 429: Rate limit exceeded

GET /dictionary/search/:term?

Search dictionary words (case-insensitive, partial match).

Authentication: None (public) Rate Limit: search (60 requests per 1 minute)

URL Parameters:

- term (string, optional): Search term
- page (number, optional): Page number
- limit (number, optional): Results per page

```
{
  "success": true,
  "data": {
    "results": [
      {
        "id": "507f1f77bcf86cd799439011",
        "word": "example",

```

```
    "meanings": ["A thing characteristic of its kind"]
  }
],
"pagination": { /* pagination object */ }
}
```

Notes:

- Results cached in Redis (60s TTL)
- Empty term returns all words (paginated)

GET /dictionary/words

Get first 100 words (for quick preview).

Authentication: Required (User or Admin JWT) Rate Limit: default

Success Response (200):

```
{
  "success": true,
  "data": {
    "words": [ /* array of word objects */ ]
  }
}
```

GET /dictionary/words/all

Get all dictionary words (paginated, admin view).

Authentication: Required (User or Admin JWT) Rate Limit: default

Query Parameters:

- page (number, optional)
- limit (number, optional)

POST /dictionary

Add a new word (admin only).

Authentication: Required (Admin JWT with manage_dictionary permission) Rate Limit: default

Request Body:

```
{
  "word": "string (required)",
  "meanings": ["string"] (required, array),
  "relatedWords": ["string"] (optional, array)
}
```

Success Response (201):

```
{
  "success": true,
  "data": {
    "word": { /* created word object */ }
  },
  "message": "Word added successfully"
}
```

Error Responses:

- 400: Missing fields, duplicate word
- 401: Not authenticated or insufficient permissions

PUT /dictionary/:id

Update a word (admin only).

Authentication: Required (Admin JWT with manage_dictionary permission) Rate Limit: default

URL Parameters:

- id (string, required): MongoDB ObjectId

```
{
  "word": "string",
  "meanings": ["string"],
  "relatedWords": ["string"]
}
```



```
}
```

Success Response (200):

```
{
  "success": true,
  "data": {
    "word": { /* updated word object */ }
  },
  "message": "Word updated successfully"
}
```

Error Responses:

- 400: Invalid ObjectId
- 401: Not authenticated or insufficient permissions
- 404: Word not found

DELETE /dictionary/:id

Delete a word (admin only).

Authentication: Required (Admin JWT with manage_dictionary permission) Rate Limit: default

URL Parameters:

- id (string, required): MongoDB ObjectId

```
{
  "success": true,
  "message": "Word deleted successfully"
}
```

Error Responses:

- 400: Invalid ObjectId
 - 401: Not authenticated or insufficient permissions
 - 404: Word not found
-

POST /dictionary/upload

Bulk upload dictionary from JSON (admin only).

Authentication: Required (Admin JWT with manage_dictionary permission) Rate Limit: uploads.dictionary (10 requests per 1 hour)

Request Body:

```
{
  "words": [
    {
      "word": "example",
      "meanings": ["definition"],
      "relatedWords": ["sample"]
    }
  ]
}
```

Success Response (200):

```
{
  "success": true,
  "data": {
    "imported": 150,
    "skipped": 5,
    "errors": []
  },
  "message": "Bulk upload completed"
}
```

GET /dictionary/export/all

Export all words as JSON (admin only).

Authentication: Required (Admin JWT with manage_dictionary permission) Rate Limit: default

Success Response (200):

```
{
  "success": true,
```

```
"data": {
  "words": [ /* all word objects */ ],
  "count": 1000
}
```

Abbreviation Endpoints

GET /abbreviations

Search abbreviations with optional query.

Authentication: None (public) Rate Limit: search

Query Parameters:

- q (string, optional): Search term
- page (number, optional)
- limit (number, optional)

```
{
  "success": true,
  "data": {
    "abbreviations": [
      {
        "id": "507f1f77bcf86cd799439011",
        "abbreviation": "API",
        "fullForm": "Application Programming Interface",
        "description": "Optional description"
      }
    ],
    "pagination": { /* pagination object */ }
  }
}
```

Notes:

- Results cached in Redis (60s TTL)

GET /abbreviations/all

Get all abbreviations (paginated).

Authentication: Required (User or Admin JWT) Rate Limit: default

Query Parameters:

- page (number, optional)
 - limit (number, optional)
-

GET /abbreviations/:abbr

Look up specific abbreviation.

Authentication: None (public) Rate Limit: search

URL Parameters:

- abbr (string, required): Abbreviation to look up

```
{
  "success": true,
  "data": {
    "abbreviation": {
      "id": "507f1f77bcf86cd799439011",
      "abbreviation": "API",
      "fullForm": "Application Programming Interface",
      "description": "Optional description"
    }
  }
}
```

Error Responses:

- 404: Abbreviation not found
-

POST /abbreviations

Create new abbreviation (admin only).

Authentication: Required (Admin JWT with manage_abbreviations permission) Rate Limit: default

Request Body:

```
{
  "abbreviation": "string (required)",
  "fullForm": "string (required)",
  "description": "string (optional)"
}
```

Success Response (201):

```
{
  "success": true,
  "data": {
    "abbreviation": { /* created object */ }
  },
  "message": "Abbreviation created successfully"
}
```

Error Responses:

- 400: Missing fields, duplicate abbreviation
- 401: Not authenticated or insufficient permissions

PUT /abbreviations/:id

Update abbreviation (admin only).

Authentication: Required (Admin JWT with manage_abbreviations permission) Rate Limit: default

URL Parameters:

- id (string, required): MongoDB ObjectId

```
{
  "abbreviation": "string",
  "fullForm": "string",
  "description": "string (optional)"
}
```

Success Response (200):

```
{
  "success": true,
  "data": {
    "abbreviation": { /* updated object */ }
  },
  "message": "Abbreviation updated successfully"
}
```

DELETE /abbreviations/:id

Delete abbreviation (admin only).

Authentication: Required (Admin JWT with manage_abbreviations permission) Rate Limit: default

URL Parameters:

- id (string, required): MongoDB ObjectId

```
{
  "success": true,
  "message": "Abbreviation deleted successfully"
}
```

POST /abbreviations/bulk

Bulk upload abbreviations (admin only).

Authentication: Required (Admin JWT with manage_abbreviations permission) Rate Limit: uploads.abbreviation (10 requests per 1 hour)

Request Body:

```
{
  "abbreviations": [
    {
      "abbreviation": "API",
      "fullForm": "Application Programming Interface"
    }
  ]
}
```

```
]
}
```

Success Response (200):

```
{
  "success": true,
  "data": {
    "imported": 50,
    "skipped": 2,
    "errors": []
  }
}
```

GET /abbreviations/export/all

Export all abbreviations (admin only).

Authentication: Required (Admin JWT with manage_abbreviations permission) Rate Limit: default

Success Response (200):

```
{
  "success": true,
  "data": {
    "abbreviations": [ /* all objects */ ],
    "count": 500
  }
}
```

Audit Endpoints

GET /audit/logs

Get paginated audit logs with optional filters (admin only).

Authentication: Required (Admin JWT with view_audit permission) Rate Limit: default

Query Parameters:

- page (number, optional): Page number
- limit (number, optional): Results per page
- type (string, optional): Filter by log type (LOGIN, DEVICECHANGE, ADMINACTION)
- userId (string, optional): Filter by user ID
- startDate (ISO string, optional): Filter from date
- endDate (ISO string, optional): Filter to date

```
{
  "success": true,
  "data": {
    "logs": [
      {
        "id": "507f1f77bcf86cd799439011",
        "type": "LOGIN",
        "userId": "507f1f77bcf86cd799439012",
        "userName": "John Doe",
        "phoneNumber": "+1234567890",
        "deviceId": "device-123",
        "metadata": {
          "ip": "192.168.1.1",
          "userAgent": "Mozilla/5.0..."
        },
        "timestamp": "2024-01-10T10:00:00.000Z"
      }
    ],
    "pagination": {
      "currentPage": 1,
      "totalPages": 10,
      "totalItems": 200,
      "itemsPerPage": 20
    }
  }
}
```

Error Responses:

- 401: Not authenticated or insufficient permissions

GET /audit/stats

Get audit statistics (admin only).

Authentication: Required (Admin JWT with view_audit permission) Rate Limit: default

Success Response (200):

```
{
  "success": true,
  "data": {
    "totalLogins": 1500,
    "totalDeviceChanges": 45,
    "totalAdminActions": 230,
    "uniqueUsers": 350,
    "recentActivity": [
      {
        "date": "2024-01-10",
        "logins": 120,
        "deviceChanges": 3
      }
    ]
  }
}
```

SSE Endpoint

GET /events

Establish Server-Sent Events connection for real-time updates.

Authentication: Required (User or Admin JWT) Rate Limit: default

Request Headers:

```
Authorization: Bearer <token>
Accept: text/event-stream
```

Response: Event stream (Content-Type: text/event-stream)

Event Types:

- connected - Initial connection confirmation

```
event: connected
data: {"message":"Connected to event stream"}
```

- PDF_ADDED - New PDF uploaded

```
event: PDF_ADDED
data: {"pdfId":"507f...", "title":"New Document", "uploadedBy":"Admin"}
```

- PDF_UPDATED - PDF modified

```
event: PDF_UPDATED
data: {"pdfId":"507f...", "title":"Updated Document"}
```

- PDF_DELETED - PDF removed

```
event: PDF_DELETED
data: {"pdfId":"507f..."}
```

Notes:

- Connection stays open indefinitely
- Heartbeat comments (:heartbeat) sent every 30s
- Nginx configured with proxy_buffering off for SSE
- Events published via Redis Pub/Sub

```
const evtSource = new EventSource('/api/events', {
  headers: { Authorization: `Bearer ${token}` }
});

evtSource.addEventListener('PDF_ADDED', (e) => {
  const data = JSON.parse(e.data);
  console.log('New PDF:', data.title);
});

evtSource.onerror = () => {
  console.log('Connection lost, reconnecting...');
};
```

Error Codes

Code	Description
RATELIMITEXCEEDED	Too many requests from this
OTPRATELIMIT_EXCEEDED	Too many OTP requests
LOGINRATELIMIT_EXCEEDED	Too many login attempts
INVALID_CREDENTIALS	Wrong password or contact
INVALID_OTP	OTP is incorrect or expired
USER_BLOCKED	User account is blocked
INSUFFICIENT_PERMISSIONS	Admin lacks required permmiss
VALIDATION_ERROR	Request validation failed
NOT_FOUND	Resource not found
DUPLICATE_ENTRY	Resource already exists

Pagination

All paginated endpoints support:

- page: Page number (1-indexed, default: 1)
- limit: Items per page (default: 20, max: 100)

```
{
  "pagination": {
    "currentPage": 1,
    "totalPages": 5,
    "totalItems": 100,
    "itemsPerPage": 20,
    "hasNextPage": true,
    "hasPrevPage": false
  }
}
```

Authentication Headers

User Authentication:

```
Authorization: Bearer <user_jwt_token>
```

Or cookie: auth_token

Admin Authentication:

```
Authorization: Bearer <admin_jwt_token>
```

Or cookie: admin_token

File Upload Limits

- Max file size: 50MB
 - Allowed PDF types: application/pdf
 - Allowed image types: image/jpeg, image/png, image/webp, image/gif
 - Upload directory: uploads//
-

Caching

Cached endpoints (Nginx proxy cache, 5 min TTL):

- /api/dictionary/word/:word
 - /api/dictionary/search/:term
 - /api/abbreviations
 - /api/abbreviations/:abbr
 - X-Cache-Status: HIT, MISS, or BYPASS
 - Dictionary search results
 - Abbreviation search results
-

Authentication

Complete guide to authentication, authorization, and security mechanisms in VayuReader Backend v2.

Overview

The backend implements a dual authentication system:

- User Authentication - OTP-based, device-bound, lifetime JWT tokens
- Admin Authentication - Password + OTP 2FA, session-based JWT tokens

User Authentication Flow

1. OTP Request (POST /api/auth/login/request-otp)

Process:

- Client sends phone_number, name, and deviceId
- Server validates phone number format
- User is created if doesn't exist, or existing user is loaded
- Block check: If user.isBlocked === true, return 401
- OTP is generated (6-digit random number)
- OTP is encrypted with deviceId using AES-256-CBC
- Encrypted OTP stored in Redis with 5-minute TTL
- SMS sent to user's phone number (async, fire-and-forget)
- Response sent to client (includes OTP in dev mode)
- Rate limit: 5 requests per 15 minutes per IP
- Device-bound encryption prevents OTP reuse on different devices
- Redis TTL ensures OTP expires after 5 minutes
- User blocking enforced at request time

```
// src/controllers/auth.controller.js
const requestLoginOtp = async (req, res) => {
  const { phone_number, name, deviceId } = req.body;

  // Find or create user
  let user = await User.findOne({ phone_number });
  if (!user) {
    user = new User({ name, phone_number });
  } else if (user.isBlocked) {
    return response.unauthorized(res, 'User blocked');
  }

  // Generate and encrypt OTP
  const otp = generateOtp(); // 6-digit random
  await saveOtp(phone_number, otp, deviceId); // Encrypt with deviceId

  // Send SMS (async)
  sendOtpSms(phone_number, otp).catch(err => console.error(err));

  return response.success(res, { message: 'OTP sent' });
};
```

2. OTP Verification (POST /api/auth/login/verify-otp)

Process:

- Client sends phone_number, otp, and deviceId
- Server retrieves encrypted OTP from Redis
- OTP is decrypted using deviceId
- If deviceId doesn't match, decryption fails → invalid OTP
- OTP compared with submitted value
- Block check: If user.isBlocked === true, return 401
- Device binding logic:
 - If user.deviceId is empty → new user
 - If user.deviceId !== deviceId → device change (logged to UserAudit)
- User's deviceId and lastLogin updated
- Lifetime JWT token generated (100-year expiration)
- Token set as HTTP-only cookie
- Login logged to UserAudit

- Rate limit: 5 requests per 1 minute per IP
- Device-bound OTP prevents phishing attacks
- Lifetime token eliminates re-authentication UX friction
- Device changes logged for audit trail
- HTTP-only cookies prevent XSS attacks

```
{
  userId: ObjectId,
  phone_number: String,
  deviceId: String,
  name: String,
  type: 'user',
  iat: Number,
  exp: Number // 100 years from now
}
```

Code Flow:

```
const verifyLoginOtp = async (req, res) => {
  const { phone_number, otp, deviceId } = req.body;

  const user = await User.findOne({ phone_number });
  if (!user) return response.badRequest(res, 'User not found');
  if (user.isBlocked) return response.unauthorized(res, 'User blocked');

  // Verify OTP (decrypts with deviceId)
  const verification = await verifyOtp(otp, phone_number, deviceId);
  if (!verification.valid) {
    return response.badRequest(res, verification.error);
  }

  // Delete OTP from Redis
  await deleteOtp(phone_number);

  // Detect device change
  const isDeviceChange = user.deviceId && user.deviceId !== deviceId;
  if (isDeviceChange) {
    logDeviceChange(user, user.deviceId, deviceId);
    user.previousDeviceId = user.deviceId;
  }

  // Update user
  user.deviceId = deviceId;
  user.lastLogin = new Date();
  await user.save();
}
```

```
// Generate lifetime JWT
const token = generateLifetimeUserToken(user._id, {
  deviceId,
  phone_number: user.phone_number,
  name: user.name
});

// Set cookie (100-year expiration)
res.cookie('auth_token', token, {
  httpOnly: true,
  secure: process.env.NODE_ENV === 'production',
  sameSite: 'lax',
  maxAge: 100 * 365 * 24 * 60 * 60 * 1000
});

return response.success(res, { user, token, deviceChanged: isDeviceChange });
};
```

3. Token Validation (Middleware)

Process (src/middleware/auth.js):

- Extract token from Authorization header or auth_token cookie
- Verify JWT signature using JWT_SECRET
- Check token type is 'user'
- Database lookup: Query User by userId from token
- Block check: If user not found or isBlocked === true, return 401
- Attach user info to req.user
- Call next()
- Token signature verification prevents tampering
- Database lookup ensures deleted/blocked users can't access resources
- Async middleware allows DB check without blocking event loop

```
const authenticateUser = async (req, res, next) => {
  try {
    const token = req.cookies.auth_token || req.headers.authorization?.split('
')[1];
    if (!token) return response.unauthorized(res, 'No token');

    const decoded = verifyToken(token);
    if (decoded.type !== 'user') {
```



```
    return response.unauthorized(res, 'Invalid token type');
  }

  // Check if user still exists and is not blocked
  const user = await User.findById(decoded.userId).select('isBlocked');
  if (!user) return response.unauthorized(res, 'User not found');
  if (user.isBlocked) return response.unauthorized(res, 'User blocked');

  req.user = {
    userId: decoded.userId,
    phone_number: decoded.phone_number,
    deviceId: decoded.deviceId,
    name: decoded.name
  };

  next();
} catch (error) {
  return response.unauthorized(res, 'Invalid token');
}
};
```

Admin Authentication Flow

1. Password + OTP Request (POST /api/admin/login/request-otp)

Process:

- Client sends contact and password
- Server finds admin by contact
- Password verified using bcrypt
- If valid, OTP generated and sent via SMS
- Temporary login token issued (5-minute expiration)
- Login token returned to client for OTP verification step
- Rate limit: 5 requests per 15 minutes
- Bcrypt password hashing (10 rounds)
- Temporary token prevents OTP bypass
- 2FA required for all admin logins

```
const requestLoginOtp = async (req, res) => {
  const { contact, password } = req.body;

  const admin = await Admin.findOne({ contact });
  if (!admin) return response.notFound(res, 'Admin not found');

  // Verify password
  const isValid = await bcrypt.compare(password, admin.passwordHash);
  if (!isValid) return response.unauthorized(res, 'Invalid credentials');

  // Generate OTP
  const otp = generateOtp();
  await saveOtp(contact, otp, 'admin'); // No device binding for admins

  // Send SMS
  sendOtpSms(contact, otp).catch(err => console.error(err));

  // Generate temporary login token (5 min expiry)
  const loginToken = jwt.sign(
    { contact, type: 'admin_login' },
    process.env.JWT_SECRET,
    { expiresIn: '5m' }
  );

  return response.success(res, { loginToken, message: 'OTP sent' });
};
```

2. OTP Verification (POST /api/admin/login/verify-otp)

Process:

- Client sends contact, otp, and loginToken
- Server verifies loginToken signature and expiration
- OTP retrieved from Redis and verified
- Admin JWT token generated (1-day expiration)
- Token set as HTTP-only cookie
- Admin login logged to AuditLog
- Rate limit: 5 requests per 1 minute
- Login token prevents OTP verification without password
- Session-based JWT (1-day expiry) for admins

- All admin actions logged

```
{
  adminId: ObjectId,
  contact: String,
  name: String,
  isSuperAdmin: Boolean,
  permissions: [String],
  type: 'admin',
  iat: Number,
  exp: Number // 1 day from now
}
```

3. Admin Middleware

Process (src/middleware/adminAuth.js):

- Extract token from Authorization header or admin_token cookie
- Verify JWT signature
- Check token type is 'admin'
- Attach admin info to req.admin
- Call next()

```
const requirePermission = (permission) => {
  return (req, res, next) => {
    if (req.admin.isSuperAdmin) return next(); // Super admin bypass
    if (!req.admin.permissions.includes(permission)) {
      return response.forbidden(res, 'Insufficient permissions');
    }
    next();
  };
};
```

Usage:

```
router.post('/pdfs/upload',
  authenticateAdmin,
  requirePermission('manage_pdfs'),
  pdfController.uploadPdf
);
```

OTP Service (src/services/otp.service.js)

Encryption

Algorithm: AES-256-CBC Key Derivation: PBKDF2 from deviceId (or 'admin' for admins) IV: Random 16 bytes per OTP

Why Device-Bound Encryption?

- Prevents OTP interception and replay on different devices
- Even if Redis is compromised, OTPs are useless without correct deviceId
- Adds layer of defense against phishing attacks

```
const crypto = require('crypto');

const encryptOtp = (otp, deviceId) => {
  const key = crypto.pbkdf2Sync(deviceId, 'salt', 100000, 32, 'sha256');
  const iv = crypto.randomBytes(16);
  const cipher = crypto.createCipheriv('aes-256-cbc', key, iv);

  let encrypted = cipher.update(otp, 'utf8', 'hex');
  encrypted += cipher.final('hex');

  return iv.toString('hex') + ':' + encrypted;
};

const decryptOtp = (encryptedData, deviceId) => {
  const [ivHex, encrypted] = encryptedData.split(':');
  const key = crypto.pbkdf2Sync(deviceId, 'salt', 100000, 32, 'sha256');
  const iv = Buffer.from(ivHex, 'hex');
  const decipher = crypto.createDecipheriv('aes-256-cbc', key, iv);

  let decrypted = decipher.update(encrypted, 'hex', 'utf8');
  decrypted += decipher.final('utf8');

  return decrypted;
};
```

Redis Storage

Key Format: otp: Value: Encrypted OTP string TTL: 5 minutes (300 seconds)

Code:

```
const saveOtp = async (phone_number, otp, deviceId) => {
  const encrypted = encryptOtp(otp, deviceId);
  await redisClient.setEx(`otp:${phone_number}`, 300, encrypted);
};

const verifyOtp = async (otp, phone_number, deviceId) => {
  const encrypted = await redisClient.get(`otp:${phone_number}`);
  if (!encrypted) {
    return { valid: false, error: 'OTP expired or not found' };
  }

  try {
    const decrypted = decryptOtp(encrypted, deviceId);
    if (decrypted !== otp) {
      return { valid: false, error: 'Invalid OTP' };
    }
    return { valid: true };
  } catch (error) {
    return { valid: false, error: 'Invalid OTP or device' };
  }
};
```

JWT Service (src/services/jwt.service.js)

Token Generation

User Token (Lifetime):

```
const generateLifetimeUserToken = (userId, payload) => {
  return jwt.sign(
    {
      userId,
      ...payload,
      type: 'user'
    },
    process.env.JWT_SECRET,
    { expiresIn: '100y' } // Effectively lifetime
  );
};
```

Admin Token (Session):

```
const generateAdminToken = (admin) => {
  return jwt.sign(
    {
      adminId: admin._id,
      contact: admin.contact,
      name: admin.name,
      isSuperAdmin: admin.isSuperAdmin,
      permissions: admin.isSuperAdmin ? PERMISSIONS : admin.permissions,
      type: 'admin'
    },
    process.env.JWT_SECRET,
    { expiresIn: '1d' }
  );
};
```

Token Verification

```
const verifyToken = (token) => {
  try {
    return jwt.verify(token, process.env.JWT_SECRET);
  } catch (error) {
    throw new Error('Invalid token');
  }
};
```

Security Best Practices

1. Password Storage

- Never store plaintext passwords
- Use bcrypt with 10+ rounds
- Validate password strength on creation

```
const bcrypt = require('bcryptjs');
```

```
// Hash password
const passwordHash = await bcrypt.hash(password, 10);

// Verify password
const isValid = await bcrypt.compare(password, admin.passwordHash);
```

2. JWT Secrets

- Minimum 32 characters (enforced in environment.js)
- Use cryptographically random strings
- Rotate secrets periodically in production
- Store in environment variables, never in code

3. Cookie Security

Production Settings:

```
res.cookie('auth_token', token, {
  httpOnly: true,          // Prevents XSS access
  secure: true,            // HTTPS only
  sameSite: 'strict',     // CSRF protection
  maxAge: 100 * 365 * 24 * 60 * 60 * 1000
});
```

Development Settings:

```
res.cookie('auth_token', token, {
  httpOnly: true,
  secure: false,          // Allow HTTP for localhost
  sameSite: 'lax',       // Allow cross-origin for testing
  maxAge: 100 * 365 * 24 * 60 * 60 * 1000
});
```

4. Rate Limiting

All authentication endpoints are rate-limited:

- OTP requests: 5 per 15 minutes
- Login attempts: 5 per 1 minute
- General API: 100 per 15 minutes

5. Input Validation

All inputs are validated and sanitized:

```
// Phone number validation
const validatePhoneNumber = (req, res, next) => {
  const { phone_number } = req.body;
  const phoneRegex = /^\\+?[1-9]\\d{1,14}$/; // E.164 format
  if (!phoneRegex.test(phone_number)) {
    return response.badRequest(res, 'Invalid phone number');
  }
  next();
};

// Sanitization
const sanitizePhone = (phone) => phone.trim().replace(/\\s/g, '');
const sanitizeName = (name) => name.trim().substring(0, 100);
```

6. CORS Configuration

```
const cors = require('cors');

app.use(cors({
  origin: process.env.ALLOWED_ORIGINS.split(','),
  credentials: true, // Allow cookies
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorization']
}));
```

7. Helmet Security Headers

```
const helmet = require('helmet');

app.use(helmet({
  contentSecurityPolicy: false, // Disabled for API
  crossOriginEmbedderPolicy: false
}));
```


User Blocking

Block User

```
node scripts/manageUsers.js user block --contact +1234567890
```

Effect:

- User cannot request OTP
- User cannot verify OTP
- Existing tokens become invalid (checked in middleware)

Unblock User

```
node scripts/manageUsers.js user unblock --contact +1234567890
```

Admin Permissions

Permission Matrix

Permission	Description	Endpoints
manage_pdfs	Create, update, delete PDFs	/api/pdfs/* (POST, PUT, DELETE)
manage_dictionary	Manage dictionary words	/api/dictionary/* (POST, PUT)
manage_abbreviations	Manage abbreviations	/api/abbreviations/* (POST, PUT)
manage_admins	Create/delete sub-admins	/api/admin/sub-admins/* (sup)
view_audit	View audit logs	/api/audit/*

Super Admin

- Has all permissions automatically
 - Can create/delete other admins
-

- Cannot be deleted via API (only via script)
-

Audit Logging

User Actions

Logged to UserAudit collection:

- LOGIN - Every successful login
- DEVICE_CHANGE - When deviceId changes
- READ_PDF - When user views a PDF

Admin Actions

Logged to AuditLog collection:

- CREATE - PDF, word, abbreviation, admin created
 - UPDATE - Resource updated
 - DELETE - Resource deleted
-

Security Checklist

- ☐ JWT_SECRET is at least 32 characters
 - ☐ HTTPS enabled in production
 - ☐ Cookies use secure: true in production
 - ☐ Rate limiting enabled on all auth endpoints
 - ☐ CORS configured with specific origins
 - ☐ Helmet security headers enabled
 - ☐ Input validation on all endpoints
 - ☐ Passwords hashed with bcrypt (10+ rounds)
 - ☐ OTPs encrypted with device binding
 - ☐ User blocking enforced in middleware
-

- [] Admin permissions checked on protected routes
 - [] Audit logging enabled for all admin actions
 - [] Redis password protected
 - [] MongoDB authentication enabled
 - [] Environment variables secured (not in git)
-

Troubleshooting

"Invalid OTP" Error

Cause: Device ID mismatch between request-otp and verify-otp Solution: Ensure client sends same deviceId in both requests

"User blocked" Error

Cause: User's isBlocked field is true Solution: Unblock user via script or admin panel

"Token expired" Error (Admin)

Cause: Admin JWT expired after 1 day Solution: Re-login to get new token

"Insufficient permissions" Error

Cause: Admin lacks required permission Solution: Grant permission or use super admin account

Data Models

Complete reference for all Mongoose schemas in VayuReader Backend v2.

Overview

The application uses MongoDB with Mongoose ODM. All models include:

- Automatic timestamps (createdAt, updatedAt)
 - Indexes for query optimization
 - Validation rules
 - Instance methods where applicable
-

User Model

File: src/models/User.js Collection: users

Represents application users who authenticate via OTP.

Schema

Field	Type	Required	Unique	Index	Description
name	String				User's full name (2-100 char
phone_number	String				Phone number (unique identif
deviceId	String				Current device ID bound to u
previousDeviceId	String				Previous device ID (audit tr
lastLogin	Date				Last login timestamp
isBlocked	Boolean				Whether user is blocked (def
createdAt	Date	Auto			Account creation timestamp
updatedAt	Date	Auto			Last update timestamp

Indexes

```
{ phone_number: 1 } // Unique index
{ deviceId: 1 }     // Non-unique index
```

Instance Methods

toSafeObject()

Returns sanitized user object for API responses (excludes sensitive fields).

```
{
  id: ObjectId,
  name: String,
  phone_number: String
}
```

Usage Example

```
const User = require('./models/User');

// Create user
const user = new User({
  name: 'John Doe',
  phone_number: '+1234567890'
});
await user.save();

// Find by phone
const user = await User.findOne({ phone_number: '+1234567890' });

// Block user
user.isBlocked = true;
await user.save();
```

Notes

- phone_number should be stored in E.164 format
- deviceId is set during OTP verification

- Device changes are logged to UserAudit
- Blocked users cannot login (checked in auth middleware)

Admin Model

File: src/models/Admin.js Collection: admins

Represents administrators with role-based permissions.

Schema

Field	Type	Required	Unique	Index	Description
name	String				Admin's full name (2-100 cha
contact	String				Contact number (unique ident
isSuperAdmin	Boolean				Super admin privileges (defa
permissions	[String]				Array of permission strings
createdBy	String				Who created this admin (defa
passwordHash	String				Bcrypt hashed password
createdAt	Date	Auto			Account creation timestamp
updatedAt	Date	Auto			Last update timestamp

Permissions

Available permissions (defined in PERMISSIONS constant):

- manage_pdfs - Create, update, delete PDFs
- manage_dictionary - Manage dictionary words
- manage_abbreviations - Manage abbreviations
- manage_admins - Create/delete sub-admins (super admin only)
- view_audit - View audit logs

Indexes

```
{ contact: 1 } // Unique index
```

```
{ name: 1 } // Non-unique index
```

Instance Methods

hasPermission(permission)

Check if admin has a specific permission. Super admins always return true.

```
if (admin.hasPermission('manage_pdfs')) {  
  // Allow PDF operations  
}
```

toSafeObject()

Returns sanitized admin object (excludes passwordHash).

```
{  
  id: ObjectId,  
  name: String,  
  contact: String,  
  isSuperAdmin: Boolean,  
  permissions: [String] // All permissions if super admin  
}
```

Usage Example

```
const Admin = require('./models/Admin');  
const bcrypt = require('bcryptjs');  
  
// Create admin  
const passwordHash = await bcrypt.hash('password123', 10);  
const admin = new Admin({  
  name: 'Admin User',  
  contact: '+1234567890',  
  passwordHash,  
  permissions: ['manage_pdfs', 'manage_dictionary']  
});  
await admin.save();  
  
// Verify password  
const isValid = await bcrypt.compare(password, admin.passwordHash);
```

```
// Check permission
if (admin.hasPermission('manage_pdfs')) {
  // Allowed
}
```

Notes

- Super admins have all permissions automatically
- Password must be hashed with bcrypt (10 rounds)
- Admin login requires 2FA (password + OTP)

PdfDocument Model

File: src/models/PdfDocument.js Collection: pdfdocuments

Represents PDF documents uploaded to the system.

Schema

Field	Type	Required	Unique	Index	Description
title	String				PDF title (1-200 chars)
description	String				Optional description (max 10
category	String				Category name
pdfUrl	String				Relative path to PDF file
thumbnailUrl	String				Relative path to thumbnail
uploadedBy	ObjectId				Reference to Admin who uploa
views	Number				View count (default: 0)
createdAt	Date	Auto			Upload timestamp
updatedAt	Date	Auto			Last update timestamp

Indexes


```
{ title: 1 }
{ category: 1 }
{ uploadedBy: 1 }
{ createdAt: -1 }
{ title: 'text', description: 'text', category: 'text' } // Full-text search
```

Instance Methods

None (uses static queries).

Usage Example

```
const PdfDocument = require('./models/PdfDocument');

// Create PDF
const pdf = new PdfDocument({
  title: 'Technical Manual',
  description: 'System documentation',
  category: 'Technical',
  pdfUrl: '/uploads/abc123/file.pdf',
  thumbnailUrl: '/uploads/abc123/thumb.jpg',
  uploadedBy: adminId
});
await pdf.save();

// Increment views
await PdfDocument.findByIdAndUpdate(
  pdfId,
  { $inc: { views: 1 } },
  { new: true }
);

// Search PDFs
const results = await PdfDocument.find({
  $text: { $search: 'technical manual' }
});
```

Notes

- Files stored in uploads// directory
- pdfUrl and thumbnailUrl are relative paths

- View count incremented on each GET request
- Deletions trigger file cleanup

Word Model

File: src/models/Word.js Collection: words

Represents dictionary words with meanings and related terms.

Schema

Field	Type	Required	Unique	Index	Description
word	String				The word (stored uppercase)
meanings	[Meaning]				Array of meaning objects
synonyms	[String]				Word-level synonyms
antonyms	[String]				Word antonyms
createdAt	Date	Auto			Creation timestamp
updatedAt	Date	Auto			Last update timestamp

Meaning Sub-Schema

Field	Type	Required	Description
partOfSpeech	String		Part of speech (noun, verb,
definition	String		The definition
synonyms	[String]		Meaning-specific synonyms
examples	[String]		Usage examples

Indexes

```
{ word: 1 } // Unique index
```

Validation

- At least one meaning required
- Word automatically converted to uppercase

Usage Example

```
const Word = require('./models/Word');

// Create word
const word = new Word({
  word: 'example',
  meanings: [
    {
      partOfSpeech: 'noun',
      definition: 'A thing characteristic of its kind',
      synonyms: ['sample', 'instance'],
      examples: ['This is an example sentence']
    }
  ],
  synonyms: ['sample'],
  antonyms: []
});
await word.save();

// Find word (case-insensitive)
const word = await Word.findOne({ word: 'EXAMPLE' });
```

Notes

- Words stored in uppercase for consistency
- Supports multiple meanings per word
- Synonyms can be at word or meaning level

Abbreviation Model

File: src/models/Abbreviation.js Collection: abbreviations

Represents abbreviations and their full forms.

Schema

Field	Type	Required	Unique	Index	Description
abbreviation	String				The abbreviation (stored upper case)
fullForm	String				Full form/expansion
createdAt	Date	Auto			Creation timestamp
updatedAt	Date	Auto			Last update timestamp

Indexes

```
{ abbreviation: 1 } // Unique index
{ abbreviation: 'text', fullForm: 'text' } // Full-text search
```

Usage Example

```
const Abbreviation = require('./models/Abbreviation');

// Create abbreviation
const abbr = new Abbreviation({
  abbreviation: 'API',
  fullForm: 'Application Programming Interface'
});
await abbr.save();

// Search
const results = await Abbreviation.find({
  $text: { $search: 'application' }
});
```

Notes

- Abbreviations stored in uppercase
- Full-text search on both fields

UserAudit Model

File: src/models/UserAudit.js Collection: useraudits

Records user activities for auditing and analytics.

Schema

Field	Type	Required	Unique	Index	Description
userId	ObjectId				Reference to User
phone_number	String				Denormalized phone number
action	String				Action type (enum)
deviceId	String				Device ID
metadata	Mixed				Additional action details
timestamp	Date				Action timestamp (default: n
createdAt	Date	Auto			Record creation
updatedAt	Date	Auto			Last update

Action Types

```
USER_ACTIONS = {
  LOGIN: 'LOGIN',
  DEVICE_CHANGE: 'DEVICE_CHANGE',
  READ_PDF: 'READ_PDF'
}
```

Metadata Structure

LOGIN:

```
{
  ip: '192.168.1.1',
  userAgent: 'Mozilla/5.0...'
}
```

DEVICE_CHANGE:

```
{
  previousDeviceId: 'old-device-123',
  newDeviceId: 'new-device-456'
}
```

READ_PDF:

```
{
  pdfId: '507f1f77bcf86cd799439011',
  title: 'Document Title'
}
```

Indexes

```
{ timestamp: -1 }
{ userId: 1, action: 1 }
{ phone_number: 1, timestamp: -1 }
{ action: 1, timestamp: -1 }
{ deviceId: 1 }
```

Usage Example

```
const UserAudit = require('./models/UserAudit');
const { USER_ACTIONS } = require('./models/UserAudit');

// Log login
await UserAudit.create({
  userId: user._id,
  phone_number: user.phone_number,
  action: USER_ACTIONS.LOGIN,
  deviceId: 'device-123',
  metadata: {
    ip: req.ip,
    userAgent: req.headers['user-agent']
  }
});

// Query user activity
const logs = await UserAudit.find({
  userId: user._id,
  action: USER_ACTIONS.READ_PDF
}).sort({ timestamp: -1 }).limit(10);
```

Notes

- Separate from admin audit logs
- Phone number denormalized for faster queries
- Indexed for time-series queries

AuditLog Model

File: src/models/AuditLog.js Collection: auditlogs

Records administrative actions for accountability and compliance.

Schema

Field	Type	Required	Unique	Index	Description
action	String				Action type (CREATE/UPDATE/D
resourceType	String				Type of resource affected
resourceId	String				ID of affected resource
adminId	ObjectId				Reference to Admin
adminName	String				Admin name (denormalized)
adminContact	String				Admin contact
details	Mixed				Additional details (old/new
timestamp	Date				Action timestamp (default: n
createdAt	Date	Auto			Record creation
updatedAt	Date	Auto			Last update

Action Types

```
[ 'CREATE', 'UPDATE', 'DELETE' ]
```

Resource Types

```
['PDF', 'DICTIONARY_WORD', 'ABBREVIATION', 'ADMIN']
```

Indexes

```
{ timestamp: -1 }
{ resourceType: 1, resourceId: 1 }
{ adminName: 1 }
{ action: 1, resourceType: 1 }
```

Usage Example

```
const AuditLog = require('./models/AuditLog');

// Log PDF creation
await AuditLog.create({
  action: 'CREATE',
  resourceType: 'PDF',
  resourceId: pdf._id.toString(),
  adminId: admin._id,
  adminName: admin.name,
  adminContact: admin.contact,
  details: {
    title: pdf.title,
    category: pdf.category
  }
});

// Log word update
await AuditLog.create({
  action: 'UPDATE',
  resourceType: 'DICTIONARY_WORD',
  resourceId: word._id.toString(),
  adminId: admin._id,
  adminName: admin.name,
  adminContact: admin.contact,
  details: {
    oldMeanings: oldWord.meanings,
    newMeanings: word.meanings
  }
});

// Query admin actions
const logs = await AuditLog.find({
```



```
adminName: 'Admin User',  
resourceType: 'PDF'  
}).sort({ timestamp: -1 });
```

Notes

- Admin info denormalized for faster queries
- details field stores old/new values for updates
- Indexed for compliance reporting

Common Patterns

Pagination

```
const page = parseInt(req.query.page) || 1;  
const limit = Math.min(parseInt(req.query.limit) || 20, 100);  
const skip = (page - 1) * limit;  
  
const results = await Model.find(query)  
  .skip(skip)  
  .limit(limit)  
  .sort({ createdAt: -1 });  
  
const total = await Model.countDocuments(query);  
  
return {  
  results,  
  pagination: {  
    currentPage: page,  
    totalPages: Math.ceil(total / limit),  
    totalItems: total,  
    itemsPerPage: limit  
  }  
};
```

Full-Text Search

```
// Requires text index
const results = await Model.find({
  $text: { $search: searchTerm }
}, {
  score: { $meta: 'textScore' }
}).sort({ score: { $meta: 'textScore' } });
```

Atomic Updates

```
// Increment view count
await PdfDocument.findByIdAndUpdate(
  pdfId,
  { $inc: { views: 1 } },
  { new: true }
);
```

Index Management

Creating Indexes

Indexes are automatically created when the server starts. To manually ensure indexes:

```
node scripts/optimizeDb.js
```

Viewing Indexes

```
const indexes = await Model.collection.getIndexes();
console.log(indexes);
```

Performance Tips

- Use indexes for frequent queries - All find() queries should use indexed fields
- Compound indexes - Create compound indexes for multi-field queries

- Text indexes - Use for full-text search (one per collection)
 - Avoid regex on non-indexed fields - Use text search instead
 - Monitor slow queries - Enable MongoDB profiling in production
-

Validation

All models use Mongoose validation:

```
// Built-in validators
{
  type: String,
  required: [true, 'Field is required'],
  minlength: [2, 'Too short'],
  maxlength: [100, 'Too long'],
  unique: true,
  trim: true,
  uppercase: true,
  enum: ['VALUE1', 'VALUE2']
}

// Custom validators
{
  validate: {
    validator: function(v) {
      return v && v.length > 0;
    },
    message: 'Custom validation message'
  }
}
```

Timestamps

All models include automatic timestamps:

```
{
```

```
timestamps: true // Adds createdAt and updatedAt
}
```

Access in queries:

```
const recentDocs = await Model.find({
  createdAt: { $gte: new Date(Date.now() - 24*60*60*1000) }
});
```

Part V: Infrastructure

Nginx Configuration

Complete guide to the Nginx reverse proxy configuration for VayuReader Backend v2.

Overview

Nginx serves as a reverse proxy with the following responsibilities:

- SSL/TLS termination - Handles HTTPS connections
 - Proxy caching - Caches read-heavy API responses
 - Static file serving - Serves uploaded PDFs and images directly
 - SSE optimization - Disables buffering for Server-Sent Events
 - Rate limiting - Connection-level limits (complementing application-level limits)
 - Security headers - HSTS, X-Frame-Options, etc.
-

Server Blocks

HTTP Server (Port 80)

Minimal configuration for development/testing. In production, this should redirect to HTTPS.

```
server {  
    listen 80;  
    listen [::]:80;  
    server_name localhost;  
  
    # Proxy all requests to backend  
    location / {  
        proxy_pass http://backend;  
        # ... proxy headers ...  
    }  
}
```

Recommendation: Add HTTP → HTTPS redirect in production:

```
server {  
    listen 80;  
    return 301 https://$host$request_uri;  
}
```

HTTPS Server (Port 443)

Main production server with SSL, caching, and optimizations.

```
server {  
    listen 443 ssl http2;  
    listen [::]:443 ssl http2;  
    server_name localhost;  
  
    client_max_body_size 50M; # Allow 50MB uploads  
  
    # SSL configuration  
    ssl_certificate /etc/nginx/certs/server.crt;  
    ssl_certificate_key /etc/nginx/certs/server.key;  
  
    # ... locations ...  
}
```

SSL/TLS Configuration

Certificate Setup

Development (Self-Signed):

```
# Generate self-signed certificate  
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \  
-keyout nginx/certs/server.key \  
-out nginx/certs/server.crt
```

```
-out nginx/certs/server.crt \  
-subj "/CN=localhost"
```

Production (Let's Encrypt):

```
# Use certbot  
certbot certonly --webroot -w /var/www/html -d yourdomain.com
```

SSL Protocols & Ciphers

```
# Modern compatibility (TLS 1.2 and 1.3 only)  
ssl_protocols TLSv1.2 TLSv1.3;  
ssl_prefer_server_ciphers on;  
ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:...;  
  
# Session caching  
ssl_session_cache shared:SSL:10m;  
ssl_session_timeout 10m;
```

Why TLS 1.2+?

- TLS 1.0 and 1.1 are deprecated (RFC 8996)
- Vulnerable to attacks (BEAST, POODLE)
- PCI DSS requires TLS 1.2+

Security Headers

```
# HSTS (force HTTPS for 1 year)  
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains"  
always;  
  
# Prevent clickjacking  
add_header X-Frame-Options "SAMEORIGIN";  
  
# XSS protection  
add_header X-XSS-Protection "1; mode=block";  
  
# Prevent MIME sniffing  
add_header X-Content-Type-Options "nosniff";
```


Location Blocks

1. SSE Endpoint (/api/events)

Purpose: Server-Sent Events for real-time PDF updates

```
location /api/events {
    proxy_pass http://backend;
    proxy_http_version 1.1;
    proxy_set_header Connection ''; # Disable keep-alive

    # CRITICAL: Disable buffering for SSE
    proxy_buffering off;
    proxy_cache off;

    # Long timeouts (24 hours)
    proxy_read_timeout 86400s;
    proxy_send_timeout 86400s;

    # Chunked transfer encoding
    chunked_transfer_encoding on;

    # Standard proxy headers
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
```

Why These Settings?

- `proxy_buffering off` - Nginx won't buffer responses, allowing real-time streaming
- `proxy_cache off` - SSE responses must not be cached
- `Connection ''` - Disables keep-alive, required for SSE
- Long timeouts - SSE connections stay open indefinitely
- `chunkedtransferencoding on` - Allows streaming without Content-Length

```
curl -N -H "Authorization: Bearer <token>" https://localhost/api/events
```

2. Cached API Endpoints

Purpose: Cache read-heavy dictionary and abbreviation lookups

```
location ~ ^/api/(dictionary/(word|search)|abbreviations($|/.*)) {
    proxy_pass http://backend;
    proxy_http_version 1.1;

    # Enable proxy cache
    proxy_cache api_cache;
    proxy_cache_valid 200 5m; # Cache successful responses for 5 minutes
    proxy_cache_key "$request_uri";

    # Add cache status header (for debugging)
    add_header X-Cache-Status $upstream_cache_status;

    # Standard proxy headers
    proxy_set_header Connection "";
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
```

Cache Zone Definition (in nginx.conf):

```
proxy_cache_path /var/cache/nginx/api_cache
    levels=1:2
    keys_zone=api_cache:10m
    max_size=100m
    inactive=60m
    use_temp_path=off;
```

Cache Status Values:

- HIT - Response served from cache
- MISS - Response not in cache, fetched from backend
- BYPASS - Cache bypassed (e.g., POST request)
- EXPIRED - Cached response expired, revalidating
- STALE - Serving stale content while revalidating

```
# First request (MISS)
curl -I https://localhost/api/dictionary/word/example
# X-Cache-Status: MISS
```

```
# Second request (HIT)
curl -I https://localhost/api/dictionary/word/example
# X-Cache-Status: HIT
```

Cache Invalidation:

```
# Manual cache clear
docker-compose exec nginx rm -rf /var/cache/nginx/api_cache/*
```

3. General API Proxy

Purpose: Proxy all other API requests (auth, admin, PDFs, etc.)

```
location /api {
    proxy_pass http://backend;
    proxy_http_version 1.1;
    proxy_set_header Connection "";
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Host $host;
    proxy_cache_bypass $http_upgrade;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
```

No Caching - Auth, admin, and write operations should never be cached.

4. Static PDF Files

Purpose: Serve PDFs directly from filesystem with optimizations

```
location ~ ^/uploads/(.*\.pdf)$ {
    alias /usr/src/app/uploads/$1;
    autoindex off; # Disable directory listing

    # Allow byte-range requests (for PDF streaming/seeking)
```

```
add_header Accept-Ranges bytes;

# Long cache (PDFs rarely change)
expires 7d;
add_header Cache-Control "public, immutable";

# Efficient file transfer
sendfile on;
tcp_nopush on;

# Rate limiting (prevent bandwidth hogging)
limit_rate 2m;          # 2 MB/s per connection
limit_rate_after 5m;    # After first 5MB
}
```

Why These Settings?

- `sendfile on` - Zero-copy file transfer (kernel → network, bypassing userspace)
- `tcp_nopush on` - Send full packets (reduces overhead)
- `Accept-Ranges bytes` - Allows PDF viewers to seek/jump pages
- `limit_rate` - Prevents single user from consuming all bandwidth
- `immutable` - Browser won't revalidate (saves requests)

```
# Download PDF
curl -O https://localhost/uploads/abc123/file.pdf

# Range request (download bytes 0-1023)
curl -H "Range: bytes=0-1023" https://localhost/uploads/abc123/file.pdf
```

5. Static Image Files (Thumbnails)

Purpose: Serve thumbnails with aggressive caching

```
location ~ ^/uploads/(.*\.(jpg|jpeg|png|webp|gif))$ {
    alias /usr/src/app/uploads/$1;
    autoindex off;

    # Very long cache (images are immutable)
    expires 30d;
    add_header Cache-Control "public, immutable";
}
```

Why 30 Days?

- Thumbnails never change (UUID filenames)
 - Reduces server load
 - Improves client performance
-

6. Other Static Uploads

Purpose: Catch-all for other uploaded files

```
location /uploads/ {
    root /usr/src/app;
    autoindex off;
    expires 30d;
    add_header Cache-Control "public, no-transform";
}
```

7. Root Location (Fallback)

Purpose: Proxy everything else to backend

```
location / {
    proxy_pass http://backend;
    proxy_http_version 1.1;
    proxy_set_header Connection "";
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Host $host;
    proxy_cache_bypass $http_upgrade;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
```

Upstream Configuration

File: nginx/nginx.conf

```
upstream backend {  
    # Single backend server (for now)  
    server app:3000;  
  
    # Connection pooling  
    keepalive 32;  
}
```

For Load Balancing (multiple backend instances):

```
upstream backend {  
    least_conn; # Route to server with fewest connections  
  
    server app1:3000 weight=1;  
    server app2:3000 weight=1;  
    server app3:3000 weight=1;  
  
    keepalive 64;  
}
```

Performance Tuning

Worker Processes

```
# Auto-detect CPU cores  
worker_processes auto;  
  
# Max connections per worker  
events {  
    worker_connections 1024;  
}
```

Total Capacity: $\text{worker_processes} \times \text{worker_connections} = \text{max concurrent connections}$

Buffers

```
http {  
    # Client body buffer  
    client_body_buffer_size 128k;  
    client_max_body_size 50m;  
  
    # Proxy buffers  
    proxy_buffer_size 4k;  
    proxy_buffers 8 4k;  
    proxy_busy_buffers_size 8k;  
}
```

Timeouts

```
http {  
    # Client timeouts  
    client_body_timeout 12s;  
    client_header_timeout 12s;  
    send_timeout 10s;  
  
    # Keep-alive  
    keepalive_timeout 65s;  
    keepalive_requests 100;  
}
```

Logging

Access Log

```
http {  
    log_format main '$remote_addr - $remote_user [$time_local] "$request" '  
                    '$status $body_bytes_sent "$http_referer" '  
                    '"$http_user_agent" "$http_x_forwarded_for" '  
                    'cache:$upstream_cache_status';
```

```
access_log /var/log/nginx/access.log main;
}
```

Custom Fields:

- \$upstreamcachestatus - Cache hit/miss status
- \$httpxforwarded_for - Original client IP (behind proxy)

Error Log

```
error_log /var/log/nginx/error.log warn;
```

Log Levels: debug, info, notice, warn, error, crit, alert, emerg

Viewing Logs

```
# Access log
docker-compose logs nginx | grep access

# Error log
docker-compose logs nginx | grep error

# Follow logs
docker-compose logs -f nginx
```

Security

Rate Limiting (Connection-Level)

```
http {
    # Define rate limit zones
    limit_req_zone $binary_remote_addr zone=api_limit:10m rate=10r/s;
    limit_req_zone $binary_remote_addr zone=auth_limit:10m rate=5r/s;

    server {
        # Apply to API endpoints
```



```
location /api {
    limit_req zone=api_limit burst=20 nodelay;
    # ...
}

# Stricter limit for auth
location /api/auth {
    limit_req zone=auth_limit burst=5 nodelay;
    # ...
}
}
```

Parameters:

- rate=10r/s - 10 requests per second average
- burst=20 - Allow bursts up to 20 requests
- nodelay - Don't delay requests within burst limit

IP Blacklisting

```
# Block specific IPs
deny 192.168.1.100;
deny 10.0.0.0/8;

# Allow all others
allow all;
```

Geo-Blocking

```
geo $blocked_country {
    default 0;
    CN 1; # Block China
    RU 1; # Block Russia
}

server {
    if ($blocked_country) {
        return 403;
    }
}
```

Monitoring

Nginx Status Page

```
location /nginx_status {
    stub_status on;
    access_log off;
    allow 127.0.0.1; # Only localhost
    deny all;
}
```

Output:

```
Active connections: 42
server accepts handled requests
 1234 1234 5678
Reading: 0 Writing: 1 Waiting: 41
```

Prometheus Exporter

Use nginx-prometheus-exporter:

```
docker run -p 9113:9113 nginx/nginx-prometheus-exporter:latest \
  -nginx.scrape-uri=http://nginx/nginx_status
```

Troubleshooting

502 Bad Gateway

Cause: Backend is down or unreachable Solution:

```
# Check backend status
docker-compose ps app
```

```
# Check backend logs
docker-compose logs app

# Restart backend
docker-compose restart app
```

504 Gateway Timeout

Cause: Backend is slow or hung Solution:

- Increase proxyreadtimeout
- Check backend performance
- Add more backend instances

SSE Not Working

Symptoms: Events not received, connection closes immediately Solution:

- Verify proxy_buffering off in /api/events location
- Check proxyreadtimeout is long enough
- Ensure client sends Accept: text/event-stream

Cache Not Working

Symptoms: X-Cache-Status: BYPASS on all requests Solution:

- Verify cache zone is defined in nginx.conf
- Check cache directory exists and is writable
- Ensure requests are GET (POST/PUT/DELETE bypass cache)

Production Checklist

- ☐ SSL certificate from trusted CA (not self-signed)
 - ☐ HTTP → HTTPS redirect enabled
 - ☐ HSTS header enabled
 - ☐ Security headers configured
 - ☐ Rate limiting enabled
-

- [] Access logs enabled
 - [] Error logs enabled
 - [] Cache directory configured
 - [] clientmaxbody_size set appropriately
 - [] Worker processes set to auto
 - [] Upstream health checks configured (if load balancing)
 - [] Monitoring/alerting configured
 - [] Log rotation configured
-

Docker Integration

Dockerfile

```
FROM nginx:alpine

# Copy custom config
COPY nginx/conf.d/default.conf /etc/nginx/conf.d/default.conf
COPY nginx/nginx.conf /etc/nginx/nginx.conf

# Create cache directory
RUN mkdir -p /var/cache/nginx/api_cache

# Expose ports
EXPOSE 80 443
```

Docker Compose

```
nginx:
  image: nginx:alpine
  ports:
    - "80:80"
    - "443:443"
  volumes:
    - ./nginx/conf.d:/etc/nginx/conf.d
    - ./nginx/nginx.conf:/etc/nginx/nginx.conf
    - ./nginx/certs:/etc/nginx/certs
```

```
- ./uploads:/usr/src/app/uploads:ro
depends_on:
- app
networks:
- backend
```

References

- [Nginx Documentation](#)
- [Nginx SSE Configuration](#)
- [Nginx Caching Guide](#)
- [SSL Best Practices](#)

Rate Limiting

Complete guide to rate limiting configuration and implementation in VayuReader Backend v2.

Overview

The backend implements two-layer rate limiting:

- Application-level (Express + Redis) - Granular per-endpoint limits
 - Connection-level (Nginx) - Broad connection limits
 - Prevent abuse and DDoS attacks
 - Ensure fair resource allocation
 - Protect SMS gateway from spam
 - Maintain system stability under load
-

Application-Level Rate Limiting

Implementation

Library: `express-rate-limit` + `rate-limit-redis` Storage: Redis (distributed across multiple instances)

Configuration: `src/config/apiLimit-config.js`

Architecture

```
Client Request
  ↓
Nginx (connection limit)
  ↓
Express Middleware
  ↓
Rate Limiter Middleware
  ↓
```

```
Redis (check/increment counter)
↓
Controller (if allowed)
```

Configuration File

File: src/config/apiLimit-config.js

```
const apiLimitConfig = {
  // Default limit for all endpoints
  default: {
    windowMs: 15 * 60 * 1000, // 15 minutes
    max: 100,                  // 100 requests
    message: 'Too many requests, please try again later'
  },

  // Authentication endpoints
  auth: {
    otp: {
      prefix: 'otp',
      windowMs: 15 * 60 * 1000, // 15 minutes
      max: 5,                    // 5 OTP requests
      message: 'Too many OTP requests, please try again later'
    },
    login: {
      prefix: 'login',
      windowMs: 60 * 1000,       // 1 minute
      max: 5,                    // 5 login attempts
      message: 'Too many login attempts, please try again later'
    },
    createAdmin: {
      prefix: 'create-admin',
      windowMs: 60 * 60 * 1000, // 1 hour
      max: 5,                    // 5 admin creations
      message: 'Too many admin creation attempts'
    }
  },

  // File uploads
  uploads: {
```

```
dictionary: {
  prefix: 'upload-dict',
  windowMs: 60 * 60 * 1000, // 1 hour
  max: 10,                  // 10 uploads
  message: 'Upload limit exceeded'
},
abbreviation: {
  prefix: 'upload-abbr',
  windowMs: 60 * 60 * 1000, // 1 hour
  max: 10,                  // 10 uploads
  message: 'Upload limit exceeded'
}
},

// Search operations
search: {
  prefix: 'search',
  windowMs: 60 * 1000,      // 1 minute
  max: 60,                  // 60 searches (1/sec average)
  message: 'Search limit exceeded, please slow down'
}
};
```

Configuration Parameters

Parameter	Type	Description
prefix	String	Redis key prefix (must be un
windowMs	Number	Time window in milliseconds
max	Number	Max requests allowed in wind
message	String	Error message when limit exc

Rate Limiter Factory

File: src/middleware/rateLimiter.js

createLimiter Function


```
const createLimiter = ({ prefix, windowMs, max, message }) => {
  return rateLimit({
    windowMs: windowMs || rateLimitConfig.windowMs,
    max: max || rateLimitConfig.maxRequests,
    store: createRedisStore(prefix),
    message: {
      success: false,
      message: message || 'Too many requests, please try again later',
      errorCode: 'RATE_LIMIT_EXCEEDED'
    },
    standardHeaders: true, // RateLimit-* headers
    legacyHeaders: false // X-RateLimit-* headers (deprecated)
  });
};
```

Redis Store

```
const createRedisStore = (prefix) => {
  try {
    return new RedisStore({
      sendCommand: async (...args) => {
        if (!redisClient.isOpen) {
          await connectRedis();
        }
        return redisClient.sendCommand(args);
      },
      prefix: `rl:${prefix}:` // e.g., "rl:otp:192.168.1.1"
    });
  } catch (error) {
    console.warn(`Redis store failed for ${prefix}, using memory store`);
    return undefined; // Falls back to memory store
  }
};
```

Why Redis?

- Distributed rate limiting across multiple server instances
- Persistent counters (survive server restarts)
- Atomic increment operations
- Automatic expiration (TTL)

Pre-Configured Limiters

apiLimiter (Default)

```
const apiLimiter = createLimiter({
  prefix: 'api',
  windowMs: 15 * 60 * 1000,
  max: 100
});
```

Applied to: Most endpoints (unless overridden)

otpLimiter

```
const otpLimiter = createLimiter({
  prefix: 'otp',
  windowMs: 15 * 60 * 1000,
  max: 5,
  message: 'Too many OTP requests, please try again later'
});
```

Applied to:

- POST /api/auth/login/request-otp
- POST /api/admin/login/request-otp
- Prevents SMS gateway abuse
- Limits cost (SMS charges per message)
- Prevents brute-force attacks

loginLimiter

```
const loginLimiter = createLimiter({
  prefix: 'login',
  windowMs: 60 * 1000,
  max: 5,
  message: 'Too many login attempts, please try again later'
});
```

Applied to:

- POST /api/auth/login/verify-otp
 - POST /api/admin/login/verify-otp
 - Prevents OTP brute-force (6-digit = 1M combinations)
 - 5 attempts per minute = 300 attempts per hour max
 - Protects against credential stuffing
-

Usage in Routes

Example: Auth Routes

```
const { otpLimiter, loginLimiter } = require('../middleware/rateLimiter');

// OTP request (5 per 15 min)
router.post('/login/request-otp',
  otpLimiter,
  authController.requestLoginOtp
);

// OTP verification (5 per 1 min)
router.post('/login/verify-otp',
  loginLimiter,
  authController.verifyLoginOtp
);
```

Example: Custom Limiter

```
const { createLimiter } = require('../middleware/rateLimiter');
const apiLimitConfig = require('../config/apiLimit-config');

// Use config from apiLimit-config.js
const bulkUploadLimiter = createLimiter({
  ...apiLimitConfig.uploads.dictionary
});

router.post('/dictionary/upload',
  authenticateAdmin,
  bulkUploadLimiter,
```

```
dictionaryController.uploadDictionary
);
```

Example: Dynamic Limiter

```
// Create limiter on-the-fly
const customLimiter = createLimiter({
  prefix: 'custom-endpoint',
  windowMs: 5 * 60 * 1000, // 5 minutes
  max: 20,
  message: 'Custom limit exceeded'
});

router.post('/custom-endpoint',
  customLimiter,
  controller.customAction
);
```

Response Headers

When rate limit is active, these headers are included:

Standard Headers (RateLimit-*)

```
RateLimit-Limit: 100
RateLimit-Remaining: 95
RateLimit-Reset: 1704902400
```

Header	Description
RateLimit-Limit	Maximum requests allowed in
RateLimit-Remaining	Requests remaining in curren
RateLimit-Reset	Unix timestamp when limit re

When Limit Exceeded

Status Code: 429 Too Many Requests

Response Body:

```
{
  "success": false,
  "message": "Too many requests, please try again later",
  "errorCode": "RATE_LIMIT_EXCEEDED"
}
```

Headers:

```
RateLimit-Limit: 100
RateLimit-Remaining: 0
RateLimit-Reset: 1704902400
Retry-After: 900
```

Redis Key Structure

Key Format

```
rl:<prefix>:<identifier>
```

Examples:

```
rl:otp:192.168.1.100
rl:login:192.168.1.100
rl:api:192.168.1.100
rl:search:192.168.1.100
```

Identifier

By default, the identifier is the client's IP address (req.ip).

Custom Identifier (e.g., by user ID):

```
const createLimiter = ({ prefix, windowMs, max, message }) => {
  return rateLimit({
    // ... other options ...
    keyGenerator: (req) => {
      return req.user?.userId || req.ip; // Use userId if authenticated
    }
  });
};
```

TTL

Keys automatically expire after windowMs:

```
TTL rl:otp:192.168.1.100
# Output: 895 (seconds remaining)
```

Monitoring

View Rate Limit Status

```
# Connect to Redis
docker-compose exec redis redis-cli

# List all rate limit keys
KEYS rl:*

# Get specific key value
GET rl:otp:192.168.1.100

# Get TTL
TTL rl:otp:192.168.1.100
```

Clear Rate Limits

```
# Clear all rate limits
docker-compose exec redis redis-cli FLUSHDB

# Clear specific prefix
docker-compose exec redis redis-cli --eval "return redis.call('del',
unpack(redis.call('keys', 'rl:otp:*')))"

# Clear specific IP
docker-compose exec redis redis-cli DEL rl:otp:192.168.1.100
```

Nginx Connection-Level Rate Limiting

File: nginx/conf.d/default.conf

Configuration

```
http {
    # Define rate limit zones
    limit_req_zone $binary_remote_addr zone=api_limit:10m rate=10r/s;
    limit_req_zone $binary_remote_addr zone=auth_limit:10m rate=5r/s;

    server {
        # Apply to API endpoints
        location /api {
            limit_req zone=api_limit burst=20 nodelay;
            proxy_pass http://backend;
        }

        # Stricter limit for auth
        location /api/auth {
            limit_req zone=auth_limit burst=5 nodelay;
            proxy_pass http://backend;
        }
    }
}
```

Parameters

Parameter	Description
zone=api_limit:10m	Zone name and memory size (1
rate=10r/s	Average rate (10 requests/se
burst=20	Allow bursts up to 20 reques
nodelay	Process burst requests immed

How It Works

- Leaky Bucket Algorithm:
 - Requests fill a "bucket" at rate
 - Bucket capacity is burst
 - Excess requests are rejected
- Example (rate=10r/s, burst=20):
 - Client sends 30 requests instantly
 - First 20 requests accepted (burst)
 - Next 10 requests queued (processed at 10r/s)
 - Last 10 requests rejected (503 error)

Response

Status Code: 503 Service Temporarily Unavailable

Body:

```
<html>
<head><title>503 Service Temporarily Unavailable</title></head>
<body>
<center><h1>503 Service Temporarily Unavailable</h1></center>
</body>
</html>
```

Best Practices

1. Set Appropriate Limits

Too Strict:

- Legitimate users get blocked
- Poor user experience
- Support tickets increase
- Abuse not prevented
- System overload
- Costs increase (SMS, bandwidth)
- Auth endpoints: 5-10 per minute
- Search endpoints: 60-120 per minute
- Upload endpoints: 10-20 per hour
- General API: 100-200 per 15 minutes

2. Use Different Limits for Different Endpoints

```
// Expensive operation (database write)
const uploadLimiter = createLimiter({ max: 10, windowMs: 60*60*1000 });

// Cheap operation (cached read)
const searchLimiter = createLimiter({ max: 60, windowMs: 60*1000 });
```

3. Whitelist Trusted IPs

```
const createLimiter = ({ prefix, windowMs, max, message }) => {
  return ratelimit({
    // ... other options ...
    skip: (req) => {
      const trustedIPs = ['192.168.1.100', '10.0.0.1'];
      return trustedIPs.includes(req.ip);
    }
  });
};
```

4. Use User-Based Limits for Authenticated Endpoints

```
keyGenerator: (req) => {
  return req.user?.userId || req.ip;
}
```

Benefits:

- More accurate tracking
- Prevents IP-based bypass (VPN, proxy)
- Better for mobile users (changing IPs)

5. Log Rate Limit Violations

```
const createLimiter = ({ prefix, windowMs, max, message }) => {
  return rateLimit({
    // ... other options ...
    handler: (req, res) => {
      console.warn(`Rate limit exceeded: ${req.ip} on ${req.path}`);
      res.status(429).json({
        success: false,
        message,
        errorCode: 'RATE_LIMIT_EXCEEDED'
      });
    }
  });
};
```

Troubleshooting

"Too many requests" Error (Legitimate User)

Cause: User exceeded rate limit Solution:

- Check if limit is too strict
- Whitelist user's IP temporarily
- Clear rate limit for that IP:

```
docker-compose exec redis redis-cli DEL rl:api:192.168.1.100
```

Rate Limits Not Working

Symptoms: No rate limiting enforced Causes:

- Redis not connected
- Middleware not applied to route
- Fallback to memory store (not distributed)

```
# Check Redis connection
docker-compose logs redis

# Check middleware order
# Rate limiter must be BEFORE controller
router.post('/endpoint',
  ratelimiter, // ← Must be here
  controller.action
);
```

Different Limits on Different Servers

Cause: Using memory store instead of Redis Solution:

- Ensure Redis is running
- Check Redis connection in logs
- Verify createRedisStore is not falling back

Rate Limit Persists After Window

Cause: Redis key not expiring Solution:

```
# Check TTL
docker-compose exec redis redis-cli TTL rl:api:192.168.1.100

# If -1 (no expiry), manually delete
docker-compose exec redis redis-cli DEL rl:api:192.168.1.100
```

Performance Impact

Redis Overhead

Per Request:

- 1 Redis INCR command (~1ms)
- 1 Redis EXPIRE command (~1ms)
- Total: ~2ms overhead
- Use pipelining for multiple commands
- Use connection pooling
- Monitor Redis latency

Memory Usage

Per IP:

- Key: ~50 bytes
- Value: ~10 bytes
- Total: ~60 bytes per IP
- $10,000 \times 60 \text{ bytes} = 600 \text{ KB}$
- Negligible memory usage

Security Considerations

- Don't rely solely on rate limiting - Use authentication, input validation, etc.
- Monitor for distributed attacks - Single IP limit can be bypassed with botnets
- Use CAPTCHA for critical endpoints - Additional layer for auth, registration
- Implement account lockout - After N failed login attempts
- Log and alert on violations - Detect attack patterns

Testing

Manual Testing

```
# Test OTP limit (5 per 15 min)
```

```
for i in {1..6}; do
  curl -X POST https://localhost/api/auth/login/request-otp \
    -H "Content-Type: application/json" \
    -d '{"phone_number":"+1234567890","name":"Test","deviceId":"test"}'
  echo ""
done

# 6th request should return 429
```

Automated Testing

```
const request = require('supertest');
const app = require('../src/app');

describe('Rate Limiting', () => {
  it('should block after 5 OTP requests', async () => {
    for (let i = 0; i < 5; i++) {
      const res = await request(app)
        .post('/api/auth/login/request-otp')
        .send({ phone_number: '+1234567890', name: 'Test', deviceId: 'test' });
      expect(res.status).toBe(200);
    }

    // 6th request should be blocked
    const res = await request(app)
      .post('/api/auth/login/request-otp')
      .send({ phone_number: '+1234567890', name: 'Test', deviceId: 'test' });
    expect(res.status).toBe(429);
    expect(res.body.errorCode).toBe('RATE_LIMIT_EXCEEDED');
  });
});
```

Production Checklist

- ☐ Redis configured and running
 - ☐ Rate limits set appropriately for each endpoint
 - ☐ Trusted IPs whitelisted (if applicable)
 - ☐ Rate limit violations logged
-

- [] Monitoring/alerting configured
 - [] Nginx connection limits configured
 - [] CAPTCHA implemented for critical endpoints
 - [] Account lockout implemented for failed logins
 - [] Rate limit headers enabled (standardHeaders: true)
 - [] Documentation updated with current limits
-

References

- [express-rate-limit Documentation](#)
- [rate-limit-redis Documentation](#)
- [Nginx Rate Limiting](#)
- [IETF Rate Limit Headers](#)

Caching Strategy

Complete guide to caching layers in VayuReader Backend v2.

Overview

Caching is implemented at multiple layers to ensure low latency and reduced database load:

- Nginx Proxy Cache - Caches HTTP responses for read-heavy endpoints.
 - Application Cache (Redis) - Caches computed results and database queries.
 - Client-Side Cache - Browser caching via HTTP headers.
-

1. Nginx Proxy Cache

Purpose: Caches entire HTTP responses for static or quasi-static data. Location: nginx/conf.d/default.conf

Endpoint	TTL	Cache Key
/api/dictionary/word/:word	5 min	\$request_uri
/api/dictionary/search/:term	5 min	\$request_uri
/api/abbreviations/*	5 min	\$request_uri

Headers:

- X-Cache-Status: Indicates cache hit/miss (HIT, MISS, BYPASS).
-

2. Application Cache (Redis)

Purpose: Stores expensive query results and frequent lookups. Implementation: src/services/search.service.js, src/services/otp.service.js

Dictionary & Abbreviations

Search results are cached in Redis to avoid hitting Elasticsearch or MongoDB repeatedly for the same query.

- Key Format: search:dict: or search:abbr:
- TTL: 60 seconds (configurable)
- Structure: JSON stringified array of results

OTP Storage

OTPs are strictly cached in Redis.

- Key Format: otp:
- TTL: 5 minutes
- Data: Encrypted OTP string (device-bound)

Implementation Pattern

```
const getCachedData = async (key) => {  
  // 1. Check Redis  
  const cached = await redisClient.get(key);  
  if (cached) return JSON.parse(cached);  
  
  // 2. Fetch from Source (DB/ES)  
  const data = await fetchDataFromDb();  
  
  // 3. Store in Redis  
  await redisClient.setEx(key, 60, JSON.stringify(data));  
  
  return data;  
};
```

3. Client-Side Caching

Purpose: Reduces network requests for static assets. Location: Nginx configuration

Asset Type	TTL	Headers
PDFs (.pdf)	7 days	Cache-Control: public, immut
Images (.jpg, .png)	30 days	Cache-Control: public, immut
Other Uploads	30 days	Cache-Control: public, no-tr

Note: API responses generally default to no-cache unless served via the specific cached Nginx locations.

Redis Configuration

Service: src/config/redis.js

- Client: Uses redis npm package.
- Connection: Single shared connection for general operations with auto-reconnect.
- Pub/Sub: Dedicated client instance for subscriptions (required by Redis protocol).

Best Practices

- Always use TTL: Never store keys indefinitely (except perhaps for configuration flags).
- Handle Failures: Application should fallback gracefully to DB if Redis is down.
- Namespace Keys: Use colons (e.g., user:123, cache:search:term) to organize keys.

Monitoring

Use docker-compose exec redis redis-cli to inspect cache:

- INFO memory: Check memory usage.
- INFO stats: Check cache hits/misses.
- DBSIZE: Total number of keys.

Elasticsearch

Guide to the search infrastructure powered by Elasticsearch.

Overview

Elasticsearch (ES) provides high-performance fuzzy search and autocomplete functionality for:

- Dictionary Words - Meanings, synonyms, antonyms.
 - Abbreviations - Full form expansion.
-

Configuration

File: `src/config/elasticsearch.js`

- Client: `@elastic/elasticsearch`
 - Connection: Connects to the ES node defined in `ELASTICSEARCH_NODE` env var.
 - Indices:
 - `vayu_words`: Dictionary data.
 - `vayu_abbreviations`: Abbreviation data.
-

Synchronization

1. Real-time Sync (Application Layer)

The `search.service.js` handles real-time sync. Whenever an admin creates, updates, or deletes a word/abbreviation, the change is immediately propagated to ES.

- Create/Update: `indexWord(doc)`, `indexAbbreviation(doc)`
-

- Delete: `deleteWord(id)`, `deleteAbbreviation(id)`

2. Bulk Sync (CLI Script)

Script: `scripts/syncElasticsearch.js`

Used to:

- Populate ES from scratch.
- Re-index data after schema changes.
- Fix consistency issues.

```
node scripts/syncElasticsearch.js
```

Features:

- Checks MongoDB connection.
- Ensures ES indices exist.
- Processes documents in batches of 500.
- Uses ES Bulk API for performance.

Search Logic

Service: `src/services/search.service.js`

Word Search Strategy

The search query uses a bool query with multiple should clauses to rank results:

- Exact Keyword Match (Boost: 10) - Matches exact uppercase word.
- Prefix Match (Boost: 5) - Word starts with query.
- Fuzzy Match (Boost: 2) - Handles typos (fuzziness: AUTO).
- Standard Match (Boost: 1) - Text analysis match.

Abbreviation Search Strategy

- Exact Abbreviation (Boost: 10)
- Fuzzy Abbreviation (Boost: 5)

- Full Form Match (Boost: 2) - Search within the description/expansion.

Data Mapping

Words Index (vayu_words):

```
{
  "word": { "type": "text", "fields": { "keyword": { "type": "keyword" } } },
  "meanings": { "type": "object" },
  "synonyms": { "type": "text" },
  "antonyms": { "type": "text" },
  "mongoId": { "type": "keyword" }
}
```

Abbreviations Index (vayu_abbreviations):

```
{
  "abbreviation": { "type": "text", "fields": { "keyword": { "type": "keyword" } } },
},
  "fullForm": { "type": "text" },
  "mongoId": { "type": "keyword" }
}
```

Troubleshooting

Connectivity

Check if ES is running:

```
curl http://localhost:9200
```

Script Errors

If syncElasticsearch.js fails:

- Ensure ES container is up (docker-compose ps).
- Check network connectivity (backend can reach ES).
- Check MongoDB path/data.

Search Returns No Results

- Verify data exists in MongoDB.
- Run the sync script to ensure ES is populated.
- Check search.service.js logs for connection timeouts.

Part VI: Services & Scripts

Services Documentation

Complete reference for all service modules in VayuReader Backend v2.

Overview

Services encapsulate business logic and are used by controllers. They are pure functions/classes that don't depend on Express, making them easily testable.

Location: `src/services/`

JWT Service

File: `src/services/jwt.service.js`

Purpose

Sign and verify JSON Web Tokens for users and admins.

Functions

`generateLifetimeUserToken(userId, payload)`

Generate a lifetime JWT token for users (100-year expiration).

Parameters:

- `userId` (ObjectId) - User's MongoDB ID
- `payload` (Object) - Additional data (`phone_number`, `deviceId`, `name`)

```
const token = generateLifetimeUserToken(user._id, {  
  phone_number: user.phone_number,  
  deviceId: 'device-123',
```

```
name: user.name
});
```

Token Payload:

```
{
  userId: "507f1f77bcf86cd799439011",
  phone_number: "+1234567890",
  deviceId: "device-123",
  name: "John Doe",
  type: "user",
  iat: 1704902400,
  exp: 4860902400 // 100 years later
}
```

generateAdminToken(admin)

Generate a session JWT token for admins (1-day expiration).

Parameters:

- admin (Object) - Admin document

```
const token = generateAdminToken(admin);
```

Token Payload:

```
{
  adminId: "507f1f77bcf86cd799439011",
  contact: "+1234567890",
  name: "Admin User",
  isSuperAdmin: true,
  permissions: ["manage_pdfs", "manage_dictionary", ...],
  type: "admin",
  iat: 1704902400,
  exp: 1704988800 // 1 day later
}
```

verifyToken(token)

Verify and decode a JWT token.

Parameters:

- token (String) - JWT string


```
try {
  const decoded = verifyToken(token);
  console.log(decoded.userId);
} catch (error) {
  console.error('Invalid token');
}
```

OTP Service

File: src/services/otp.service.js

Purpose

Generate, encrypt, store, and verify OTPs using Redis.

Functions

generateOtp()

Generate a random 6-digit OTP.

Returns: String (6 digits)

Example:

```
const otp = generateOtp(); // "123456"
```

saveOtp(phone_number, otp, deviceId)

Encrypt OTP with deviceId and store in Redis.

Parameters:

- phone_number (String) - User's phone number
- otp (String) - 6-digit OTP
- deviceId (String) - Device identifier

```
await saveOtp('+1234567890', '123456', 'device-123');
```

Redis Key: otp:+1234567890 TTL: 5 minutes (300 seconds)

verifyOtp(otp, phone_number, deviceId)

Verify OTP by decrypting with deviceId.

Parameters:

- otp (String) - Submitted OTP
- phone_number (String) - User's phone number
- deviceId (String) - Device identifier

```
{
  valid: true|false,
  error: "Error message" // if invalid
}
```

Example:

```
const result = await verifyOtp('123456', '+1234567890', 'device-123');
if (result.valid) {
  // OTP is correct
} else {
  console.error(result.error);
}
```

deleteOtp(phone_number)

Delete OTP from Redis after successful verification.

Parameters:

- phone_number (String)

```
await deleteOtp('+1234567890');
```

shouldSkipSend()

Check if SMS sending should be skipped (dev mode).

Returns: Boolean

Example:

```
if (shouldSkipSend()) {  
  console.log('Dev mode: OTP not sent via SMS');  
}
```

SMS Service

File: src/services/sms.service.js

Purpose

Send OTPs via external SMS gateway.

Functions

sendOtpSms(phone_number, otp)

Send OTP via SMS gateway (async, fire-and-forget).

Parameters:

- phone_number (String)
- otp (String)

```
sendOtpSms('+1234567890', '123456').catch(err => {  
  console.error('SMS failed:', err);  
});
```

Gateway Configuration:

```
const gatewayUrl = process.env.OTP_GATEWAY_URL;  
const message = `Your OTP is: ${otp}`;  
  
await axios.post(gatewayUrl, {  
  to: phone_number,  
  message  
});
```

User Audit Service

File: src/services/userAudit.service.js

Purpose

Log user activities (login, device change, PDF reads).

Functions

logLogin(user, deviceId)

Log successful user login.

Parameters:

- user (Object) - User document
- deviceId (String)

```
await logLogin(user, 'device-123');
```

Creates:

```
{
  userId: user._id,
  phone_number: user.phone_number,
  action: 'LOGIN',
  deviceId: 'device-123',
  metadata: {
    ip: req.ip,
    userAgent: req.headers['user-agent']
  },
  timestamp: new Date()
}
```

logDeviceChange(user, oldDeviceId, newDeviceId)

Log device change event.

Parameters:

- user (Object)

- oldDeviceId (String)
- newDeviceId (String)

```
await logDeviceChange(user, 'old-device', 'new-device');
```

logPdfRead(user, pdf)

Log PDF view event.

Parameters:

- user (Object)
- pdf (Object) - PDF document

```
await logPdfRead(user, pdf);
```

Search Service

File: src/services/search.service.js

Purpose

Centralized search logic with Redis caching.

Functions

searchDictionary(term, options)

Search dictionary words with caching.

Parameters:

- term (String) - Search term
- options (Object) - { page, limit }

```
{
  results: [Word],
  pagination: { ... }
```

```
}
```

Caching: 60 seconds TTL

Example:

```
const results = await searchDictionary('example', { page: 1, limit: 20 });
```

searchAbbreviation(term, options)

Search abbreviations with caching.

Parameters:

- term (String)
- options (Object)

```
const results = await searchAbbreviation('API', { page: 1, limit: 20 });
```

SSE Service

File: src/services/sse.service.js

Purpose

Publish/subscribe events via Redis Pub/Sub for Server-Sent Events.

Functions

publishEvent(channel, data)

Publish event to Redis channel.

Parameters:

- channel (String) - Channel name (pdf.events, audit.events)
- data (Object) - Event data

```
await publishEvent('pdf.events', {
  type: 'PDF_ADDED',
  payload: {
    pdfId: pdf._id,
    title: pdf.title,
    uploadedBy: admin.name
  }
});
```

subscribe(channel, callback)

Subscribe to Redis channel.

Parameters:

- channel (String)
- callback (Function) - Called with (message) on each event

```
await subscribe('pdf.events', (message) => {
  const event = JSON.parse(message);
  console.log('Event:', event.type);
});
```

closeConnection(clientId)

Close SSE connection for a client.

Parameters:

- clientId (String)

PDF Service

File: src/services/pdf.service.js

Purpose

Handle PDF file operations (upload, delete, metadata).

Functions

uploadPdf(file, metadata, admin)

Process PDF upload.

Parameters:

- file (Object) - Multer file object
- metadata (Object) - { title, description, category }
- admin (Object) - Admin document

```
const pdf = await uploadPdf(req.files.pdf[0], {
  title: 'Document Title',
  description: 'Description',
  category: 'Technical'
}, admin);
```

deletePdf(pdfId)

Delete PDF and associated files.

Parameters:

- pdfId (ObjectId)

```
await deletePdf('507f1f77bcf86cd799439011');
```

Common Patterns

Error Handling

```
const someService = async (param) => {
  try {
    // Service logic
    return result;
  } catch (error) {
    console.error('[Service] Error:', error);
  }
}
```



```
    throw error; // Re-throw for controller to handle
  }
};
```

Caching Pattern

```
const searchWithCache = async (term) => {
  const cacheKey = `search:${term}`;

  // Check cache
  const cached = await redisClient.get(cacheKey);
  if (cached) {
    return JSON.parse(cached);
  }

  // Query database
  const results = await Model.find({ /* query */ });

  // Cache results
  await redisClient.setEx(cacheKey, 60, JSON.stringify(results));

  return results;
};
```

Async Fire-and-Forget

```
const notifyUser = async (userId, message) => {
  sendNotification(userId, message).catch(err => {
    console.error('Notification failed:', err);
    // Don't throw - this is non-critical
  });
};
```

Testing Services

Unit Test Example

```
const { generateOtp, saveOtp, verifyOtp } = require('../services/otp.service');

describe('OTP Service', () => {
  it('should generate 6-digit OTP', () => {
    const otp = generateOtp();
    expect(otp).toMatch(/^\d{6}$/);
  });

  it('should save and verify OTP', async () => {
    const otp = '123456';
    await saveOtp('+1234567890', otp, 'device-123');

    const result = await verifyOtp(otp, '+1234567890', 'device-123');
    expect(result.valid).toBe(true);
  });

  it('should reject wrong device ID', async () => {
    const otp = '123456';
    await saveOtp('+1234567890', otp, 'device-123');

    const result = await verifyOtp(otp, '+1234567890', 'wrong-device');
    expect(result.valid).toBe(false);
  });
});
```

Service Dependencies

```
graph TD
  Controller --> JWTService
  Controller --> OTPService
  Controller --> SMSService
  Controller --> UserAuditService
  Controller --> SearchService
  Controller --> SSEService
  Controller --> PDFService

  OTPService --> Redis
```

```
SearchService --> Redis
SSEService --> Redis

UserAuditService --> MongoDB
SearchService --> MongoDB
PDFService --> MongoDB

SMSService --> ExternalAPI
```

Best Practices

- Keep services pure - No Express dependencies
- Single responsibility - One service, one concern
- Error handling - Always catch and log errors
- Async/await - Use modern async patterns
- Testability - Easy to unit test in isolation
- Documentation - JSDoc comments for all functions
- Logging - Log important operations
- Caching - Use Redis for expensive operations

CLI Scripts

Complete reference for all command-line scripts in VayuReader Backend v2.

Overview

The scripts/ directory contains operational utilities for database management, user administration, and system maintenance.

Location: scripts/

manageUsers.js

Purpose: Delete, block, or unblock users and admins from the command line.

Usage

```
node scripts/manageUsers.js <type> <action> <identifier_flag> <value>
```

Arguments

Argument	Values	Description	
	user \	admin	Target entity type
	delete \	block \	unblock
	--contact \	--id \	--device
	String	The actual identifier value	

Identifier Flags

- `--contact` - Phone number for users, contact for admins
- `--id` - MongoDB `_id` (ObjectId)
- `--device` - Device ID (users only)

Examples

```
# Block a user by phone number
node scripts/manageUsers.js user block --contact +1234567890

# Unblock a user by ID
node scripts/manageUsers.js user unblock --id 507f1f77bcf86cd799439011

# Delete a user by device ID
node scripts/manageUsers.js user delete --device abc123def456

# Delete an admin by contact
node scripts/manageUsers.js admin delete --contact +9876543210

# Delete an admin by ID
node scripts/manageUsers.js admin delete --id 507f1f77bcf86cd799439012
```

Internals

- Loads environment variables from `.env`
- Connects to MongoDB
- Finds the target document based on identifier
- Performs the requested action:
 - `delete`: `Model.deleteOne({ _id })`
 - `block`: Sets `user.isBlocked = true` (users only)
 - `unblock`: Sets `user.isBlocked = false` (users only)
- Logs result to console
- Disconnects from MongoDB

Notes

- Admin blocking is not supported (use delete instead)
- Blocked users cannot login (enforced in auth middleware)
- Deleted users' tokens become invalid immediately
- Use with caution in production

seedAdmin.js

Purpose: Create the initial super admin account.

Usage

```
node scripts/seedAdmin.js
```

Environment Variables Required

- SUPERADMINNAME - Admin's full name
- SUPERADMINCONTACT - Admin's phone number
- SUPERADMINPASSWORD - Admin's password (will be hashed)

Example

```
# Set environment variables
export SUPER_ADMIN_NAME="Super Admin"
export SUPER_ADMIN_CONTACT="+1234567890"
export SUPER_ADMIN_PASSWORD="SecurePassword123!"

# Run script
node scripts/seedAdmin.js
```

Output

```
Connecting to MongoDB...
Connected to MongoDB
Creating super admin...
Super admin created successfully!
Name: Super Admin
Contact: +1234567890
Disconnected from DB
```

Internals

- Checks if super admin already exists (by contact)

- If exists, exits with message
- Hashes password using bcrypt (10 rounds)
- Creates admin document with:
 - isSuperAdmin: true
 - All permissions
 - createdBy: 'System'
- Saves to database

Notes

- Run once during initial setup
- Password is hashed with bcrypt
- Super admin has all permissions automatically
- Cannot be deleted via API (only via script)

create-super-admin.js

Purpose: Interactive script to create a super admin (alternative to seedAdmin.js).

Usage

```
node scripts/create-super-admin.js
```

Interactive Prompts

```
Enter super admin name: Super Admin
Enter contact number: +1234567890
Enter password: *****
Confirm password: *****
```

Internals

- Uses readline for interactive input
- Validates password match

- Hashes password with bcrypt
- Creates super admin document

Notes

- More user-friendly than seedAdmin.js
 - Useful for manual admin creation
 - Password is hidden during input
-

reset-admin-password.js

Purpose: Reset an admin's password.

Usage

```
node scripts/reset-admin-password.js <contact> <new_password>
```

Arguments

- - Admin's contact number
- - New password (will be hashed)

Example

```
node scripts/reset-admin-password.js +1234567890 NewSecurePassword123!
```

Output

```
Connecting to MongoDB...
Admin found: Super Admin
Password updated successfully!
```

Internals

- Finds admin by contact
- Hashes new password with bcrypt
- Updates passwordHash field
- Saves document

Notes

- Use when admin forgets password
- Password must meet strength requirements
- Admin must re-login after reset

create-user.js

Purpose: Manually create a user account (for testing).

Usage

```
node scripts/create-user.js <name> <phone_number>
```

Arguments

- - User's full name
- - User's phone number

Example

```
node scripts/create-user.js "John Doe" +1234567890
```

Output

```
User created successfully!
```

```
ID: 507f1f77bcf86cd799439011
Name: John Doe
Phone: +1234567890
```

Internals

- Checks if user already exists
- Creates user document
- Saves to database

Notes

- Useful for testing without OTP flow
 - User can login normally via OTP
 - No device binding until first login
-

optimizeDb.js

Purpose: Optimize database performance by creating indexes and analyzing collections.

Usage

```
node scripts/optimizeDb.js
```

What It Does

- Creates Indexes:
 - User: phone_number, deviceId
 - Admin: contact, name
 - PdfDocument: title, category, uploadedBy, createdAt, text index
 - Word: word, text index
 - Abbreviation: abbreviation, text index
 - UserAudit: userId, phone_number, action, timestamp
 - AuditLog: resourceType, resourceId, adminName, timestamp
-

- Analyzes Collections:
 - Document count
 - Average document size
 - Total size
 - Index count
- Provides Recommendations:
 - Slow query detection
 - Missing indexes
 - Large collections

Output

```
Optimizing database...

Creating indexes...
User indexes created
Admin indexes created
PdfDocument indexes created
Word indexes created
Abbreviation indexes created
UserAudit indexes created
AuditLog indexes created

Analyzing collections...
Users: 1,234 documents (avg 512 bytes, total 631 KB)
Admins: 5 documents (avg 256 bytes, total 1.3 KB)
PDFs: 567 documents (avg 1024 bytes, total 581 KB)
...

Recommendations:
- Consider archiving old audit logs (>90 days)
- UserAudit collection is large (10 MB), consider partitioning
```

Internals

- Uses `Model.collection.createIndex()` for each index
- Queries `db.collection.stats()` for analysis
- Calculates recommendations based on thresholds

Notes

- Run after schema changes
 - Run periodically in production (monthly)
 - Safe to run multiple times (idempotent)
-

syncElasticsearch.js

Purpose: Sync MongoDB data to Elasticsearch for advanced search.

Usage

```
node scripts/syncElasticsearch.js [--full]
```

Flags

- --full - Full re-index (deletes existing data)
- (no flag) - Incremental sync (only new/updated documents)

Example

```
# Full re-index
node scripts/syncElasticsearch.js --full

# Incremental sync
node scripts/syncElasticsearch.js
```

What It Syncs

- Dictionary Words:
 - Index: dictionary
 - Fields: word, meanings, synonyms, antonyms
 - Abbreviations:
 - Index: abbreviations
 - Fields: abbreviation, fullForm
 - PDFs:
-

- Index: pdfs
- Fields: title, description, category

Output

```
Connecting to Elasticsearch...
Connected to Elasticsearch

Syncing dictionary words...
Indexed 1,234 words

Syncing abbreviations...
Indexed 567 abbreviations

Syncing PDFs...
Indexed 89 PDFs

Sync completed!
Total documents: 1,890
```

Internals

- Connects to Elasticsearch (URL from .env)
- Creates indexes if they don't exist
- Queries MongoDB for documents
- Bulk indexes to Elasticsearch
- Logs progress and errors

Notes

- Requires Elasticsearch running
- Full re-index can take time for large datasets
- Incremental sync uses updatedAt timestamp
- See docs/ELASTICSEARCH.md for details

cleanupLogs.js

Purpose: Delete old audit logs to free up space.

Usage

```
node scripts/cleanupLogs.js --days <number> [--dry-run]
```

Arguments

- `--days` - Retention period in days (records older than this will be deleted)
- `--dry-run` - Simulate deletion without actually removing data

Example

```
# Delete logs older than 90 days
node scripts/cleanupLogs.js --days 90

# Dry run to see what would be deleted
node scripts/cleanupLogs.js --days 60 --dry-run
```

Output

```
Cleanup Configuration:
- Retention Days: 90
- Cutoff Date: 2023-10-12T10:00:00.000Z
- Mode: LIVE

Found logs older than 90 days:
- UserAudit: 1,500 records
- AuditLog: 250 records

Deleting old UserAudit logs...
Deleted 1500 UserAudit records

Deleting old AuditLog logs...
Deleted 250 AuditLog records

Cleanup completed successfully!
```

Notes

- Targets both UserAudit and AuditLog collections
 - Recommended to run via cron job provided in docs/SCRIPTS.md
 - Backup database before running with short retention periods
-

sms-simulator.js

Purpose: Simulate SMS gateway for development (no actual SMS sent).

Usage

```
node scripts/sms-simulator.js
```

What It Does

- Starts HTTP server on port 8080
- Receives OTP requests from backend
- Logs OTP to console (instead of sending SMS)
- Returns success response

Output

```
SMS Simulator running on port 8080

[2024-01-10 10:00:00] OTP Request
To: +1234567890
Message: Your OTP is: 123456
```

Configuration

Set in .env:

```
OTP_GATEWAY_URL=http://sms-simulator:8080/send
```

```
SKIP_OTP_SEND=false # Set to true to skip entirely
```

Internals

- Express server listening on /send
- Parses request body for phone number and message
- Logs to console
- Returns { success: true }

Notes

- For development only
- Allows testing OTP flow without SMS costs
- Can be replaced with real SMS gateway in production

Common Patterns

Running Scripts in Docker

```
# Execute script in running container
docker-compose exec app node scripts/manageUsers.js user block --contact
+1234567890

# Run script in new container
docker-compose run --rm app node scripts/seedAdmin.js
```

Environment Variables

All scripts load .env automatically:

```
require('dotenv').config();

const MONGODB_URI = process.env.MONGODB_URI;
```


Database Connection

Standard pattern:

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGODB_URI);
    console.log('Connected to MongoDB');
  } catch (error) {
    console.error('DB Connection Error:', error);
    process.exit(1);
  }
};

const run = async () => {
  await connectDB();

  // ... script logic ...

  await mongoose.disconnect();
  process.exit(0);
};

run();
```

Error Handling

```
try {
  const user = await User.findOne({ phone_number });
  if (!user) {
    console.log('User not found');
    process.exit(0);
  }

  // ... perform action ...

  console.log('Success!');
} catch (error) {
  console.error('Error:', error.message);
  process.exit(1);
}
```

Best Practices

- Always use .env - Never hardcode credentials
 - Validate inputs - Check arguments before processing
 - Confirm destructive actions - Prompt user before deleting
 - Log everything - Console output for audit trail
 - Exit cleanly - Always disconnect from DB
 - Use exit codes - 0 for success, 1 for error
 - Handle errors gracefully - Catch and log all errors
-

Troubleshooting

"Cannot connect to MongoDB"

Cause: MongoDB not running or wrong connection string Solution:

```
# Check MongoDB status
docker-compose ps mongodb

# Check connection string in .env
cat .env | grep MONGODB_URI

# Restart MongoDB
docker-compose restart mongodb
```

"User not found"

Cause: Wrong identifier or user doesn't exist Solution:

- Verify phone number format (E.164)
- Check ObjectId is valid
- Query database manually to confirm

"Permission denied"

Cause: Script requires admin privileges Solution:

```
# Run with sudo (if needed)
sudo node scripts/manageUsers.js ...

# Or change file permissions
chmod +x scripts/manageUsers.js
```

Creating New Scripts

Template

```
#!/usr/bin/env node

/**
 * Script Name
 *
 * Description of what the script does.
 *
 * Usage: node scripts/script-name.js <args>
 */

require('dotenv').config();
const mongoose = require('mongoose');

// Import models
const User = require('../src/models/User');

// Parse arguments
const args = process.argv.slice(2);

const printUsage = () => {
  console.log('Usage: node scripts/script-name.js <arg1> <arg2>');
  process.exit(1);
};

if (args.length < 2) printUsage();
```

```
const [arg1, arg2] = args;

// Database connection
const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGODB_URI);
    console.log('Connected to MongoDB');
  } catch (error) {
    console.error('DB Connection Error:', error);
    process.exit(1);
  }
};

// Main logic
const run = async () => {
  await connectDB();

  try {
    // ... your logic here ...

    console.log('Success!');
  } catch (error) {
    console.error('Error:', error.message);
  } finally {
    await mongoose.disconnect();
    console.log('Disconnected from DB');
    process.exit(0);
  }
};

// Execute
run();
```

Adding to package.json

```
{
  "scripts": {
    "script:name": "node scripts/script-name.js"
  }
}
```

Usage:

```
npm run script:name -- arg1 arg2
```

Security Considerations

- Never log sensitive data - Passwords, tokens, etc.
 - Validate all inputs - Prevent injection attacks
 - Use parameterized queries - Avoid NoSQL injection
 - Limit script access - Only authorized users
 - Audit script usage - Log who ran what and when
 - Encrypt sensitive files - If storing credentials
-

Automation

Cron Jobs

```
# Run database optimization weekly
0 2 * * 0 cd /app && node scripts/optimizeDb.js >> /var/log/optimize.log 2>&1

# Sync Elasticsearch daily
0 3 * * * cd /app && node scripts/syncElasticsearch.js >> /var/log/sync.log 2>&1
```

CI/CD Integration

```
# .github/workflows/deploy.yml
- name: Seed admin
  run: node scripts/seedAdmin.js
  env:
    SUPER_ADMIN_NAME: ${ secrets.ADMIN_NAME }
    SUPER_ADMIN_CONTACT: ${ secrets.ADMIN_CONTACT }
    SUPER_ADMIN_PASSWORD: ${ secrets.ADMIN_PASSWORD }
```

Script Inventory

Script	Purpose	Frequency	Destructive
manageUsers.js	User/admin management	As needed	Yes
seedAdmin.js	Create super admin	Once (setup)	No
create-super-admin.js	Interactive admin creation	As needed	No
reset-admin-password.js	Reset admin password	As needed	Modifies
create-user.js	Create test user	As needed	No
optimizeDb.js	Database optimization	Weekly	No
syncElasticsearch.js	Elasticsearch sync	Daily	Full re-index
cleanupLogs.js	Delete old logs	Monthly	Yes
sms-simulator.js	SMS gateway simulator	Dev only	No

Server-Sent Events

Real-time notification architecture for VayuReader Backend v2.

Overview

The backend uses Server-Sent Events (SSE) to push real-time updates to connected clients. This eliminates the need for polling and ensures instant UI refreshes when content changes.

Endpoint: GET /api/events

Architecture

- Redis Pub/Sub: Services publish events to Redis channels.
- SSE Router/Controller: Subscribes to Redis and forwards messages to open HTTP connections.
- Nginx: Proxies the long-lived connection without buffering.

Why Redis Pub/Sub?

Using Redis allows the application to scale horizontally in the future. Multiple backend instances can broadcast events to all connected clients regardless of which instance holds the client connection.

Event Types

Event Name	Payload Description	Trigger
------------	---------------------	---------

connected	Connection status message	Client connects
PDF_ADDED	{ pdfld, title, category }	Admin uploads PDF
PDF_UPDATED	{ pdfld, title }	Admin updates PDF
PDF_DELETED	{ pdfld }	Admin deletes PDF

Note: Future events for dictionary/abbreviations updates can be added easily.

Client Integration

JavaScript Example

```
const connectSSE = (token) => {
  const eventSource = new EventSource('/api/events', {
    headers: { 'Authorization': `Bearer ${token}` }
  });

  eventSource.onopen = () => {
    console.log('SSE Connected');
  };

  eventSource.addEventListener('PDF_ADDED', (event) => {
    const data = JSON.parse(event.data);
    // Refresh PDF list or showing toast notification
    showToast(`New PDF: ${data.title}`);
  });

  eventSource.onerror = (err) => {
    console.error('SSE Error', err);
    eventSource.close();
    // Implement reconnection logic here (exponential backoff)
    setTimeout(() => connectSSE(token), 5000);
  };
};
```

Connection Handling

- Authorization: Client must send a valid JWT token (usually handled via cookie or header if the

EventSource polyfill supports it).

- Reconnection: Browsers automatically attempt to reconnect native EventSource. Custom logic might be needed for error handling.
 - Heartbeat: The server sends a heartbeat comment (: keepalive) every 30 seconds to prevent timeouts from load balancers or firewalls.
-

Server Configuration

Nginx: Explicitly disables buffering for this endpoint (see NGINX.md).

```
location /api/events {  
    proxy_buffering off;  
    proxy_cache off;  
    proxy_read_timeout 24h;  
}
```

Express: Headers for SSE:

```
res.setHeader('Content-Type', 'text/event-stream');  
res.setHeader('Cache-Control', 'no-cache');  
res.setHeader('Connection', 'keep-alive');
```

Scalability Considerations

- Connection Limits: Each SSE connection consumes an open socket. Nginx and Node.js limits must be tuned for high concurrency (e.g., 10k+ users).
 - Memory: Minimal memory per connection, but aggregate usage grows with count.
 - Alternatives: For millions of connections, consider a dedicated push service/gateway (e.g., Pusher, Firebase) or specific Websocket microservice.
-

SSE Integration Guide

Real-time PDF metadata updates via Server-Sent Events and Redis Pub/Sub.

Architecture

```
Admin → PDF Controller → Redis Pub/Sub → SSE Controller → Connected Clients
```

Why this approach:

- Redis Pub/Sub: Supports horizontal scaling - multiple server instances can share events
- SSE over WebSockets: Simpler, automatic reconnection, works over standard HTTP

Endpoint

```
GET /api/events
Authorization: Bearer <token>
Accept: text/event-stream
```

Authentication required: Uses the same unified auth as other endpoints.

Event Types

Event	Trigger	Payload Fields
connected	Initial connection	message
PDF_ADDED	New PDF uploaded	id, title, category, created
PDF_UPDATED	PDF modified	id, title, category, updated

PDF_DELETED	PDF removed	id, title, timestamp
-------------	-------------	----------------------

Example event:

```
event: PDF_ADDED
data: {"id":"507f1f77bcf86cd799439011","title":"Flight Manual v2","category":"Operations","createdAt":"2026-01-07T16:00:00.000Z","timestamp":"2026-01-07T16:00:00.123Z"}
```

Client Integration

JavaScript (Browser/Node.js)

```
const token = 'your-jwt-token';
const eventSource = new EventSource(`/api/events?token=${token}`);

// Connection established
eventSource.addEventListener('connected', (e) => {
  console.log('Connected to events stream');
});

// New PDF uploaded
eventSource.addEventListener('PDF_ADDED', (e) => {
  const pdf = JSON.parse(e.data);
  console.log('New PDF:', pdf.title);
  // Refresh your PDF list or add to local state
});

// PDF updated
eventSource.addEventListener('PDF_UPDATED', (e) => {
  const pdf = JSON.parse(e.data);
  console.log('PDF updated:', pdf.id);
});

// PDF deleted
eventSource.addEventListener('PDF_DELETED', (e) => {
  const pdf = JSON.parse(e.data);
```

```
    console.log('PDF deleted:', pdf.id);
    // Remove from local state
  });

// Handle errors
eventSource.onerror = (error) => {
  console.error('SSE error:', error);
  // EventSource auto-reconnects, but you may want to handle UI state
};
```

React Hook

```
import { useEffect, useCallback } from 'react';

export function usePdfEvents(token, onPdfChange) {
  useEffect(() => {
    const eventSource = new EventSource(`/api/events?token=${token}`);

    eventSource.addEventListener('PDF_ADDED', (e) => {
      onPdfChange?.({ type: 'added', data: JSON.parse(e.data) });
    });

    eventSource.addEventListener('PDF_UPDATED', (e) => {
      onPdfChange?.({ type: 'updated', data: JSON.parse(e.data) });
    });

    eventSource.addEventListener('PDF_DELETED', (e) => {
      onPdfChange?.({ type: 'deleted', data: JSON.parse(e.data) });
    });

    return () => eventSource.close();
  }, [token, onPdfChange]);
}

// Usage
function PdfList() {
  const handlePdfChange = useCallback((event) => {
    if (event.type === 'added') {
      // Add to list
    } else if (event.type === 'deleted') {
      // Remove from list
    }
  }, []);

  usePdfEvents(authToken, handlePdfChange);
}
```

```
}
```

React Native / Expo

Use the react-native-sse package:

```
import EventSource from 'react-native-sse';

const es = new EventSource(`${API_BASE}/api/events`, {
  headers: { Authorization: `Bearer ${token}` }
});

es.addEventListener('PDF_ADDED', (e) => {
  const pdf = JSON.parse(e.data);
  // Update state
});
```

Heartbeat

The server sends a heartbeat comment every 30 seconds to keep the connection alive:

```
:heartbeat
```

Clients should ignore lines starting with :.

Scaling Considerations

- Multiple server instances: Redis Pub/Sub ensures all instances receive events
- Connection limits: Each SSE connection uses a dedicated Redis subscriber
- Reconnection: Browsers auto-reconnect; mobile clients may need manual handling
- Nginx configuration: proxy_buffering off is critical for real-time delivery

Testing

curl

```
curl -N -H "Authorization: Bearer <token>" \  
  -H "Accept: text/event-stream" \  
  https://localhost/api/events
```

Simple HTML test page

```
<!DOCTYPE html>  
<html>  
<body>  
<h1>SSE Test</h1>  
<pre id="log"></pre>  
<script>  
  const token = 'YOUR_TOKEN';  
  const es = new EventSource(`/api/events?token=${token}`);  
  const log = document.getElementById('log');  
  
  es.onmessage = (e) => log.textContent += e.data + '\n';  
  ['connected', 'PDF_ADDED', 'PDF_UPDATED', 'PDF_DELETED'].forEach(evt => {  
    es.addEventListener(evt, (e) => {  
      log.textContent += `${evt}: ${e.data}\n`;  
    });  
  });  
</script>  
</body>  
</html>
```

Audit System

Comprehensive guide to the auditing system in VayuReader Backend v2.

Overview

The auditing system provides accountability and analytics by tracking:

- Admin Actions (AuditLog): Changes to data (create, update, delete).
 - User Activity (UserAudit): Logins, device changes, and content consumption.
-

1. Admin Audit Logs

Model: src/models/AuditLog.js Storage: MongoDB collection auditlogs

What is Logged

- Resource Changes: Modifications to PDFs, Dictionary Words, and Abbreviations.
- Admin Management: Creation and deletion of sub-admins.

Log Structure

```
{
  action: "UPDATE",
  resourceType: "DICTIONARY_WORD",
  resourceId: "507f1f...",
  adminName: "Super Admin",
  details: {
    oldMeanings: [...],
    newMeanings: [...]
  },
  timestamp: ISODate("...")
}
```

Accessing Logs

Admins with `view_audit` permission can query logs via API:

- GET `/api/audit/logs`: Filter by type, date, or admin.
- GET `/api/audit/stats`: Aggregate statistics (counts by type).

Retention Policy

Currently, logs are retained indefinitely. For production, a retention policy (e.g., 1 year) should be implemented using a TTL index or cron job.

2. User Audit Logs

Model: `src/models/UserAudit.js` Storage: MongoDB collection `useraudits`

What is Logged

- LOGIN: Successful authentication via OTP.
- DEVICE_CHANGE: Login from a new device ID (security critical).
- READ_PDF: User viewed a document.

Device Tracking

Device changes are critical for security monitoring.

```
{
  action: "DEVICE_CHANGE",
  userId: "...",
  phone_number: "+91...",
  deviceId: "new-device-id",
  metadata: {
    previousDeviceId: "old-device-id"
  }
}
```


Analytics

User audit logs power internal dashboards to show:

- Daily Active Users (DAU)
 - Most popular documents
 - User retention rates
-

Logging Implementation

Service: `src/services/userAudit.service.js`

Logging is asynchronous and non-blocking. Failures to log do not block the main request flow (fire-and-forget).

```
// Example: Logging a login
const logLogin = async (user, deviceId) => {
  try {
    await UserAudit.create({ /* ... */ });
  } catch (err) {
    console.error('Audit log failed:', err);
  }
};
```

Best Practices

- Denormalization: Important fields (user phone, admin name) are denormalized into logs to ensure history is preserved even if the original user/admin is deleted.
 - Immutability: Audit logs should never be updated or deleted via API.
 - Indexing: Timestamps and Action types are indexed for fast retrieval.
-

Part VII: Operations

Security Hardening

VayuReader Backend v2

Version: 2.0 Date: January 2026 Classification: RESTRICTED - For Authorized Personnel Only

1. Overview

This document provides security hardening guidelines for deploying VayuReader Backend v2 in production environments. It covers configuration best practices, security checklists, and compliance considerations.

2. Pre-Deployment Security Checklist

2.1 Environment Configuration

- ☐ JWT_SECRET is strong (minimum 32 characters, cryptographically random)
- ☐ JWT_SECRET is not committed to version control
- ☐ MongoDB authentication enabled (username/password configured)
- ☐ Redis password protected (if exposed to network)
- ☐ SKIPOTPSSEND=false in production
- ☐ NODE_ENV=production set
- ☐ Specific ALLOWED_ORIGINS configured (no wildcards)

2.2 Network Security

- ☐ HTTPS only (HTTP redirects to HTTPS)
- ☐ TLS 1.2+ enforced (TLS 1.0/1.1 disabled)
- ☐ Strong cipher suites configured in Nginx
- ☐ Internal services not exposed (MongoDB, Redis, Node.js ports closed)
- ☐ Firewall configured (only ports 80, 443 open to public)

2.3 Application Security

- ☐ Rate limiting enabled on all endpoints
- ☐ Helmet security headers enabled
- ☐ CORS properly configured with specific origins
- ☐ Cookie flags set (HttpOnly, Secure, SameSite)
- ☐ Input validation on all endpoints
- ☐ NoSQL injection protection enabled

2.4 Operational Security

- ☐ Audit logging enabled for all admin actions
- ☐ User activity logging enabled
- ☐ Log rotation configured
- ☐ Backup strategy implemented
- ☐ SSL certificate renewal automated

3. Secure Configuration

3.1 Nginx SSL Configuration

File: nginx/conf.d/default.conf

```
# Enforce HTTPS
server {
    listen 80;
    listen [::]:80;
    server_name _;
    return 301 https://$host$request_uri;
}
```

```

server {
    listen 443 ssl http2;
    listen [::]:443 ssl http2;
    server_name your-domain.com;

    # SSL Certificate (use certbot for Let's Encrypt)
    ssl_certificate /etc/nginx/certs/fullchain.pem;
    ssl_certificate_key /etc/nginx/certs/privkey.pem;

    # Modern SSL Configuration
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_prefer_server_ciphers on;
    ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-
ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384;

    # SSL Session Caching
    ssl_session_cache shared:SSL:10m;
    ssl_session_timeout 1d;
    ssl_session_tickets off;

    # OCSP Stapling
    ssl_stapling on;
    ssl_stapling_verify on;
    resolver 8.8.8.8 8.8.4.4 valid=300s;

    # Security Headers
    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains;
preload" always;
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-XSS-Protection "1; mode=block" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;
    add_header Content-Security-Policy "default-src 'self'; script-src 'self';
style-src 'self' 'unsafe-inline'; img-src 'self' data: https:; font-src 'self';
connect-src 'self'; frame-ancestors 'self';" always;

    # ... rest of configuration
}

```

3.2 MongoDB Security

Docker Compose Configuration:

```
mongo:
```

```
image: mongo:6
container_name: vayureader_db
environment:
  - MONGO_INITDB_ROOT_USERNAME=admin
  - MONGO_INITDB_ROOT_PASSWORD=${MONGO_PASSWORD}
volumes:
  - mongo-data:/data/db
networks:
  - backend-network
restart: unless-stopped
# No ports exposed to host
```

Application Connection:

```
MONGODB_URI=mongodb://admin:${MONGO_PASSWORD}@mongo:27017/vayureader?authSource=admin
```

3.3 Redis Security

Docker Compose Configuration:

```
redis:
  image: redis:7-alpine
  container_name: vayureader_cache
  command: redis-server --appendonly yes --requirepass ${REDIS_PASSWORD}
  volumes:
    - redis-data:/data
  networks:
    - backend-network
  restart: unless-stopped
  # No ports exposed to host
```

Application Connection:

```
REDIS_URL=redis://:${REDIS_PASSWORD}@redis:6379
```

3.4 Cookie Security

Production Cookie Configuration:

```
// User token cookie
res.cookie('auth_token', token, {
```

```
    httpOnly: true,      // Prevents JavaScript access (XSS protection)
    secure: true,        // HTTPS only
    sameSite: 'strict', // CSRF protection
    maxAge: 100 * 365 * 24 * 60 * 60 * 1000, // 100 years
    path: '/',
    domain: '.yourdomain.com' // Set for cross-subdomain access if needed
  });

// Admin token cookie
res.cookie('admin_token', token, {
  httpOnly: true,
  secure: true,
  sameSite: 'strict',
  maxAge: 24 * 60 * 60 * 1000, // 1 day
  path: '/',
  domain: '.yourdomain.com'
});
```

4. Password Policy

4.1 Admin Password Requirements

Enforced in src/middleware/validation.js:

```
const passwordPolicy = {
  minLength: 8,
  maxLength: 128,
  requireUppercase: true,
  requireLowercase: true,
  requireNumbers: true,
  requireSpecialChars: true,
  pattern:
    /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,}$/
};

const validatePassword = (password) => {
  if (password.length < 8) {
    return { valid: false, error: 'Password must be at least 8 characters' }
  }
};
```

```
    }
    if (password.length > 128) {
      return { valid: false, error: 'Password must not exceed 128 characters'
    };
  }
  if (!passwordPolicy.pattern.test(password)) {
    return {
      valid: false,
      error: 'Password must contain uppercase, lowercase, number, and
special character'
    };
  }
  return { valid: true };
};
```

4.2 Password Hashing

- Algorithm: bcrypt
- Salt Rounds: 10 (recommended minimum)
- Example:

```
const bcrypt = require('bcryptjs');
const SALT_ROUNDS = 10;

const hashPassword = async (password) => {
  return await bcrypt.hash(password, SALT_ROUNDS);
};

const verifyPassword = async (password, hash) => {
  return await bcrypt.compare(password, hash);
};
```

5. Rate Limiting Strategy

5.1 Endpoint-Specific Limits

Endpoint Category	Rate Limit	Window	Purpose
OTP Request	5 requests	15 minutes	Prevent SMS abuse
Login Verification	5 requests	1 minute	Prevent brute force
Admin Creation	5 requests	1 hour	Prevent account spam
Search API	60 requests	1 minute	Protect database
File Upload	10 requests	1 hour	Prevent storage abuse
General API	100 requests	15 minutes	Fair usage

5.2 Account Lockout

After 5 failed login attempts within 15 minutes:

```
const handleLoginAttempt = async (contact, success) => {
  const key = `login_attempts:${contact}`;

  if (success) {
    await redisClient.del(key);
    return;
  }

  const attempts = await redisClient.incr(key);
  await redisClient.expire(key, 15 * 60); // 15 minutes

  if (attempts >= 5) {
    // Log security event
    console.warn(`Account lockout triggered for: ${contact}`);
    // Optionally notify admin
  }
};
```

6. Input Validation & Sanitization

6.1 NoSQL Injection Prevention

Middleware: express-mongo-sanitize

```
const mongoSanitize = require('express-mongo-sanitize');

// Remove $ and . from request to prevent NoSQL injection
app.use(mongoSanitize({
  replaceWith: '_',
  onSanitize: ({ req, key }) => {
    console.warn(`NoSQL Injection attempt blocked: ${key}`);
  }
}));
```

6.2 ReDoS Prevention

Custom regex escaping:

```
const escapeRegex = (string) => {
  return string.replace(/[\.\*\+\?\^\$\{\}\(\)\[\]\|\]/g, '\\$&');
};

// Safe usage
const safePattern = new RegExp(`^${escapeRegex(userInput)}$, 'i');
```

6.3 File Upload Validation

```
const allowedMimeTypes = {
  pdf: ['application/pdf'],
  image: ['image/jpeg', 'image/png', 'image/webp']
};

const validateUpload = (file, type) => {
  const allowed = allowedMimeTypes[type] || [];

  if (!allowed.includes(file.mimetype)) {
    return { valid: false, error: 'Invalid file type' };
  }

  // Check magic bytes (file signature)
  const signatures = {
    'application/pdf': [0x25, 0x50, 0x44, 0x46], // %PDF
    'image/jpeg': [0xFF, 0xD8, 0xFF],
    'image/png': [0x89, 0x50, 0x4E, 0x47]
  };

  const fileBuffer = fs.readFileSync(file.path);
```

```
const signature = signatures[file.mimetype];

if (signature) {
  const matches = signature.every((byte, i) => fileBuffer[i] === byte);
  if (!matches) {
    return { valid: false, error: 'File signature mismatch' };
  }
}

return { valid: true };
};
```

7. Audit Logging Requirements

7.1 Events to Log

User Events:

- Successful login
- Failed login attempts
- Device changes
- PDF access
- Password changes
- All CRUD operations
- Permission changes
- User management actions
- Bulk operations

7.2 Log Format

```
{
  timestamp: "2026-01-19T10:00:00.000Z",
  action: "CREATE",
  resourceType: "PDF",
  resourceId: "507f1f77bcf86cd799439011",
  adminId: "507f1f77bcf86cd799439012",
  adminName: "Admin User",
```

```
adminContact: "+919876543210",
ip: "192.168.1.100",
userAgent: "Mozilla/5.0...",
details: {
  title: "New Document",
  category: "Operations"
}
}
```

7.3 Log Retention

- User Activity Logs: 1 year minimum
 - Admin Audit Logs: 3 years minimum (compliance)
 - Security Events: 5 years (investigations)
-

8. Security Headers

8.1 Helmet Configuration

```
const helmet = require('helmet');

app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ["'self'"],
      scriptSrc: ["'self'"],
      styleSrc: ["'self'", "unsafe-inline"],
      imgSrc: ["'self'", "data:", "https:"],
      fontSrc: ["'self'"],
      connectSrc: ["'self'"],
      frameSrc: ["'none'"],
      objectSrc: ["'none'"],
      upgradeInsecureRequests: []
    }
  },
  crossOriginEmbedderPolicy: false,
  crossOriginResourcePolicy: { policy: "cross-origin" },
  hsts: {
```

```
    maxAge: 31536000,  
    includeSubDomains: true,  
    preload: true  
  }  
}));
```

8.2 Expected Response Headers

```
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload  
X-Content-Type-Options: nosniff  
X-Frame-Options: SAMEORIGIN  
X-XSS-Protection: 1; mode=block  
Referrer-Policy: strict-origin-when-cross-origin  
X-DNS-Prefetch-Control: off  
X-Download-Options: noopen  
X-Permitted-Cross-Domain-Policies: none
```

9. Incident Response

9.1 Security Incident Types

Type	Severity	Response Time
Suspected breach	Critical	Immediate
Account compromise	High	1 hour
DDoS attack	High	15 minutes
Rate limit abuse	Medium	4 hours
Failed login spike	Low	24 hours

9.2 Response Procedures

Account Compromise:

- Block affected user immediately

- Invalidate all tokens (rotate JWT secret if widespread)
 - Notify user via alternative channel
 - Review audit logs
 - Document incident
 - Enable additional rate limiting
 - Block suspicious IPs at firewall
 - Scale up resources if needed
 - Enable CDN protection (CloudFlare)
 - Document attack patterns
-

10. Periodic Security Tasks

10.1 Daily

- ☐ Review failed login attempts
- ☐ Check container health
- ☐ Monitor rate limit violations

10.2 Weekly

- ☐ Review audit logs for anomalies
- ☐ Check SSL certificate expiration
- ☐ Verify backup completion

10.3 Monthly

- ☐ Rotate database credentials
- ☐ Review user access patterns
- ☐ Update dependencies for security patches
- ☐ Run vulnerability scan

10.4 Quarterly

- ☐ Rotate JWT secret
 - ☐ Review and update firewall rules
-

- ☐ Conduct security audit
 - ☐ Update documentation
-

11. Compliance Considerations

11.1 Data Protection

- All data encrypted in transit (TLS 1.2+)
- Sensitive data (passwords, OTPs) encrypted at rest
- PII minimized and access controlled
- Audit trail maintained for all access

11.2 Access Control

- Role-based access control (RBAC)
 - Principle of least privilege
 - Multi-factor authentication for admins
 - Session timeout enforcement
-

Document Revision History

Version	Date	Author	Changes
1.0	Jan 2026	Dev Team	Initial security guide

Performance Optimization

This document outlines the performance optimizations implemented for VayuReader Backend v2 to handle heavy read-intensive loads (200K+ users at peak).

Architecture Overview



Optimizations Implemented

1. Redis Caching (Application Layer)

Dictionary Lookups:

- Cache Key: word:
- TTL: 24 hours
- Impact: ~95% cache hit rate for common words
- Cache Key: abbr:
- TTL: 24 hours
- Impact: ~90% cache hit rate
- Cache Key: abbr:search: / word:search:
- TTL: 30 minutes
- Impact: Reduces DB load for repeated searches
- Cache Key: pdf:categories
- TTL: 1 hour
- Impact: Single DB query per hour instead of per request

2. Nginx Optimizations (Gateway Layer)

Static File Serving:

```
# PDFs served directly by Nginx (bypass Node.js)
location ~ ^/uploads/.*\.pdf$ {
    expires 7d;
    add_header Cache-Control "public, immutable";
    limit_rate 2m;          # Rate limit per connection
    limit_rate_after 5m;    # Start rate limiting after 5MB
}
```

Proxy Caching:

```
# Cache dictionary/abbreviation API responses at Nginx level
proxy_cache api_cache;
proxy_cache_valid 200 5m;
```

Connection Optimizations:

- HTTP/2 enabled
- Keepalive connections to backend (32 persistent)
- SSL session caching
- Sendfile + TCP optimizations

3. MongoDB Optimizations (Database Layer)

Indexes Created:

```
// Words collection
{ word: 1 } // Unique, for lookups
{ word: 'text' } // For text search

// Abbreviations collection
{ abbreviation: 1 } // Unique, for lookups
{ abbreviation: 'text', fullForm: 'text' } // For search

// PDF Documents collection
{ category: 1 } // For category filtering
{ createdAt: -1 } // For sorting by date
{ title: 'text', content: 'text' } // For search
{ viewCount: -1 } // For popular PDFs
```

Query Optimizations:

- `.lean()` used on all read queries (30-40% faster)
- Projection (`.select()`) to limit returned fields
- `Promise.all()` for parallel query execution

4. Node.js Optimizations

Cluster Mode:

```
npm run start:cluster
# Spawns 1 worker per CPU core
```

Compression:

- gzip compression enabled for all API responses
- Reduces bandwidth by ~70%

Performance Metrics

Metric	Before	After	Improvement
Dictionary Lookup (cold)	50ms	15ms	70%
Dictionary Lookup (cached)	50ms	2ms	96%
Abbreviation Lookup	30ms	3ms	90%
PDF List (100 items)	120ms	40ms	67%
PDF Download throughput	100 concurrent	500+ concurrent	5x

Recommended Server Specs for 200K Peak Users

Minimum Production Setup:

- CPU: 4+ cores
- RAM: 8GB (4GB for Node, 2GB Redis, 2GB MongoDB)
- Storage: SSD with 100GB+ for PDFs
- Network: 1 Gbps

Recommended Production Setup:

- CPU: 8+ cores
- RAM: 16GB+
- Storage: NVMe SSD
- Network: 10 Gbps or CDN for static files

Scaling Strategies

Horizontal Scaling (Recommended):

- Run multiple containers behind a load balancer
- Use external Redis cluster (Redis Cluster or AWS ElastiCache)
- Use MongoDB replica set for read scaling

CDN for PDFs (Highly Recommended):

For 200K users downloading PDFs, consider:

- CloudFlare (free tier available)
- AWS CloudFront
- Azure CDN

Monitoring & Alerting

Key Metrics to Monitor:

- Redis hit rate (target: >90%)
- MongoDB query times (target: <50ms p95)
- Node.js event loop lag (target: <100ms)
- Memory usage (target: <80%)

Recommended Tools:

- PM2 for process management and monitoring
- Prometheus + Grafana for metrics
- MongoDB Atlas built-in monitoring

Periodic Maintenance

Run weekly:

```
npm run db:optimize
```

This script:

- Verifies all indexes exist
- Compacts collections
- Reports usage statistics

Cache Invalidation

Cache is automatically invalidated when:

- Admin creates/updates/deletes words
- Admin creates/updates/deletes abbreviations
- Bulk uploads are performed

Troubleshooting

High latency during peak:

- Check Redis connection pool
- Verify MongoDB indexes with db:optimize
- Consider increasing Nginx proxy cache size

Memory growing unbounded:

- Check for connection leaks
- Verify Redis is evicting properly
- Consider adding maxmemory-policy allkeys-lru to Redis config

PDFs downloading slowly:

- Check Nginx limit_rate settings
- Verify sendfile is enabled
- Consider adding a CDN

Deployment Guide

Instructions for deploying VayuReader Backend v2 to production.

Prerequisites

- Docker Engine & Docker Compose
 - Domain name (for SSL)
 - Access to SMS Gateway (if using real SMS)
-

1. Environment Setup

Create a `.env.production` file (do NOT commit this to Git). Reference `CONFIGURATION.md` for all variables.

Critical Production Variables:

```
NODE_ENV=production
MONGODB_URI=mongodb://user:password@mongo-host:27017/vayureader?authSource=admin
JWT_SECRET=<complex_random_string_min_32_chars>
REDIS_URL=redis://redis-host:6379
OTP_GATEWAY_URL=https://sms-provider.com/send
ALLOWED_ORIGINS=https://admin.yourdomain.com
SKIP_OTP_SEND=false
```

2. Docker Deployment

Production Compose File

Use `docker-compose.prod.yml` (ensure it exists or create based on standard compose).

```
# Build and Run
docker-compose -f docker-compose.prod.yml up --build -d
```

Improvements for Production

- Multi-stage Build: Ensure Dockerfile uses a multi-stage process to discard dev dependencies and limit image size.
 - Restart Policies: Set `restart: always` for all services.
 - Logging: Configure a logging driver (e.g., `json-file` with `max-size`).
-

3. SSL/TLS Setup

- Generate Certificates: Use Certbot (Let's Encrypt).

```
certbot certonly --webroot -w /var/www/html -d api.yourdomain.com
```

- Mount Certificates: Ensure Nginx volume mounts point to the generated certs.
 - Update Nginx Config: Verify `ssl_certificate` paths in `default.conf`.
-

4. Production Checklist

- ☐ Security:
 - ☐ Change all default passwords (Mongo, Redis).
 - ☐ firewall rules (only 80/443 open to public).
 - ☐ `JWT_SECRET` is strong.
 - ☐ Data:
 - ☐ Persistent volumes configured for MongoDB and Uploads.
 - ☐ Backup strategy in place for DB and Uploads.
 - ☐ Performance:
 - ☐ Indexes ensured (`npm run optimize-db`).
-

- ☐ Nginx caching enabled.
 - ☐ Monitoring:
 - ☐ Logs are being collected/rotated.
 - ☐ Uptime monitoring configured (ping endpoint).
-

5. Updates & Rollbacks

Zero-Downtime Deployment:

- Generally requires orchestration (Kubernetes/Swarm) or blue-green deployment with a load balancer.
- With docker-compose, there will be brief downtime during container restart.

```
git pull origin main
docker-compose -f docker-compose.prod.yml build
docker-compose -f docker-compose.prod.yml up -d
```


Testing

Guidelines for testing VayuReader Backend v2.

Testing Frameworks

- Jest: Unit and Integration testing runner.
- Supertest: HTTP assertions for integration tests.

Running Tests

1. Run All Tests

```
npm test
```

2. Run with Coverage

```
npm run test:coverage
```

3. Run Specific File

```
npx jest src/tests/auth.test.js
```

4. Watch Mode

```
npm run test:watch
```

Test Storage

Tests are located in `src/tests/`.

Writing Tests

Unit Tests

Focus on testing individual functions in isolation (e.g., Services).

- Mock dependencies (e.g., database, Redis).
- Test edge cases.

```
describe('OTP Service', () => {
  it('should generate valid 6-digit OTP', () => {
    const otp = generateOtp();
    expect(otp).toMatch(/^[0-9]{6}$/);
  });
});
```

Integration Tests

Focus on API endpoints.

- Use a test database.
- Test the full flow (Request -> Controller -> Service -> DB -> Response).
- Setup/Teardown: Connect/disconnect DB before/after tests.

```
const request = require('supertest');
const app = require('../app');

describe('POST /api/auth/login', () => {
  it('should return 400 for invalid phone', async () => {
    const res = await request(app)
      .post('/api/auth/login/request-otp')
      .send({ phone_number: 'invalid' });
    expect(res.statusCode).toEqual(400);
  });
});
```

Test Database

Use `cross-env` to set `NODEENV=test`. Ensure `MONGODBURI` in `.env.test` points to a separate database (e.g., `vayureader_test`) to avoid wiping development data.

Continuous Integration

Tests are automatically run in the CI pipeline (e.g., GitHub Actions) on every Pull Request.

Contributing Guidelines

Guidelines for contributing to VayuReader Backend v2.

Development Workflow

- Fork & Clone: Clone the repository to your local machine.
 - Branch: Create a feature branch (git checkout -b feature/my-feature).
 - Code: Implement your changes.
 - Test: Run npm test to ensure no regressions.
 - Lint: Run npm run lint to fix style issues.
 - Commit: Use descriptive commit messages.
 - Push: Push to your fork and submit a Pull Request.
-

Coding Standards

Style Guide

We use ESLint with Prettier.

- Indentation: 4 spaces.
- Semicolons: Always.
- Quotes: Single quotes.

```
npm run lint
```

Fix Auto-fixable Issues:

```
npm run lint:fix
```

Naming Conventions

- Variables/Functions: camelCase (getUser, isValid).
 - Classes/Models: PascalCase (User, PdfDocument).
 - Constants: UPPERSNAKECASE (MAXRETRIES, JWTSECRET).
 - Filenames: camelCase (auth.controller.js, user.model.js).
-

Commit Messages

Follow the Conventional Commits specification:

- feat: add user blocking
 - fix: resolve null pointer in pdf service
 - docs: update API endpoints
 - chore: update dependencies
 - refactor: optimize search query
-

Pull Request Process

- Description: Clearly explain what the PR does.
 - Related Issue: Link to the issue ID (e.g., Fixes #123).
 - Screenshots: If applicable (e.g., Admin Dashboard changes).
 - Review: Wait for at least one code review approval.
 - Merge: Squash and merge is preferred.
-

Project Structure

- src/config - Configuration & env vars.
 - src/controllers - Request handlers.
 - src/middleware - Express middleware (Auth, Validation).
 - src/models - Mongoose schemas.
-

- `src/routes` - API route definitions.
- `src/services` - Business logic (Pure functions).
- `scripts` - Operational scripts.
- `docs` - Documentation.